Tina4 for Ruby Developers

This is not a framework

Andre van Zuydam

v3.1.1 - 2026

Table of Contents

1. Chapter 1: Getting Started with Tina4 Ruby
2. Chapter 2: Routing
3. Chapter 3: Request & Response
4. Chapter 4: Templates
5. Chapter 5: Database
6. Chapter 6: ORM
7. Chapter 7: Authentication
8. Chapter 8: Middleware
9. Chapter 9: Sessions & Cookies
10. Chapter 10: API Documentation with Swagger
11. Chapter 11: Queue System
12. Chapter 12: Real-time with WebSocket
13. Chapter 13: Email with Messenger
14. Chapter 14: Caching
15. Chapter 15: Frontend with tina4css
16. Chapter 16: GraphQL
17. Chapter 17: Testing
18. Chapter 18: Dev Tools
19. Chapter 19: CLI & Scaffolding
20. Chapter 20: Deployment
21. Chapter 21: Building a Complete App
22. Chapter 22: Vibe Coding with AI

Chapter 1: Getting Started with Tina4 Ruby

1. What Is Tina4 Ruby

Tina4 Ruby is a zero-dependency web framework for Ruby 3.1+ that gives you routing, an ORM, a template engine, authentication, queues, WebSocket, and 70 other features in a single gem under 5,000 lines of code.

It is part of the Tina4 family -- four identical frameworks in Python, PHP, Ruby, and Node.js. Everything you learn here transfers directly to any of the other three languages. The project structure is the same. The template syntax is the same. The CLI commands are the same. The `.env` variables are the same.

Tina4 Ruby uses `snake_case` for method names (`fetch_one`, `soft_delete`, `has_many`) following Ruby community conventions. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a working Tina4 Ruby project with an API endpoint and a rendered HTML page.

2. Prerequisites and Installation

What You Need

Before installing Tina4, make sure you have:

1. **Ruby 3.1 or later** -- check with:

```
ruby -v
```

You should see output like:

```
ruby 3.3.0 (2023-12-25 revision 5124f9ac75) [arm64-darwin23]
```

If you see a version lower than 3.1, upgrade Ruby first.

2. **Bundler** -- Ruby's dependency manager. Check with:

```
bundle --version
```

You should see:

```
Bundler version 2.5.6
```

If Bundler is not installed, install it with `gem install bundler`.

3. **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

```
# macOS (Homebrew)
brew install tina4stack/tap/tina4

# Linux / macOS (install script)
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash

# Windows (PowerShell)
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI is installed:

```
tina4 --version
```

```
tina4 0.1.0
```

4. **SQLite3 development libraries** -- these ship with most systems:

```
# macOS (already included)
# Ubuntu/Debian
sudo apt-get install libsqlite3-dev

# Fedora
sudo dnf install sqlite-devel
```

Creating a New Project

Use the Tina4 CLI to scaffold a new project:

```
tina4 init my-store
```

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: Ruby (Gemfile)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
# ...
```

Now install the Ruby dependencies:

```
cd my-store
bundle install
```

```
Fetching gem metadata from https://rubygems.org/...
Resolving dependencies...
Installing tina4 (3.0.0)
Bundle complete! 1 Gemfile dependency, 1 gem installed.
```

That is right -- **one gem**. No dependency tree. No version conflicts. Just `tina4`.

Starting the Dev Server

```
tina4 serve

 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|
```

```
  Tina4 Ruby v3.0.0
  Server running at http://0.0.0.0:7147
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7147`. You should see the Tina4 welcome page.

Open `http://localhost:7147/health` in your browser or curl it:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.0.0",
  "framework": "tina4-ruby"
}
```

Your Tina4 Ruby project is running.

3. Project Structure Walkthrough

Let us look at what `tina4 init` created:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
■■■ Gemfile                 # Gem dependencies
■■■ Gemfile.lock            # Locked dependency versions
■■■ app.rb                  # Application entry point
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ migrations/         # SQL migration files
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
# ...
```

Key directories:

• **`src/routes/`** -- Every `.rb` file here is auto-loaded at startup. Put your route definitions here. Organize into subdirectories if you want.

• **`src/orm/`** -- Every `.rb` file here is auto-loaded. Put your ORM model classes here.

• **`src/templates/`** -- Frond looks here when you call `response.render("my-page.html", data)`.

• **`src/public/`** -- Files served directly. `src/public/images/logo.png` is available at `/images/logo.png`.

• **`data/`** -- The default SQLite database (`app.db`) lives here. Gitignored because databases should not be in version control.

4. Your First Route

Let us create an API endpoint that returns a JSON greeting.

Create the file `src/routes/greeting.rb`:

```
Tina4::Router.get("/api/greeting/{name}") do |request, response|
  name = request.params["name"]
  response.json({
    message: "Hello, #{name}!",
    timestamp: Time.now.iso8601
  })
end
```

Save the file. If the dev server is running with live reload, it picks up the change automatically. If not, restart the server with `tina4 serve`.

Test It

Open your browser to:

```
http://localhost:7147/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Or use curl:

```
curl http://localhost:7147/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00+00:00"}
```

Notice the difference: the browser shows pretty-printed JSON (because of browser extensions or dev mode), while curl shows compact JSON. You can force pretty output by adding `?pretty=true`:

```
curl "http://localhost:7147/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Understanding What Happened

1. You created a file in `src/routes/`. Tina4 auto-discovered it at startup.

2. `Tina4::Router.get("/api/greeting/{name}")` registered a GET route with a path parameter `{name}`.

3. When you requested `/api/greeting/Alice`, the router matched the pattern and called your handler block.

4. `request.params["name"]` gave you the value `"Alice"` from the URL.

5. `response.json(...)` serialized the hash to JSON, set `Content-Type: application/json`, and returned a `200 OK` response.

Adding More HTTP Methods

Let us add a POST endpoint. Update `src/routes/greeting.rb`:

```
Tina4::Router.get("/api/greeting/{name}") do |request, response|
  name = request.params["name"]
  response.json({
    message: "Hello, #{name}!",
    timestamp: Time.now.iso8601
  })
end

Tina4::Router.post("/api/greeting") do |request, response|
  name = request.body["name"] || "World"
  language = request.body["language"] || "en"

  greetings = {
    "en" => "Hello",
    "es" => "Hola",
    "fr" => "Bonjour",
    "de" => "Hallo",
    "ja" => "Konnichiwa"
  }

  greeting = greetings[language] || greetings["en"]

  response.json({
    message: "#{greeting}, #{name}!",
    language: language
# ...
```

Test the POST endpoint:

```
curl -X POST http://localhost:7147/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (the second argument to `response.json`).

5. Your First Template

Tina4 uses the **Frond** template engine -- a zero-dependency, Twig-compatible engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, this will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
```

```
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0; }
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em; }
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
# ...
```

This base layout defines two blocks (`title` and `content`) that child templates can override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
                {% if not loop.last %}
                    {# Don't add separator after the last item #}
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
# ...
```

Create the Route That Renders the Template

Create `src/routes/pages.rb`:

```
Tina4::Router.get("/products") do |request, response|
  products = [
    {
      name: "Wireless Keyboard",
      description: "Ergonomic wireless keyboard with backlit keys.",
      price: 79.99,
      in_stock: true
```

```
    },
    {
      name: "USB-C Hub",
      description: "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
      price: 49.99,
      in_stock: true
    },
    {
      name: "Monitor Stand",
      description: "Adjustable aluminum monitor stand with cable management.",
      price: 129.99,
      in_stock: false
    },
    {
      name: "Mechanical Mouse",
      description: "High-precision wireless mouse with 16,000 DPI sensor.",
      price: 59.99,
      in_stock: true
# ...
```

See It in the Browser

Open `http://localhost:7147/products` in your browser. You should see:

• A dark navigation bar at the top with "Home" and "Products" links

• The heading "Our Products"

• A subheading showing "Showing 4 products"

• Four product cards, each with a name, description, price, and stock badge

• The "Monitor Stand" card shows a red "Out of Stock" badge

• The other three show green "In Stock" badges

How Template Rendering Works

1. `response.render("products.html", { products: products })` tells Frond to render `src/templates/products.html` with the given data.

2. Frond sees `{% extends "base.html" %}` and loads the base template.

3. The `{% block content %}` in `products.html` replaces the same block in `base.html`.

4. `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

5. `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.

6. `{% for product in products %}` loops through the array.

7. `{% if product.in_stock %}` conditionally renders the stock badge.

8. `{{ products | length }}` returns the count of items in the array.

About tina4css

The `tina4.css` file included in the base template is Tina4's built-in CSS utility framework. It provides layout utilities, typography, and common UI patterns without needing Bootstrap or Tailwind. It is auto-provided when you scaffold a project -- you do not need to download it separately.

6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

| Variable | Default Value | What It Means |

|----------|---------------|---------------|

| `TINA4_PORT` | `7147` | Server runs on port 7147 |

| `DATABASE_URL` | `sqlite:///data/app.db` | SQLite database in the `data/` directory |

| `TINA4_LOG_LEVEL` | `ALL` | All log messages are output |

| `CORS_ORIGINS` | `*` | All origins allowed (fine for development) |

| `TINA4_RATE_LIMIT` | `60` | 60 requests per minute per IP |

To change the port, add it to `.env`:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

For the complete `.env` reference with all 68 variables, see Book 0, Chapter 4.

7. The Dev Dashboard

With `TINA4_DEBUG=true`, Tina4 provides a built-in development dashboard. First, enable it by adding a console token to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart the server and navigate to:

```
http://localhost:7147/tina4/console
```

Enter your token (`my-dev-token`) when prompted. You will see:

• **System Overview** -- framework version, Ruby version, uptime, memory usage, database status

• **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.

• **Error Log** -- unhandled exceptions with stack traces and occurrence counts

• **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)

• **WebSocket Monitor** -- active WebSocket connections with metadata

• **Routes** -- all registered routes with their methods, paths, and middleware

The console is a powerful debugging tool. It shows you exactly what your application is doing without adding print statements or log calls to your code.

Additionally, when you visit any HTML page (like `/products`), you will see a **debug overlay** -- a toolbar at the bottom of the page showing:

• Request details (method, URL, duration)

• Database queries executed (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

This overlay is only visible when `TINA4_DEBUG=true`. It is never shown in production.

8. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

1. Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

2. If `name` is missing, defaults to `"Stranger"`

3. Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

4. The `time_of_day` should be calculated from the server's current hour:

• 5:00 - 11:59 = "morning"

• 12:00 - 16:59 = "afternoon"

• 17:00 - 20:59 = "evening"

• 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7147/api/greet?name=Sarah"
curl "http://localhost:7147/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

1. Displays a list of at least 5 products (hardcoded for now)

2. Each product has: name, category, price, and a boolean `featured` flag

3. Featured products should be visually highlighted (different background color, border, or badge)

4. The page should show the total number of products and the number of featured products

5. Use template inheritance -- create a layout template and a page template that extends it

6. Include `tina4.css` and `frond.js`

Your products data should look like this in your route handler:

```
products = [
  { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
  { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
  { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
  { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true },
  { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
]
```

Expected browser output:

• A page titled "Our Store"

• Text showing "5 products, 3 featured"

• A list of product cards with name, category, price, and a "Featured" badge on the highlighted items

• Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

9. Solutions

Solution A: Greeting API

Create `src/routes/greet.rb`:

```
Tina4::Router.get("/api/greet") do |request, response|
  name = request.query["name"] || "Stranger"
  hour = Time.now.hour

  time_of_day = if hour >= 5 && hour < 12
                  "morning"
                elsif hour >= 12 && hour < 17
                  "afternoon"
                elsif hour >= 17 && hour < 21
                  "evening"
                else
                  "night"
                end

  response.json({
    greeting: "Welcome, #{name}!",
    time_of_day: time_of_day
  })
end
```

Test:

```
curl "http://localhost:7147/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7147/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px; font-size: 
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
# ...
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.rb`:

```
Tina4::Router.get("/store") do |request, response|
  products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
```

```
    { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true },
    { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
  ]

  featured_count = products.count { |p| p[:featured] }

  response.render("store.html", {
    products: products,
    featured_count: featured_count
  })
end
```

Open `http://localhost:7147/store` in your browser. You should see:

• A dark header reading "Our Store"

• Text showing "5 products, 3 featured"

• Five product cards in a grid

• Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge

• Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border

• Each card shows the product name, category, and price formatted with two decimal places

10. Gotchas

1. File not auto-discovered

Problem: You created a route file but nothing happens when you visit the URL.

Cause: The file is not in `src/routes/`. Double-check the path. It must be inside `src/routes/` (or a subdirectory of it), and the file must end with `.rb`.

Fix: Move the file to `src/routes/your-file.rb` and restart the server.

2. "Uninitialized constant" errors

Problem: `NameError: uninitialized constant Tina4::Router` or similar.

Cause: The Tina4 gem is not loaded, or the file is not being required by the framework.

Fix: Make sure your `Gemfile` includes `gem "tina4"` and you have run `bundle install`. Route files in `src/routes/` are auto-loaded -- you do not need `require` statements.

3. JSON response shows HTML

Problem: Your JSON endpoint returns HTML instead of JSON.

Cause: You returned a string instead of using `response.json`. If you return a plain string, Tina4 treats it as HTML.

Fix: Always use `response.json(data)` for JSON endpoints, not `puts data.to_json`.

4. Template not found

Problem: `Template "my-page.html" not found` error.

Cause: The template file is not in `src/templates/`, or there is a typo in the filename.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `response.render` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7147)`

Cause: Another process (or another Tina4 instance) is using port 7147.

Fix: Either stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: In some setups, live reload may not be active. Also, browser caching can serve stale versions.

Fix: Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that does not help, restart the dev server with `Ctrl+C` and `tina4 serve`.

7. .env not loaded

Problem: Environment variables seem to have no effect.

Cause: The `.env` file must be at the project root (same directory as `Gemfile`). If it is in a subdirectory, Tina4 will not find it.

Fix: Move `.env` to the project root.

8. Debug mode in production

Problem: Your production site shows stack traces and query details.

Cause: `TINA4_DEBUG=true` in production.

Fix: Set `TINA4_DEBUG=false` in your production `.env`. This hides all debug information, enables HTML minification, and activates `.broken` file health checks.

Chapter 2: Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. You type `/products` in your browser, the framework finds the function that handles `/products`, runs it, and sends back the result. That mapping is called routing.

In Tina4, you define routes in Ruby files inside `src/routes/`. Every `.rb` file in that directory (and its subdirectories) is auto-loaded when the server starts. You do not need to register files or update a central config -- just drop a file in and it works.

Here is the simplest possible route:

```
Tina4::Router.get("/hello") do |request, response|
  response.json({ message: "Hello, World!" })
end
```

Save that as `src/routes/hello.rb`, start the server with `tina4 serve`, and visit `http://localhost:7147/hello`:

```
{"message":"Hello, World!"}
```

That is it. One line registers the route, one block handles the request.

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one has a method on `Tina4::Router`:

```
Tina4::Router.get("/products") do |request, response|
  response.json({ action: "list all products" })
end

Tina4::Router.post("/products") do |request, response|
  response.json({ action: "create a product" }, 201)
end

Tina4::Router.put("/products/{id}") do |request, response|
  id = request.params["id"]
  response.json({ action: "replace product #{id}" })
end

Tina4::Router.patch("/products/{id}") do |request, response|
  id = request.params["id"]
  response.json({ action: "update product #{id}" })
end

Tina4::Router.delete("/products/{id}") do |request, response|
  id = request.params["id"]
  response.json({ action: "delete product #{id}" })
end
```

Test each one:

```
curl http://localhost:7147/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7147/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7147/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7147/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7147/products/42
```

```
{"action":"delete product 42"}
```

Use `GET` for reading, `POST` for creating, `PUT` for full replacement, `PATCH` for partial updates, and `DELETE` for removal. This follows the REST convention and makes your API predictable.

3. Path Parameters

Path parameters let you capture values from the URL. Wrap the parameter name in curly braces:

```
Tina4::Router.get("/users/{id}/posts/{post_id}") do |request, response|
  user_id = request.params["id"]
  post_id = request.params["post_id"]

  response.json({
    user_id: user_id,
    post_id: post_id
  })
end
```

```
curl http://localhost:7147/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice that `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are always strings by default.

Typed Parameters

You can enforce a type by adding a colon and the type after the parameter name:

```
Tina4::Router.get("/orders/{id:int}") do |request, response|
  id = request.params["id"]
  response.json({
    order_id: id,
    type: id.class.name
  })
end
```

```
curl http://localhost:7147/orders/42
```

```
{"order_id":42,"type":"Integer"}
```

If you pass a non-integer value, the route will not match and you will get a 404:

```
curl http://localhost:7147/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

| Type | Matches | Example |

|------|---------|---------|

| `int` | Digits only | `{id:int}` matches `42` but not `abc` |

| `float` | Decimal numbers | `{price:float}` matches `19.99` |

| `alpha` | Letters only | `{slug:alpha}` matches `hello` but not `hello123` |

| `alphanumeric` | Letters and digits | `{code:alphanumeric}` matches `abc123` |

4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them via `request.query`:

```
Tina4::Router.get("/search") do |request, response|
  q = request.query["q"] || ""
  page = (request.query["page"] || 1).to_i
  limit = (request.query["limit"] || 10).to_i

  response.json({
    query: q,
    page: page,
    limit: limit,
    offset: (page - 1) * limit
  })
end
```

```
curl "http://localhost:7147/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

If a query parameter is missing, `request.query["key"]` will return `nil`, so always use `||` to provide defaults.

5. Route Groups

When you have a set of routes that share a common prefix, use `Tina4::Router.group` to avoid repeating yourself:

```
Tina4::Router.group("/api/v1") do

  Tina4::Router.get("/users") do |request, response|
    response.json({ users: [] })
  end

  Tina4::Router.get("/users/{id:int}") do |request, response|
    id = request.params["id"]
    response.json({ user: { id: id, name: "Alice" } })
  end

  Tina4::Router.post("/users") do |request, response|
    response.json({ created: true }, 201)
  end

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

end
```

The routes above register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. You write short paths inside the group, and Tina4 prepends the prefix automatically.

```
curl http://localhost:7147/api/v1/users
```

```
{"users":[]}
```

```
curl http://localhost:7147/api/v1/products
```

```
{"products":[]}
```

Groups can be nested:

```
Tina4::Router.group("/api") do
  Tina4::Router.group("/v1") do
    Tina4::Router.get("/status") do |request, response|
      response.json({ version: "1.0" })
    end
  end

  Tina4::Router.group("/v2") do
    Tina4::Router.get("/status") do |request, response|
      response.json({ version: "2.0" })
    end
  end
end
```

```
curl http://localhost:7147/api/v1/status
```

```
{"version":"1.0"}
```

```
curl http://localhost:7147/api/v2/status
```

```
{"version":"2.0"}
```

6. Middleware

Middleware is code that runs before (or after) your route handler. Use it for authentication, logging, rate limiting, input validation, or anything that should happen on multiple routes.

Middleware on a Single Route

Pass middleware as the third argument to any route method:

```
log_request = lambda do |request, response, next_handler|
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  $stderr.puts "[#{Time.now.strftime('%Y-%m-%d %H:%M:%S')}] #{request.method} #{request.path}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  $stderr.puts "  Completed in #{duration}ms"

  result
end

Tina4::Router.get("/api/data", middleware: "log_request") do |request, response|
  response.json({ data: [1, 2, 3] })
end
```

The middleware receives `request`, `response`, and `next_handler`. Call `next_handler.call(request, response)` to continue to the route handler. If you do not call `next_handler`, the route handler never runs

-- useful for blocking unauthorized requests.

Blocking Middleware

Here is middleware that checks for an API key:

```
require_api_key = lambda do |request, response, next_handler|
  api_key = request.headers["X-API-Key"] || ""

  if api_key != "my-secret-key"
    return response.json({ error: "Invalid API key" }, 401)
  end

  next_handler.call(request, response)
end

Tina4::Router.get("/api/secret", middleware: "require_api_key") do |request, response|
  response.json({ secret: "The answer is 42" })
end
```

```
curl http://localhost:7147/api/secret
```

```
{"error":"Invalid API key"}
```

The response status is `401 Unauthorized`.

```
curl http://localhost:7147/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Middleware on a Group

Apply middleware to an entire group by passing it as an option to `Tina4::Router.group`:

```
Tina4::Router.group("/api/admin", middleware: "require_auth") do

  Tina4::Router.get("/dashboard") do |request, response|
    response.json({ page: "admin dashboard" })
  end

  Tina4::Router.get("/users") do |request, response|
    response.json({ page: "user management" })
  end

end
```

Every route inside the group now requires the `Authorization` header. You do not need to add the middleware to each route individually.

Multiple Middleware

Chain multiple middleware by passing an array:

```
Tina4::Router.get("/api/important", middleware: ["log_request", "require_api_key", "require_auth"]) do |reques
  response.json({ data: "important stuff" })
end
```

Middleware runs in order: `log_request` first, then `require_api_key`, then `require_auth`, then the route handler. If any middleware does not call `next_handler`, the chain stops there.

7. Route Decorators: @noauth and @secured

Tina4 provides two special decorators for controlling authentication on routes.

@noauth -- Public Routes

When your application has global authentication middleware, use the `@noauth` annotation to mark specific routes as public:

```
# @noauth
Tina4::Router.get("/api/public/info") do |request, response|
  response.json({
    app: "My Store",
    version: "1.0.0"
  })
end
```

The `@noauth` comment tells Tina4 to skip authentication checks for this route, even if global auth middleware is configured in `.env` or applied to the parent group.

@secured -- Protected GET Routes

The `@secured` annotation explicitly marks a GET route as requiring authentication:

```
# @secured
Tina4::Router.get("/api/profile") do |request, response|
  # request.user is populated by the auth middleware
  response.json({ user: request.user })
end
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are considered secured. `GET` routes are not -- they are public unless you add `@secured`. This matches the common pattern where reading data is public but modifying data requires authentication.

8. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match anything after it:

```
Tina4::Router.get("/docs/*") do |request, response|
  path = request.params["*"] || ""
  response.json({
    section: "docs",
    path: path
  })
end
```

```
curl http://localhost:7147/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7147/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

Register a catch-all to handle any unmatched URL:

```
Tina4::Router.get("/*") do |request, response|
  response.json({
    error: "Page not found",
    path: request.path
```

```
  }, 404)
end
```

This route should be defined last (or in a file that sorts alphabetically after your other route files) so it does not shadow your real routes. Tina4 matches routes in the order they are registered -- the first match wins.

Alternatively, you can create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 automatically uses this template for any unmatched route when the template file exists.

9. Route Listing via CLI

As your application grows, you will want to see all registered routes at a glance. Use the Tina4 CLI:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          require_auth        public
GET      /api/admin/users              require_auth        public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

The `Auth` column shows whether a route is public, secured (default for non-GET methods), explicitly `@noauth`, or explicitly `@secured`.

You can also filter by method:

```
tina4 routes --method POST

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Or search for a specific path pattern:

```
tina4 routes --filter users

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
```

```
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/users              require_auth        public
```

10. Organizing Route Files

You are free to organize route files any way you like. Tina4 loads every `.rb` file in `src/routes/` recursively. Here are two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.rb     # All product routes
■■■ users.rb        # All user routes
■■■ orders.rb       # All order routes
■■■ pages.rb        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.rb
■   ■■■ users.rb
■   ■■■ orders.rb
■■■ admin/
■   ■■■ dashboard.rb
■   ■■■ settings.rb
■■■ pages/
    ■■■ home.rb
    ■■■ about.rb
```

Both patterns work identically. The directory structure has no effect on the URL paths -- only the route definitions inside the files matter. Choose whichever pattern keeps your project navigable.

11. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data is stored in a Ruby array (no database yet -- we will add that in Chapter 5).

Requirements

Create a file `src/routes/product_api.rb` with the following routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all products. Support `?category=` filter. |

| `GET` | `/api/products/{id:int}` | Get a single product by ID. Return 404 if not found. |

| `POST` | `/api/products` | Create a new product. Return 201. |

| `PUT` | `/api/products/{id:int}` | Replace a product. Return 404 if not found. |

| `DELETE` | `/api/products/{id:int}` | Delete a product. Return 204 with no body. |

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Start with this seed data:

```
products = [
  { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, in_stock: true },
  { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true },
  { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, in_stock: false },
  { id: 4, name: "Standing Desk", category: "Office", price: 549.99, in_stock: true },
  { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true }
]
```

Test with:

```
# List all
curl http://localhost:7147/api/products

# Filter by category
curl "http://localhost:7147/api/products?category=Fitness"

# Get one
curl http://localhost:7147/api/products/3

# Create
curl -X POST http://localhost:7147/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7147/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7147/api/products/3

# Not found
curl http://localhost:7147/api/products/999
```

12. Solution

Create `src/routes/product_api.rb`:

```
# In-memory product store (resets on server restart)
$products = [
  { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, in_stock: true },
  { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true },
  { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, in_stock: false },
  { id: 4, name: "Standing Desk", category: "Office", price: 549.99, in_stock: true },
  { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true }
]

$next_id = 6

# List all products, optionally filter by category
Tina4::Router.get("/api/products") do |request, response|
  category = request.query["category"]

  if category
    filtered = $products.select { |p| p[:category].downcase == category.downcase }
    response.json({ products: filtered, count: filtered.length })
  else
    response.json({ products: $products, count: $products.length })
  end
```

```
end

# Get a single product by ID
Tina4::Router.get("/api/products/{id:int}") do |request, response|
# ...
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock":true},{"id"
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id":5,"name":"Run
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

13. Gotchas

1. Trailing Slashes Matter

Problem: `/products` works but `/products/` returns a 404 (or vice versa).

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention and stick with it. If you want both to work, register the route without a trailing slash -- Tina4 will redirect `/products/` to `/products` automatically when `TINA4_TRAILING_SLASH_REDIRECT=true` is set in `.env`.

2. Parameter Names Must Be Unique in a Path

Problem: `/users/{id}/posts/{id}` does not work as expected -- both parameters have the same name.

Cause: The second `{id}` overwrites the first in `request.params`.

Fix: Use distinct names: `/users/{user_id}/posts/{post_id}`.

3. Method Conflicts

Problem: You defined `Tina4::Router.get("/items/{id}", ...)` and `Tina4::Router.get("/items/{action}", ...)` and the wrong handler runs.

Cause: Both patterns match `/items/42`. The first one registered wins.

Fix: Use typed parameters to disambiguate: `Tina4::Router.get("/items/{id:int}", ...)` will only match integers, leaving `/items/export` free for the other route. Alternatively, restructure your paths: `/items/{id:int}` and `/items/actions/{action}`.

4. Route Handler Must Return a Response

Problem: Your route handler runs but the browser shows an empty page or a 500 error.

Cause: You forgot to use `response.json` or `response.render`. Without a return value from the response object, the handler returns `nil` and Tina4 does not know what to send back.

Fix: Always use `response.json(...)` or `response.html(...)` or `response.render(...)`. Every handler must produce a response.

5. Block Syntax Matters

Problem: Your route handler raises a syntax error about unexpected blocks.

Cause: Ruby blocks with `do...end` and `{...}` have different precedence. For multi-line route handlers, always use `do...end`.

Fix: Use `do |request, response| ... end` for route blocks. The curly brace form `{ |request, response| ... }` works for single-line handlers but can cause parsing issues with method arguments.

6. Middleware Must Be a Named Function or String

Problem: Passing an inline lambda as middleware causes unexpected behavior.

Cause: Tina4 expects middleware to be referenced by name (a string), not as an inline block. The string is resolved to a method or lambda at runtime.

Fix: Define your middleware as a named method or lambda and pass the name as a string: `"my_middleware"`, not `lambda { |req, res, next_h| ... }`.

7. Group Prefix Must Start with a Slash

Problem: `Tina4::Router.group("api/v1")` produces routes like `/api/v1/users` but they do not match.

Cause: The group prefix should start with `/` for consistency. While Tina4 may auto-correct this, it is better to be explicit.

Fix: Always start group prefixes with `/`: `Tina4::Router.group("/api/v1")`.

Chapter 3: Request & Response

1. The Two Objects You Always Get

Every route handler in Tina4 receives two arguments: `request` and `response`. The request tells you what the client sent. The response is how you send something back. Together they are the entire HTTP conversation.

```
Tina4::Router.get("/echo") do |request, response|
  response.json({
    method: request.method,
    path: request.path,
    your_ip: request.ip
  })
end
```

```
curl http://localhost:7147/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

That is the pattern for every route: inspect the request, build the response, return it.

2. The Request Object

The `request` object gives you access to everything the client sent. Here is the complete list of properties.

method

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
request.method # "GET"
```

path

The URL path without query parameters:

```
# Request to /api/users?page=2
request.path # "/api/users"
```

params

Path parameters from the URL pattern (see Chapter 2):

```
# Route: /users/{id}/posts/{post_id}
# Request: /users/5/posts/42
request.params["id"]      # "5" (or 5 if typed as {id:int})
request.params["post_id"] # "42"
```

query

Query string parameters as a hash:

```
# Request: /search?q=keyboard&page=2&sort=price
request.query["q"]    # "keyboard"
request.query["page"] # "2"
request.query["sort"] # "price"
```

body

The parsed request body. For JSON requests, this is a hash. For form submissions, it contains the form fields:

```
# POST with {"name": "Widget", "price": 9.99}
request.body["name"]  # "Widget"
request.body["price"] # 9.99
```

headers

Request headers as a hash. Header names are normalized to their original casing:

```
request.headers["Content-Type"]  # "application/json"
request.headers["Authorization"] # "Bearer eyJhbGci..."
request.headers["X-Custom"]      # "my-value"
```

ip

The client's IP address:

```
request.ip # "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers when behind a reverse proxy.

cookies

Cookies sent by the client:

```
request.cookies["session_id"]  # "abc123"
request.cookies["preferences"] # "dark-mode"
```

files

Uploaded files (covered in detail in section 7):

```
request.files["avatar"] # File object with name, type, size, tmp_path
```

Inspecting the Full Request

Here is a route that dumps everything:

```
Tina4::Router.post("/debug/request") do |request, response|
  response.json({
    method: request.method,
    path: request.path,
    params: request.params,
    query: request.query,
    body: request.body,
    headers: request.headers,
    ip: request.ip,
    cookies: request.cookies
  })
end
```

```
curl -X POST "http://localhost:7147/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
```

```
    "Content-Type": "application/json",
    "X-Custom": "hello",
    "Host": "localhost:7147",
    "User-Agent": "curl/8.4.0",
    "Accept": "*/*",
    "Content-Length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

3. The Response Object

The `response` object is your toolkit for sending data back to the client. Every method on it returns the response, so you can chain calls together.

json -- JSON Response

The most common response for APIs. Pass any hash or value and it becomes JSON:

```
response.json({ name: "Alice", age: 30 })
```

```
{"name":"Alice","age":30}
```

Pass a status code as the second argument:

```
response.json({ id: 7, name: "Widget" }, 201)
```

This returns `201 Created` with the JSON body.

html -- Raw HTML Response

Return an HTML string directly:

```
response.html("<h1>Hello</h1><p>This is HTML.</p>")
```

Sets `Content-Type: text/html; charset=utf-8` automatically.

text -- Plain Text Response

Return plain text:

```
response.text("Just a plain string.")
```

Sets `Content-Type: text/plain; charset=utf-8`.

render -- Template Response

Render a Frond template with data (covered in depth in Chapter 4):

```
response.render("products.html", {
  products: products,
  title: "Our Products"
})
```

Tina4 looks for the template in `src/templates/`, renders it with the provided data, and returns the HTML.

redirect -- Redirect Response

Send the client to a different URL:

```
response.redirect("/login")
```

This sends a `302 Found` redirect by default. Pass a different status code for permanent redirects:

```
response.redirect("/new-location", 301)
```

file -- File Download Response

Send a file to the client for download:

```
response.file("/path/to/report.pdf")
```

This sets the appropriate `Content-Type` based on the file extension and adds a `Content-Disposition` header so the browser downloads the file rather than displaying it.

You can set a custom filename:

```
response.file("/path/to/report.pdf", "monthly-report-march-2026.pdf")
```

4. Status Codes

Every response method accepts a status code. Here are the most common ones you will use:

| Code | Meaning | When to Use |

|------|---------|-------------|

| `200` | OK | Default. Successful GET, PUT, PATCH. |

| `201` | Created | Successful POST that created a resource. |

| `204` | No Content | Successful DELETE. No body needed. |

| `301` | Moved Permanently | URL has permanently changed. |

| `302` | Found | Temporary redirect. |

| `400` | Bad Request | Invalid input from the client. |

| `401` | Unauthorized | Missing or invalid authentication. |

| `403` | Forbidden | Authenticated but not allowed. |

| `404` | Not Found | Resource does not exist. |

| `409` | Conflict | Duplicate or conflicting data. |

| `422` | Unprocessable Entity | Valid JSON but fails business rules. |

| `500` | Internal Server Error | Something went wrong on the server. |

You can also set the status explicitly with the `status` method and chain it:

```
response.status(201).json({ id: 7, created: true })
```

This is equivalent to `response.json({ id: 7, created: true }, 201)` but some developers find the chained form more readable.

5. Custom Headers

Set response headers with the `header` method:

```
Tina4::Router.get("/api/data") do |request, response|
```

```
  response
    .header("X-Request-Id", SecureRandom.hex(8))
    .header("X-Rate-Limit-Remaining", "57")
    .header("Cache-Control", "no-cache")
    .json({ data: [1, 2, 3] })
end
```

```
curl -v http://localhost:7147/api/data 2>&1 | grep "< X-"
```

```
< X-Request-Id: 65f3a7b8c1234567
< X-Rate-Limit-Remaining: 57
```

Headers are case-insensitive in HTTP, but it is conventional to use `Title-Case` for custom headers and prefix them with `X-`.

CORS Headers

Tina4 handles CORS automatically based on the `CORS_ORIGINS` setting in `.env`. The default `*` allows all origins. For production, restrict it:

```
CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

You rarely need to set CORS headers manually, but you can if needed:

```
response
  .header("Access-Control-Allow-Origin", "https://myapp.com")
  .header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE")
  .header("Access-Control-Allow-Headers", "Content-Type, Authorization")
  .json({ data: "value" })
```

6. Cookies

Set cookies on the response:

```
Tina4::Router.post("/login") do |request, response|
  # After validating credentials...
  response
    .cookie("session_id", "abc123xyz", {
      http_only: true,
      secure: true,
      same_site: "Strict",
      max_age: 3600,       # 1 hour in seconds
      path: "/"
    })
    .json({ message: "Logged in" })
end
```

Cookie options:

| Option | Type | Default | Description |

|--------|------|---------|-------------|

| `http_only` | bool | `false` | Cannot be accessed by JavaScript |

| `secure` | bool | `false` | Only sent over HTTPS |

| `same_site` | string | `"Lax"` | `"Strict"`, `"Lax"`, or `"None"` |

| `max_age` | int | session | Lifetime in seconds |

| `path` | string | `"/"` | URL path scope |

| `domain` | string | current | Domain scope |

Read cookies from the request:

```
Tina4::Router.get("/profile") do |request, response|
  session_id = request.cookies["session_id"]

  if session_id.nil?
    return response.json({ error: "Not logged in" }, 401)
  end

  response.json({ session: session_id })
end
```

Delete a cookie by setting its `max_age` to `0`:

```
response
  .cookie("session_id", "", { max_age: 0, path: "/" })
  .json({ message: "Logged out" })
```

7. File Uploads

Uploaded files are available via `request.files`. Each file is an object with properties for the file's metadata and a temporary path.

Handling a Single File Upload

```
Tina4::Router.post("/api/upload") do |request, response|
  if request.files["image"].nil?
    return response.json({ error: "No file uploaded" }, 400)
  end

  file = request.files["image"]

  response.json({
    name: file.name,        # "photo.jpg"
    type: file.type,        # "image/jpeg"
    size: file.size,        # 245760 (bytes)
    tmp_path: file.tmp_path # Temporary file location
  })
end
```

Test with curl:

```
curl -X POST http://localhost:7147/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "name": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760,
  "tmp_path": "/tmp/tina4_upload_abc123"
}
```

Saving the Uploaded File

The uploaded file is stored in a temporary location. Move it to a permanent location:

```
Tina4::Router.post("/api/upload") do |request, response|
  if request.files["image"].nil?
    return response.json({ error: "No file uploaded" }, 400)
  end
```

```
  file = request.files["image"]

  # Validate file type
  allowed_types = ["image/jpeg", "image/png", "image/gif", "image/webp"]
  unless allowed_types.include?(file.type)
    return response.json({ error: "Invalid file type. Allowed: JPEG, PNG, GIF, WebP" }, 400)
  end

  # Validate file size (max 5MB)
  max_size = 5 * 1024 * 1024
  if file.size > max_size
    return response.json({ error: "File too large. Maximum size: 5MB" }, 400)
  end

  # Generate a unique filename
  extension = File.extname(file.name)
  filename = "img_#{SecureRandom.hex(8)}#{extension}"
  upload_dir = File.join(__dir__, "../../public/uploads")
  destination = File.join(upload_dir, filename)

# ...
```

```
curl -X POST http://localhost:7147/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "filename": "img_a1b2c3d4e5f6a7b8.jpg",
  "url": "/uploads/img_a1b2c3d4e5f6a7b8.jpg",
  "size": 245760
}
```

The uploaded file is now available at `http://localhost:7147/uploads/img_a1b2c3d4e5f6a7b8.jpg`.

Handling Multiple Files

When the HTML form uses `multiple` or you have multiple file inputs:

```
Tina4::Router.post("/api/upload-many") do |request, response|
  results = []

  request.files.each do |key, file|
    extension = File.extname(file.name)
    filename = "file_#{SecureRandom.hex(8)}#{extension}"
    upload_dir = File.join(__dir__, "../../public/uploads")
    destination = File.join(upload_dir, filename)

    FileUtils.mkdir_p(upload_dir) unless Dir.exist?(upload_dir)
    FileUtils.mv(file.tmp_path, destination)

    results << {
      original_name: file.name,
      saved_as: filename,
      url: "/uploads/#{filename}"
    }
  end

  response.json({ uploaded: results, count: results.length }, 201)
end
```

8. File Downloads

Send files to the client using `response.file`:

```
Tina4::Router.get("/api/reports/{filename}") do |request, response|
  filename = request.params["filename"]
  filepath = File.join(__dir__, "../../data/reports", filename)

  unless File.exist?(filepath)
    return response.json({ error: "Report not found" }, 404)
  end

  response.file(filepath)
end
```

The browser will download the file. Tina4 automatically detects the MIME type from the file extension and sets the appropriate headers.

To force a specific download filename:

```
response.file(filepath, "Q1-2026-Sales-Report.pdf")
```

9. Content Negotiation

Sometimes the same endpoint should return different formats based on what the client asks for. Check the `Accept` header:

```
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params["id"]
  product = {
    id: id,
    name: "Wireless Keyboard",
    price: 79.99
  }

  accept = request.headers["Accept"] || "application/json"

  if accept.include?("text/html")
    response.render("product-detail.html", { product: product })
  elsif accept.include?("text/plain")
    text = "Product ##{id}: #{product[:name]} - $#{product[:price]}"
    response.text(text)
  else
    # Default to JSON
    response.json(product)
  end
end
```

```
# JSON (default)
curl http://localhost:7147/api/products/1
```

```
{"id":1,"name":"Wireless Keyboard","price":79.99}
```

```
# Plain text
curl http://localhost:7147/api/products/1 -H "Accept: text/plain"
```

```
Product #1: Wireless Keyboard - $79.99
```

```
# HTML (would render the template)
curl http://localhost:7147/api/products/1 -H "Accept: text/html"
```

```
<!DOCTYPE html>
<html>...rendered template...</html>
```

10. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back. Create two endpoints:

Requirements

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/images` | Upload an image. Validate type and size. Return the image URL. |

| `GET` | `/api/images/{filename}` | Return the uploaded image file. Return 404 if not found. |

Rules:

1. Only accept JPEG, PNG, and WebP files

2. Maximum file size: 2MB

3. Save files to `src/public/uploads/` with a unique filename

4. Return the original filename, the saved filename, file size in KB, and the URL

5. The GET endpoint should serve the file directly (not JSON)

Test with:

```
# Upload
curl -X POST http://localhost:7147/api/images \
  -F "image=@/path/to/photo.jpg"

# Download
curl http://localhost:7147/api/images/img_a1b2c3d4e5f6a7b8.jpg --output downloaded.jpg
```

11. Solution

Create `src/routes/images.rb`:

```
require "securerandom"
require "fileutils"

Tina4::Router.post("/api/images") do |request, response|
  # Check if a file was uploaded
  if request.files["image"].nil?
    return response.json({ error: "No image file provided. Use field name 'image'." }, 400)
  end

  file = request.files["image"]

  # Validate file type
  allowed_types = ["image/jpeg", "image/png", "image/webp"]
  unless allowed_types.include?(file.type)
    return response.json({
      error: "Invalid file type",
      received: file.type,
      allowed: allowed_types
    }, 400)
  end

  # Validate file size (max 2MB)
```

```
  max_size = 2 * 1024 * 1024
  if file.size > max_size
    return response.json({
# ...
```

Expected output for upload:

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_a1b2c3d4e5f6a7b8.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_a1b2c3d4e5f6a7b8.jpg"
}
```

(Status: `201 Created`)

Expected output for invalid type:

```
{
  "error": "Invalid file type",
  "received": "application/pdf",
  "allowed": ["image/jpeg", "image/png", "image/webp"]
}
```

(Status: `400 Bad Request`)

Expected output for file too large:

```
{
  "error": "File too large",
  "size_bytes": 5242880,
  "max_bytes": 2097152
}
```

(Status: `400 Bad Request`)

The GET endpoint returns the raw image file with the correct `Content-Type` header. The browser displays the image directly, and curl with `--output` saves it to disk.

12. Gotchas

1. Forgetting the response method

Problem: Your handler runs (you can see log output) but the browser shows an empty response or a 500 error.

Cause: You did not call `response.json`, `response.html`, or another response method. Ruby returns the last expression in a block, but if the last expression is not a response, Tina4 cannot send it to the client.

Fix: Always end your handler with a response method call: `response.json(...)`, `response.html(...)`, or `response.render(...)`.

2. Body Is Nil for JSON Requests

Problem: `request.body` is `nil` or empty even though you are sending JSON.

Cause: You forgot the `Content-Type: application/json` header in your request. Without it, Tina4 does not know to parse the body as JSON.

Fix: Always include `-H "Content-Type: application/json"` when sending JSON with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` requires `headers: {"Content-Type": "application/json"}`.

3. Content-Type Mismatch

Problem: You call `response.json` but the client receives HTML, or you call `response.html` but the client gets plain text.

Cause: A middleware or error handler is overwriting the response. Or you are using `puts` instead of a response method.

Fix: Make sure your handler uses `response.json(...)`, `response.html(...)`, or another response method. Do not use `puts` -- it bypasses the response object entirely.

4. File Uploads Return Empty

Problem: `request.files` is empty even though you are uploading a file.

Cause: The form is not using `enctype="multipart/form-data"`, or the curl command is using `-d` instead of `-F`.

Fix: For HTML forms, use `<form enctype="multipart/form-data">`. For curl, use `-F "field=@file.jpg"` (with `@`), not `-d`.

5. Redirect Loops

Problem: The browser shows "too many redirects" or hangs.

Cause: You have a route that redirects to another route, which redirects back to the first one. For example, `/login` redirects to `/dashboard`, and `/dashboard` redirects to `/login` because the user is not authenticated.

Fix: Check your redirect logic carefully. Use the browser's network inspector to trace the redirect chain. Make sure your auth check does not redirect authenticated users away from pages they should access.

6. Cookie Not Set

Problem: You called `response.cookie(...)` but the browser does not show the cookie.

Cause: If `secure` is `true`, the cookie is only sent over HTTPS. During local development with `http://localhost`, the cookie is silently dropped.

Fix: Set `secure: false` during development, or use `secure: ENV["TINA4_DEBUG"] != "true"` to auto-switch based on environment.

7. Large Request Body Rejected

Problem: POST requests with large bodies return a 413 error.

Cause: The request body exceeds the configured maximum size.

Fix: Increase `TINA4_MAX_BODY_SIZE` in `.env`. The default is `10mb`. For file upload endpoints, you may need `50mb` or more:

```
TINA4_MAX_BODY_SIZE=50mb
```

Chapter 4: Templates

1. Why Templates

In Chapter 1, you saw `response.render("products.html", data)` and it produced a full HTML page. That rendering was done by **Frond**, Tina4's built-in template engine. Frond is a zero-dependency, Twig-compatible engine that ships with the framework. If you have used Twig, Jinja2, or Nunjucks, you already know 90% of Frond.

Templates live in `src/templates/`. When you call `response.render("page.html", data)`, Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter covers every feature of the template engine so you can build real pages.

2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
Tina4::Router.get("/welcome") do |request, response|
  response.render("welcome.html", { name: "Alice" })
end
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

Expected browser output:

```
Hello, Alice!
```

Accessing Nested Data

Use dot notation for nested hashes:

```
data = {
  user: {
    name: "Alice",
    email: "alice@example.com",
    address: {
      city: "Cape Town",
      country: "South Africa"
    }
  }
}

response.render("profile.html", data)
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

Output:

```
Alice lives in Cape Town, South Africa.
```

Expressions

You can use basic expressions inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

3. Template Inheritance

Template inheritance is the most powerful feature. Define a base layout once, then extend it in every page.

Base Layout

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
# ...
```

This base defines four blocks: `title`, `head`, `content`, and `scripts`. Child templates override only the blocks they need.

Child Template

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}

{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
```

```
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
Tina4::Router.get("/about") do |request, response|
  response.render("about.twig", {
    founded_year: 2020,
    team_size: 12,
    office_count: 3
  })
end
```

Expected browser output: A full HTML page with the nav, the "About Us" content, and the footer. The `<title>` tag says "About Us". The `head` and `scripts` blocks are empty because the child template did not override them.

Using `{{ parent() }}`

Sometimes you want to add to a block rather than replace it entirely. Use `{{ parent() }}`:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

The `head` block now contains everything from the base template's `head` block plus the extra stylesheet.

4. Includes

Break your templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.in_stock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

The `product` variable is automatically available inside the included template because it exists in the current scope (the for loop).

Passing Variables to Includes

You can pass specific variables with `with`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

With `only`, the included template can only access `site_name` -- it cannot see any other variables from the parent.

5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

The `loop` Variable

Inside a for loop, Frond provides a special `loop` variable:

| Property | Type | Description |

|----------|------|-------------|

| `loop.index` | int | Current iteration (1-based) |

| `loop.index0` | int | Current iteration (0-based) |

| `loop.first` | bool | True on the first iteration |

| `loop.last` | bool | True on the last iteration |

| `loop.length` | int | Total number of items |

| `loop.revindex` | int | Iterations remaining (1-based) |

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
```

```
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Empty Lists

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

If `products` is empty (or not defined), the `else` block renders instead.

Looping Over Key-Value Pairs

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

6. Conditionals

if / elseif / else

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

Ternary Operator

For inline conditionals:

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

Testing for Existence

Use `is defined` to check if a variable exists:

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

Truthiness

These values are considered false: `false`, `nil`, `0`, `""` (empty string), `[]` (empty array). Everything else is true.

```
{% if items %}
    <p>{{ items | length }} items found.</p>
{% else %}
    <p>No items.</p>
{% endif %}
```

7. Filters

Filters transform values. Apply them with the `|` (pipe) character:

```
{{ name | upper }}
```

Here are the most commonly used filters:

Text Filters

| Filter | Input | Output | Description |

|--------|-------|--------|----------|

| `upper` | `"hello"` | `"HELLO"` | Uppercase |

| `lower` | `"HELLO"` | `"hello"` | Lowercase |

| `capitalize` | `"hello world"` | `"Hello world"` | Capitalize first letter |

| `title` | `"hello world"` | `"Hello World"` | Capitalize each word |

| `trim` | `" hello "` | `"hello"` | Remove whitespace |

| `striptags` | `"<b>bold</b>"` | `"bold"` | Remove HTML tags |

Number Filters

| Filter | Input | Output | Description |

|--------|-------|--------|----------|

| `number_format(2)` | `1234.5` | `"1,234.50"` | Format number |

| `round` | `3.7` | `4` | Round to nearest integer |

| `round(2)` | `3.14159` | `3.14` | Round to N decimal places |

| `abs` | `-5` | `5` | Absolute value |

Array Filters

| Filter | Input | Output | Description |

|--------|-------|--------|----------|

| `length` | `[1,2,3]` | `3` | Count items |

| `join(", ")` | `["a","b","c"]` | `"a, b, c"` | Join with separator |

| `first` | `[1,2,3]` | `1` | First item |

| `last` | `[1,2,3]` | `3` | Last item |

| `reverse` | `[1,2,3]` | `[3,2,1]` | Reverse order |

| `sort` | `[3,1,2]` | `[1,2,3]` | Sort ascending |

| `slice(1, 2)` | `[1,2,3,4]` | `[2,3]` | Slice from offset, length |

Date Filter

```
<p>Published: {{ created_at | date("F j, Y") }}</p>
<p>Time: {{ created_at | date("H:i") }}</p>
```

With a timestamp or date string as input:

• `date("F j, Y")` outputs `"March 22, 2026"`

• `date("H:i")` outputs `"14:30"`

• `date("Y-m-d")` outputs `"2026-03-22"`

The `default` Filter

Provide a fallback value when a variable is nil or undefined:

```
<p>{{ subtitle | default("No subtitle") }}</p>
<p>{{ user.nickname | default(user.name) | default("Anonymous") }}</p>
```

The `escape` and `raw` Filters

By default, all `{{ }}` output is auto-escaped for HTML safety. This prevents XSS attacks.

```
{{ user_input }}
{# If user_input is "<script>alert('xss')</script>", outputs:
   &lt;script&gt;alert('xss')&lt;/script&gt; #}
```

If you trust the content and need raw HTML:

```
{{ trusted_html | raw }}
```

Use `raw` sparingly. Only apply it to content you fully control, never to user input.

Chaining Filters

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

8. Macros

Macros are reusable template functions. Think of them as components you define once and call many times.

Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
```

```
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ value | defaul
    </div>
{% endmacro %}
```

Using Macros

Import macros in your template:

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}
        {{ input("phone", "Phone Number", "tel", user.phone) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

Expected output (simplified):

```
<div class="alert alert-success">
    Your profile has been updated.
</div>

<form method="POST" action="/profile">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" value="Alice">
    </div>
    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" value="alice@example.com">
    </div>
    <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" value="">
    </div>

    <a href="#" class="btn btn-primary">Save Changes</a>
    <a href="/dashboard" class="btn btn-secondary">Cancel</a>
</form>
```

9. Special Tags

{% raw %} -- Literal Output

When you need to output literal `{{ }}` or `{% %}` (for example, in a Vue.js or Angular template):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

This outputs the literal text `{{ message }}` without trying to process it as a Frond expression.

{% spaceless %} -- Remove Whitespace

Remove whitespace between HTML tags:

```
{% spaceless %}
    <div>
        <span>Hello</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span></div>
```

Useful for inline elements where whitespace creates unwanted gaps.

{% autoescape %} -- Control Escaping

Override the auto-escaping behavior for a block:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside the block outputs without HTML escaping. Equivalent to using `| raw` on every variable, but more convenient for large blocks of trusted content.

Comments

Template comments are invisible in the output:

```
{# This comment will not appear in the HTML output #}

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

10. tina4css Integration

Every Tina4 project includes `tina4.css`, a built-in CSS utility framework. It is available at `/css/tina4.css` and provides layout, typography, and common UI patterns without external dependencies.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

Layout Classes

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
```

```
        <div class="col-6">Right half</div>
    </div>
</div>
```

Common Components

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>

<!-- Cards -->
<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content here</div>
    <div class="card-footer">Footer</div>
</div>

<!-- Alerts -->
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>
<div class="alert alert-warning">Please review your input.</div>

<!-- Forms -->
<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

Utility Classes

```
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding all around</div>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

You do not need Bootstrap or Tailwind. If you prefer those, you can replace `tina4.css` with your preferred framework -- just change the `<link>` tag.

11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

Requirements

1. Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`

2. Create a macro file at `src/templates/catalog-macros.twig` with:

• A `productCard(product)` macro that renders a styled card with name, category, price, stock status, and optional featured badge

• A `categoryFilter(categories, active)` macro that renders filter buttons

3. Create a page template at `src/templates/catalog.twig` that:

• Extends the base layout

• Uses the macros

• Shows a heading with the total product count

• Shows category filter buttons (All, and one per unique category)

• Shows product cards in a grid

• Shows featured products with a distinct style

• Handles the case when no products match the filter

4. Create a route at `GET /catalog` that accepts an optional `?category=` filter

Data

Use this product list in your route handler:

```
products = [
  { name: "Espresso Machine", category: "Kitchen", price: 299.99, in_stock: true, featured: true },
  { name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true, featured: false },
  { name: "Standing Desk", category: "Office", price: 549.99, in_stock: true, featured: true },
  { name: "Blender", category: "Kitchen", price: 89.99, in_stock: false, featured: false },
  { name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true, featured: false },
  { name: "Desk Lamp", category: "Office", price: 39.99, in_stock: true, featured: true },
  { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, in_stock: true, featured: false }
]
```

Test with:

```
http://localhost:7147/catalog
http://localhost:7147/catalog?category=Kitchen
http://localhost:7147/catalog?category=Fitness
```

12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .header p { margin: 4px 0 0; opacity: 0.8; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radius: 20px; tex
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 16p
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; padding: 16px; trans
        .product-card:hover { border-color: #adb5bd; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; margin: 0 0 8px; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.75em; font-weight:
```

```
        .badge-featured { background: #f39c12; color: white; }
# ...
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.in_stock %}
                <span class="badge badge-stock">In Stock</span>
            {% else %}
                <span class="badge badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active' : '' }}">All
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active' : '' }}">{{ ca
# ...
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_category %} in 
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
# ...
```

Create `src/routes/catalog.rb`:

```
Tina4::Router.get("/catalog") do |request, response|
```

```
  all_products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, in_stock: true, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, in_stock: true, featured: true },
    { name: "Blender", category: "Kitchen", price: 89.99, in_stock: false, featured: false },
    { name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true, featured: false },
    { name: "Desk Lamp", category: "Office", price: 39.99, in_stock: true, featured: true },
    { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, in_stock: true, featured: false }
  ]

  # Get unique categories
  categories = all_products.map { |p| p[:category] }.uniq.sort

  # Filter by category if specified
  active_category = request.query["category"] || ""
  products = if active_category.empty?
               all_products
             else
               all_products.select { |p| p[:category].downcase == active_category.downcase }
             end

  response.render("catalog.twig", {
    products: products,
    categories: categories,
# ...
```

Expected browser output for `/catalog`:

• A dark header with "Product Catalog" and "7 products"

• Filter buttons: All (active), Fitness, Kitchen, Office

• A grid of 7 product cards

• Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge

• The Blender card shows an "Out of Stock" badge in red

Expected browser output for `/catalog?category=Kitchen`:

• Header shows "3 products in Kitchen"

• The Kitchen filter button is active

• Only three cards: Espresso Machine, Blender, Cast Iron Skillet

13. Gotchas

1. `{% extends %}` Must Be the First Tag

Problem: Template inheritance does not work. The page renders without the base layout.

Cause: `{% extends "base.twig" %}` must be the very first tag in the template. If there is any text, whitespace, or even a comment before it, inheritance breaks.

Fix: Make `{% extends %}` the absolute first thing in the file. Move any `{% from %}` imports to after the extends tag.

2. Undefined Variables Show Nothing

Problem: `{{ username }}` renders as empty instead of showing an error.

Cause: Frond silently outputs nothing for undefined variables. This is by design (like Twig), but it can hide bugs.

Fix: Use the `default` filter: `{{ username | default("Guest") }}`. Or check first with `{% if username is defined %}`.

3. Auto-Escaping Prevents HTML Output

Problem: You pass HTML content like `"<strong>bold</strong>"` but it appears as literal text in the page.

Cause: Auto-escaping converts `<` to `<` and `>` to `>` for security.

Fix: If the content is trusted, use `{{ content | raw }}`. Never use `raw` on user-supplied input.

4. Variable Scope in Includes

Problem: A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

Cause: Loop variables are scoped to the loop. They do not leak into the outer scope.

Fix: If you need to accumulate values, use `{% set %}` before the loop and update it inside. Or restructure your template to keep all logic within the loop.

5. Macro Arguments Are Positional

Problem: Calling `{{ button("Click", style="danger") }}` does not work as expected.

Cause: Frond macros use positional arguments, not keyword arguments. The order matters.

Fix: Pass arguments in the order they are defined: `{{ button("Click", "/url", "danger") }}`. If you have many optional arguments, consider passing a single object.

6. Template File Extension Does Not Matter

Problem: You are not sure whether to use `.html`, `.twig`, or `.tpl`.

Cause: Frond does not care about the file extension. It processes any file in `src/templates/`.

Fix: Pick one extension and be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML files. Both work identically.

7. Filters Are Not Ruby Methods

Problem: You try `{{ items | count }}` or `{{ name | upcase }}` and get an error.

Cause: Frond filters are not Ruby methods. The filter names follow Twig conventions, not Ruby conventions.

Fix: Use `{{ items | length }}` instead of `count` or `size`, `{{ name | upper }}` instead of `upcase`, `{{ text | lower }}` instead of `downcase`. See the filter table in section 7 for the full list.

Chapter 5: Database

1. From Arrays to Real Data

In Chapters 2 and 3, all our data lived in Ruby arrays that reset every time the server restarted. That is fine for learning routing and responses, but real applications need persistent storage. This chapter covers Tina4's database layer -- raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 supports five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is the same across all of them. You switch databases by changing one line in `.env`.

2. Connecting to a Database

The Default: SQLite

When you scaffold a project with `tina4 init`, Tina4 creates a SQLite database at `data/app.db`. The default `.env` includes:

```
TINA4_DEBUG=true
```

With no explicit `DATABASE_URL`, Tina4 defaults to `sqlite:///data/app.db`. That is why the health check at `/health` already shows `"database": "connected"` without any configuration.

Connection Strings for Other Databases

Set `DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

Separate Credentials

If you prefer to keep credentials out of the connection string (recommended for production), use separate environment variables:

```
DATABASE_URL=postgres://localhost:5432/myapp
DATABASE_USERNAME=myuser
DATABASE_PASSWORD=secretpassword
```

Tina4 merges these with the connection string at startup. The credentials in the separate variables take precedence over any embedded in the URL.

Programmatic Connection

You can also create a database connection directly in Ruby code:

```
db = Tina4::Database.new("sqlite://app.db", username: nil, password: nil)
```

Verifying the Connection

After updating `.env`, restart the server and check:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
  "version": "3.0.0",
  "framework": "tina4-ruby"
}
```

If the database is not reachable, you will see `"database": "disconnected"` with an error message.

3. Getting the Database Object

In your route handlers, access the database via the `Tina4::Database` class:

```
Tina4::Router.get("/api/test-db") do |request, response|
  db = Tina4::Database.connection

  result = db.fetch("SELECT 1 + 1 AS answer")

  response.json(result)
end
```

```
curl http://localhost:7147/api/test-db
```

```
{"answer": 2}
```

`Tina4::Database.connection` returns the active database connection. You call methods like `fetch`, `execute`, and `fetch_one` on this object.

4. Raw Queries

fetch -- Get Multiple Rows

```
db = Tina4::Database.connection

# Returns an array of hashes
products = db.fetch("SELECT * FROM products WHERE price > 50")
```

Each row is a hash with column names as keys:

```
# products looks like:
[
  { "id" => 1, "name" => "Keyboard", "price" => 79.99 },
  { "id" => 4, "name" => "Standing Desk", "price" => 549.99 }
]
```

fetch_one -- Get a Single Row

```
product = db.fetch_one("SELECT * FROM products WHERE id = 1")
# Returns: { "id" => 1, "name" => "Keyboard", "price" => 79.99 }
```

If no row matches, `fetch_one` returns `nil`.

execute -- Run a Statement

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
db.execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)")
db.execute("UPDATE products SET price = 89.99 WHERE id = 1")
db.execute("DELETE FROM products WHERE id = 5")
db.execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at TEXT)")
```

Full Example: A Simple Query Route

```
Tina4::Router.get("/api/products") do |request, response|
  db = Tina4::Database.connection

  products = db.fetch("SELECT * FROM products ORDER BY name")

  response.json({
    products: products,
    count: products.length
  })
end
```

```
curl http://localhost:7147/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Keyboard", "price": 79.99, "in_stock": 1},
    {"id": 2, "name": "Mouse", "price": 29.99, "in_stock": 1},
    {"id": 3, "name": "Monitor", "price": 399.99, "in_stock": 0}
  ],
  "count": 3
}
```

5. Parameterised Queries

Never concatenate user input into SQL strings. This is how SQL injection attacks happen:

```
# NEVER do this:
db.fetch("SELECT * FROM products WHERE name = '#{user_input}'")
```

Instead, use parameterised queries. Pass parameters as the second argument:

```
db = Tina4::Database.connection

# Named parameters
product = db.fetch_one(
  "SELECT * FROM products WHERE id = :id",
  { id: 42 }
)

# Positional parameters
products = db.fetch(
  "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
  [10.00, 100.00]
)
```

The database driver handles escaping. Your input is never part of the SQL string.

A Safe Search Endpoint

```
Tina4::Router.get("/api/products/search") do |request, response|
```

```
  db = Tina4::Database.connection

  q = request.query["q"] || ""
  max_price = (request.query["max_price"] || 99999).to_f

  if q.empty?
    return response.json({ error: "Query parameter 'q' is required" }, 400)
  end

  products = db.fetch(
    "SELECT * FROM products WHERE name LIKE :query AND price <= :max_price ORDER BY name",
    { query: "%#{q}%", max_price: max_price }
  )

  response.json({
    query: q,
    max_price: max_price,
    results: products,
    count: products.length
  })
end
```

```
curl "http://localhost:7147/api/products/search?q=key&max_price=100"
```

```
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

6. Transactions

When you need multiple operations to succeed or fail together, use transactions:

```
Tina4::Router.post("/api/orders") do |request, response|
  db = Tina4::Database.connection
  body = request.body

  begin
    db.start_transaction

    # Create the order
    db.execute(
      "INSERT INTO orders (customer_id, total, status) VALUES (:customer_id, :total, 'pending')",
      { customer_id: body["customer_id"], total: body["total"] }
    )

    # Get the new order ID
    order = db.fetch_one("SELECT last_insert_rowid() AS id")
    order_id = order["id"]

    # Create order items
    body["items"].each do |item|
      db.execute(
        "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:order_id, :product_id, :qty,
        {
          order_id: order_id,
          product_id: item["product_id"],
          qty: item["quantity"],
```

```
# ...
```

If any step fails, `rollback` undoes everything. The database is never left in a half-finished state.

Important: You must call `commit` to save the changes. If you forget, the transaction will be rolled back when the connection closes.

7. Schema Inspection

Tina4 provides methods to inspect your database structure at runtime:

get_tables

```
db = Tina4::Database.connection
tables = db.get_tables
```

Returns an array of table names:

```
["orders", "order_items", "products", "users"]
```

get_columns

```
columns = db.get_columns("products")
```

Returns an array of column definitions:

```
[
  { "name" => "id", "type" => "INTEGER", "nullable" => false, "primary" => true },
  { "name" => "name", "type" => "TEXT", "nullable" => false, "primary" => false },
  { "name" => "price", "type" => "REAL", "nullable" => true, "primary" => false },
  { "name" => "in_stock", "type" => "INTEGER", "nullable" => true, "primary" => false }
]
```

table_exists?

```
if db.table_exists?("products")
  # Table exists, safe to query
end
```

A Schema Info Endpoint

```
Tina4::Router.get("/api/schema") do |request, response|
  db = Tina4::Database.connection
  tables = db.get_tables

  schema = {}
  tables.each do |table|
    schema[table] = db.get_columns(table)
  end

  response.json({ tables: schema })
end
```

8. Batch Operations with execute_many

Insert or update many rows efficiently:

```
db = Tina4::Database.connection

products = [
```

```
  { name: "Widget A", price: 9.99 },
  { name: "Widget B", price: 14.99 },
  { name: "Widget C", price: 19.99 },
  { name: "Widget D", price: 24.99 }
]

db.execute_many(
  "INSERT INTO products (name, price) VALUES (:name, :price)",
  products
)
```

`execute_many` prepares the statement once and executes it for each item in the array. This is significantly faster than calling `execute` in a loop because the SQL only needs to be parsed once.

9. Helper Methods: insert, update, delete

Tina4 provides shorthand methods so you do not have to write SQL for simple operations.

insert

```
db = Tina4::Database.connection

# Insert a single row
db.insert("products", {
  name: "Wireless Mouse",
  price: 34.99,
  in_stock: 1
})

# Insert multiple rows
db.insert("products", [
  { name: "USB Cable", price: 9.99, in_stock: 1 },
  { name: "HDMI Cable", price: 14.99, in_stock: 1 },
  { name: "DisplayPort Cable", price: 19.99, in_stock: 0 }
])
```

update

```
# Update rows matching a filter
db.update("products", { price: 39.99, in_stock: 1 }, "id = :id", { id: 7 })
```

The third argument is the WHERE clause, and the fourth is the parameters for it.

delete

```
# Delete rows matching a filter
db.delete("products", "id = :id", { id: 7 })
```

These helper methods generate the SQL for you. They are convenient for simple CRUD operations but do not replace raw queries for complex joins, subqueries, or aggregations.

10. Migrations

Migrations are versioned SQL scripts that evolve your database schema over time. Instead of manually running `CREATE TABLE` statements, you write migration files and Tina4 applies them in order.

Creating a Migration

Use the CLI:

```
tina4 migrate:create create_products_table
```

```
Created migration: src/migrations/20260322143000_create_products_table.sql
```

The filename starts with a timestamp so migrations always run in chronological order.

Edit the generated file `src/migrations/20260322143000_create_products_table.sql`:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The `-- UP` section runs when applying the migration. The `-- DOWN` section runs when rolling back.

Running Migrations

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_products_table.sql
Migrations complete. 1 applied.
```

Checking Migration Status

```
tina4 migrate:status
```

```
Migration                                       Status     Applied At
---------                                       ------     ----------
20260322143000_create_products_table.sql         applied    2026-03-22 14:30:00
20260322150000_create_orders_table.sql           pending    -
```

Rolling Back

```
tina4 migrate:rollback
```

```
Rolling back last migration...
  [ROLLED BACK] 20260322150000_create_orders_table.sql
Rollback complete. 1 rolled back.
```

This runs the `-- DOWN` section of the most recently applied migration.

A Real Migration Sequence

Here is what a typical project's migrations look like:

```
src/migrations/
■■■ 20260322143000_create_products_table.sql
■■■ 20260322143100_create_users_table.sql
■■■ 20260322143200_create_orders_table.sql
■■■ 20260322143300_create_order_items_table.sql
■■■ 20260323091500_add_email_index_to_users.sql
```

The last migration might look like:

```
-- UP
CREATE INDEX idx_users_email ON users (email);

-- DOWN
DROP INDEX IF EXISTS idx_users_email;
```

Migrations are applied in filename order. Each migration runs only once -- Tina4 tracks which ones have been applied in a `_migrations` table.

11. Query Caching

For read-heavy applications, enable query caching:

```
TINA4_DB_CACHE=true
```

When enabled, Tina4 caches the results of `fetch` and `fetch_one` calls. Identical queries with identical parameters return cached results instead of hitting the database again.

The cache is automatically invalidated when you call `execute`, `insert`, `update`, or `delete` on the same table.

You can also control caching per-query:

```
# Force a fresh query (bypass cache)
products = db.fetch("SELECT * FROM products", [], false) # third arg = use cache

# Clear the entire cache
db.clear_cache
```

12. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

Requirements

1. Create a migration that creates a `notes` table with columns:

• `id` -- integer, primary key, auto-increment

• `title` -- text, not null

• `content` -- text, not null

• `tag` -- text, default "general"

• `created_at` -- text, default current timestamp

• `updated_at` -- text, default current timestamp

2. Build these API endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/notes` | List all notes. Support `?tag=` and `?search=` filters. |

| `GET` | `/api/notes/{id:int}` | Get a single note. 404 if not found. |

| `POST` | `/api/notes` | Create a note. Validate title and content are not empty. |

| `PUT` | `/api/notes/{id:int}` | Update a note. 404 if not found. |

| `DELETE` | `/api/notes/{id:int}` | Delete a note. 204 on success, 404 if not found. |

Test with:

```
# Create
curl -X POST http://localhost:7147/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "tag": "personal"}'

# List all
curl http://localhost:7147/api/notes

# Search
curl "http://localhost:7147/api/notes?search=shopping"

# Filter by tag
curl "http://localhost:7147/api/notes?tag=personal"

# Get one
curl http://localhost:7147/api/notes/1

# Update
curl -X PUT http://localhost:7147/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7147/api/notes/1
```

13. Solution

Migration

Create `src/migrations/20260322143000_create_notes_table.sql`:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_notes_table.sql
Migrations complete. 1 applied.
```

Routes

Create `src/routes/notes.rb`:

```
# List all notes with optional filters
Tina4::Router.get("/api/notes") do |request, response|
  db = Tina4::Database.connection

  tag = request.query["tag"] || ""
  search = request.query["search"] || ""

  sql = "SELECT * FROM notes"
  params = {}
  conditions = []

  unless tag.empty?
    conditions << "tag = :tag"
    params[:tag] = tag
  end

  unless search.empty?
    conditions << "(title LIKE :search OR content LIKE :search)"
    params[:search] = "%#{search}%"
  end

  sql += " WHERE #{conditions.join(' AND ')}" unless conditions.empty?
  sql += " ORDER BY updated_at DESC"

  notes = db.fetch(sql, params)
# ...
```

Expected output for create:

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

(Status: `201 Created`)

Expected output for list:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for search:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
```

```
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for validation error:

```
{"errors": ["Title is required", "Content is required"]}
```

(Status: `400 Bad Request`)

14. Gotchas

1. Forgetting commit

Problem: You call `start_transaction`, run your queries, but the changes disappear on the next request.

Cause: Without `commit`, the transaction is rolled back when the connection closes.

Fix: Always call `db.commit` after your transaction succeeds. Use a `begin/rescue` block with `db.rollback` in the rescue.

2. Connection String Formats

Problem: The database will not connect and you see a cryptic error about the connection string.

Cause: Each database engine expects a specific URL format. A common mistake is using `mysql://user:pass@host/db` when the engine expects the port.

Fix: Always include the port: `mysql://localhost:3306/mydb`. Here are the default ports:

| Engine | Default Port |

|--------|-------------|

| PostgreSQL | 5432 |

| MySQL | 3306 |

| MSSQL | 1433 |

| Firebird | 3050 |

| SQLite | (file path, no port) |

3. SQLite File Paths

Problem: SQLite creates a new empty database instead of using the existing one.

Cause: The path in `DATABASE_URL` is relative and resolves to the wrong directory, or you used `sqlite://` (two slashes) instead of `sqlite:///` (three slashes).

Fix: Use three slashes for a relative path: `sqlite:///data/app.db`. For an absolute path, use four slashes: `sqlite:////var/data/app.db`. The third slash separates the scheme from the path; the fourth starts the absolute path.

4. Parameterised Queries with LIKE

Problem: `WHERE name LIKE :q` with `{ q: "%search%" }` works, but `WHERE name LIKE '%:q%'` does not.

Cause: Parameters inside quotes are treated as literal text, not as placeholders.

Fix: Include the `%` wildcards in the parameter value, not in the SQL: `{ q: "%#{search}%" }`. The SQL should be `WHERE name LIKE :q`.

5. Boolean Values in SQLite

Problem: You insert `true` or `false` but the database stores `1` or `0`. When you read it back, you get integers, not booleans.

Cause: SQLite does not have a native boolean type. It stores booleans as integers.

Fix: Cast in your Ruby code: `row["in_stock"] == 1` or use `!!row["in_stock"]` to convert to a boolean. Ruby treats `0` as truthy (unlike some other languages), so be explicit with comparisons.

6. Migration Already Applied

Problem: You edited a migration file and ran `tina4 migrate` again, but nothing changed.

Cause: Tina4 tracks applied migrations by filename. Once applied, a migration will not run again even if you change its contents.

Fix: Create a new migration for schema changes. Do not edit applied migrations. If you are in early development and want to start fresh, use `tina4 migrate:rollback` to undo and then `tina4 migrate` to reapply.

7. fetch Returns Empty Array, Not Nil

Problem: You check `if result.nil?` but it never matches, even when the table is empty.

Cause: `fetch` always returns an array. An empty result is `[]`, not `nil`. Only `fetch_one` returns `nil` when no row matches.

Fix: Check with `if result.empty?` or `if result.length == 0`.

Chapter 6: ORM

1. From SQL to Objects

In Chapter 5 you wrote raw SQL for every operation. That works, but it gets tedious. You end up writing the same `INSERT INTO products (name, price, ...) VALUES (:name, :price, ...)` patterns over and over. The ORM (Object-Relational Mapper) lets you work with Ruby classes instead. You define a class, map it to a table, and call methods like `save`, `load`, and `delete`.

Tina4's ORM is minimal by design. It does not try to hide SQL completely -- it gives you convenient methods for common operations and stays out of the way when you need raw queries.

2. Defining a Model

ORM models live in `src/orm/`. Every `.rb` file in that directory is auto-loaded, just like route files.

Create `src/orm/product.rb`:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :category, default: "Uncategorized"
  float_field :price, default: 0.00
  boolean_field :in_stock, default: true
  string_field :created_at
  string_field :updated_at

  table_name "products"
end
```

That is a complete model. Let us break it down:

• **Extends `Tina4::ORM`** -- This gives you `save`, `load`, `delete`, `select`, and other methods.

• **Field declarations** -- Each field maps to a database column. The field name uses `snake_case` which maps directly to the column name. Tina4 handles the conversion automatically.

• **`table_name`** -- The database table this model maps to. If you omit it, Tina4 infers it from the class name: `Product` becomes `products`, `OrderItem` becomes `order_items`.

• **`primary_key: true`** -- Marks the primary key column. Defaults to `id`.

• **Default values** -- Fields with defaults (like `category: "Uncategorized"`) are used when creating new records without specifying those fields.

3. Field Types

Use the appropriate field declaration for your data types:

| Ruby Declaration | Database Type (SQLite) | Database Type (PostgreSQL) | Notes |

|----------|----------|----------|----------|

| `integer_field` | INTEGER | INTEGER | Whole numbers |

| `string_field` | TEXT | VARCHAR(255) | Text fields |

| `float_field` | REAL | DOUBLE PRECISION | Decimal numbers |

| `boolean_field` | INTEGER | BOOLEAN | SQLite stores as 0/1 |

Nullable Fields

Fields are nullable by default. To require a value:

```
string_field :name, required: true
string_field :description  # nullable by default
```

Primary Keys and Auto-Increment

By default, Tina4 treats the primary key field as auto-incrementing. When you call `save` on a new object (where the primary key is not set), the database generates the ID:

```
product = Product.new
product.name = "Widget"
product.price = 9.99
product.save

puts product.id  # Auto-generated: 1, 2, 3, ...
```

4. Creating and Saving Records

save -- Insert or Update

The `save` method inserts a new record or updates an existing one, depending on whether the primary key is set:

```
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  product = Product.new
  product.name = body["name"]
  product.category = body["category"] || "Uncategorized"
  product.price = (body["price"] || 0).to_f
  product.in_stock = body["in_stock"] != false
  product.save

  response.json(product.to_h, 201)
end
```

```
curl -X POST http://localhost:7147/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Wireless Keyboard", "category": "Electronics", "price": 79.99}'
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

Updating an Existing Record

When `id` is already set, `save` performs an UPDATE:

```
Tina4::Router.put("/api/products/{id:int}") do |request, response|
  product = Product.new
  product.load(request.params["id"])

  if product.id.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  body = request.body
  product.name = body["name"] || product.name
  product.price = (body["price"] || product.price).to_f
  product.category = body["category"] || product.category
  product.save

  response.json(product.to_h)
end
```

5. Loading Records

load -- Get by Primary Key

```
product = Product.new
product.load(42)

if product.id.nil?
  # Product with ID 42 not found
end
```

`load` populates the object's properties from the database row matching the primary key. If no row matches, the properties remain at their default values (or nil).

A Simple Get Endpoint

```
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  product = Product.new
  product.load(request.params["id"])

  if product.id.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  response.json(product.to_h)
end
```

```
curl http://localhost:7147/api/products/1
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

6. Deleting Records

delete

```
Tina4::Router.delete("/api/products/{id:int}") do |request, response|
  product = Product.new
  product.load(request.params["id"])

  if product.id.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  product.delete

  response.json(nil, 204)
end
```

`delete` removes the row from the database. The object still exists in memory but the database row is gone.

7. Querying with select

The `select` method lets you find records with filters, ordering, and pagination:

Basic Select

```
product = Product.new
products = product.select("*")
```

Returns an array of Product objects with all records.

Filtering

```
product = Product.new

# Simple filter
electronics = product.select("*", "category = :category", { category: "Electronics" })

# Multiple conditions
affordable = product.select("*", "price < :max_price AND in_stock = :in_stock", {
  max_price: 100,
  in_stock: 1
})
```

Ordering

```
product = Product.new
sorted = product.select("*", "", {}, "price DESC")
```

The fourth argument is the ORDER BY clause.

Pagination

```
product = Product.new

page = 1
per_page = 10
offset = (page - 1) * per_page

products = product.select("*", "", {}, "name ASC", per_page, offset)
```

The fifth argument is LIMIT, the sixth is OFFSET.

A Full List Endpoint with Filters

```
Tina4::Router.get("/api/products") do |request, response|
  product = Product.new

  category = request.query["category"] || ""
  min_price = (request.query["min_price"] || 0).to_f
  max_price = (request.query["max_price"] || 999999).to_f
  page = (request.query["page"] || 1).to_i
  per_page = (request.query["per_page"] || 20).to_i
  sort = request.query["sort"] || "name"
  order = (request.query["order"] || "ASC").upcase

  # Build filter
  conditions = []
  params = {}

  unless category.empty?
    conditions << "category = :category"
    params[:category] = category
  end

  conditions << "price >= :min_price AND price <= :max_price"
  params[:min_price] = min_price
  params[:max_price] = max_price

  filter = conditions.join(" AND ")
# ...
```

```
curl "http://localhost:7147/api/products?category=Electronics&sort=price&order=DESC&page=1&per_page=5"
```

```
{
  "products": [
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true}
  ],
  "page": 1,
  "per_page": 5,
  "count": 2
}
```

8. Creating Tables from Models

Instead of writing a migration manually, you can generate the table from your model:

```
product = Product.new
product.create_table
```

You can also use the CLI:

```
tina4 orm:create-table Product
```

```
Created table "products" with 7 columns.
```

This is convenient during early development. For production, use migrations (Chapter 5) so schema changes are versioned and reversible.

9. Relationships

has_many -- One-to-Many

A user has many posts:

Create `src/orm/user.rb`:

```
class User < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :email
  string_field :created_at

  table_name "users"

  has_many :posts, class_name: "Post", foreign_key: "user_id"
end
```

Create `src/orm/post.rb`:

```
class Post < Tina4::ORM
  integer_field :id, primary_key: true
  integer_field :user_id
  string_field :title
  string_field :body
  string_field :created_at

  table_name "posts"

  belongs_to :user, class_name: "User", foreign_key: "user_id"
  has_many :comments, class_name: "Comment", foreign_key: "post_id"
end
```

The `foreign_key` option specifies the column on the related table. `has_many :posts, class_name: "Post", foreign_key: "user_id"` means: find all rows in `posts` where `user_id` equals this user's ID.

has_one -- One-to-One

```
has_one :profile, class_name: "Profile", foreign_key: "user_id"
```

`has_one` works like `has_many` but returns a single object instead of an array.

belongs_to -- Inverse Relationship

The inverse of `has_many`. A post belongs to a user:

```
belongs_to :user, class_name: "User", foreign_key: "user_id"
```

`belongs_to :user, class_name: "User", foreign_key: "user_id"` means: load the User where `users.id` equals `self.user_id`.

Using Relationships

```
Tina4::Router.get("/api/users/{id:int}") do |request, response|
  user = User.new
  user.load(request.params["id"])

  if user.id.nil?
    return response.json({ error: "User not found" }, 404)
  end

  posts = user.posts

  response.json({
    user: user.to_h,
    posts: posts.map(&:to_h),
    post_count: posts.length
```

```
  })
end
```

```
curl http://localhost:7147/api/users/1
```

```
{
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"},
  "posts": [
    {"id": 1, "user_id": 1, "title": "First Post", "body": "Hello world!"},
    {"id": 3, "user_id": 1, "title": "Second Post", "body": "Another one."}
  ],
  "post_count": 2
}
```

10. Eager Loading

Calling relationship methods inside a loop creates the N+1 query problem. If you load 100 users and then call `user.posts` for each one, that is 101 queries (1 for users + 100 for posts).

Use the `include` parameter with `select` to eager-load relationships:

```
user = User.new
users = user.select("*", "", {}, "name ASC", 20, 0, ["posts"])
```

The seventh argument is an array of relationship names to include. This runs just 2 queries (one for users, one for all related posts) and stitches the results together.

to_h with Nested Includes

When eager loading is active, `to_h` includes the related data:

```
user = User.new
users = user.select("*", "", {}, "", 0, 0, ["posts"])

result = users.map(&:to_h)

response.json(result)
```

```
[
  {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "posts": [
      {"id": 1, "title": "First Post", "body": "Hello world!"},
      {"id": 3, "title": "Second Post", "body": "Another one."}
    ]
  },
  {
    "id": 2,
    "name": "Bob",
    "email": "bob@example.com",
    "posts": [
      {"id": 2, "title": "Bob's Post", "body": "Hi there."}
    ]
  }
]
```

Nested Eager Loading

Load multiple levels deep with dot notation:

```
user = User.new
users = user.select("*", "", {}, "", 0, 0, ["posts", "posts.comments"])
```

This loads users, their posts, and each post's comments in 3 queries total.

11. Soft Delete

If your model has a `deleted_at` field, Tina4 supports soft delete -- marking records as deleted without actually removing them from the database:

```
class Post < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :title
  string_field :body
  string_field :deleted_at

  table_name "posts"

  soft_delete true
end
```

With `soft_delete true`:

- `post.delete` sets `deleted_at` to the current timestamp instead of deleting the row
- `select` automatically excludes rows where `deleted_at` is not null
- `post.force_delete` permanently removes the row

Restoring Soft-Deleted Records

```
post = Post.new
post.load(5)  # Load even if soft-deleted
post.deleted_at = nil
post.save
```

Including Soft-Deleted Records in Queries

```
post = Post.new
all_posts = post.select("*", "", {}, "", 0, 0, [], true)  # eighth arg = include deleted
```

12. Auto-CRUD

One of Tina4's most powerful features is auto-CRUD -- it automatically generates REST endpoints for any ORM model.

Add the `auto_crud` declaration to your model:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :category, default: "Uncategorized"
  float_field :price, default: 0.00
  boolean_field :in_stock, default: true

  table_name "products"

  auto_crud true
end
```

With `auto_crud true`, Tina4 automatically registers these routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all with pagination |

| `GET` | `/api/products/{id}` | Get one by ID |

| `POST` | `/api/products` | Create a new record |

| `PUT` | `/api/products/{id}` | Update a record |

| `DELETE` | `/api/products/{id}` | Delete a record |

The endpoint prefix is derived from the table name: `products` becomes `/api/products`.

```
curl http://localhost:7147/api/products
```

```
{
  "data": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": true}
  ],
  "total": 2,
  "page": 1,
  "per_page": 20
}
```

Auto-CRUD supports query parameters for filtering, sorting, and pagination out of the box:

```
curl "http://localhost:7147/api/products?category=Electronics&sort=price&order=desc&page=1&per_page=10"
```

You can still define custom routes alongside auto-CRUD. Your custom routes take precedence over the auto-generated ones.

13. Exercise: Build a Blog

Build a blog with three models: User, Post, and Comment. Use relationships, eager loading, and auto-CRUD.

Requirements

1. Create three models in `src/orm/`:

User -- `users` table:

• `id` (integer, primary key)

• `name` (string)

• `email` (string)

• `created_at` (string)

• Has many posts

Post -- `posts` table:

• `id` (integer, primary key)

• `user_id` (integer, foreign key)

• `title` (string)

• `body` (string)

• `published` (boolean, default false)

• `created_at` (string)

• Belongs to user, has many comments

Comment -- `comments` table:

• `id` (integer, primary key)

• `post_id` (integer, foreign key)

• `author_name` (string)

• `body` (string)

• `created_at` (string)

• Belongs to post

2. Create migrations for all three tables.

3. Build custom endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/blog/posts` | List published posts with author info (eager load user) |

| `GET` | `/api/blog/posts/{id:int}` | Get a post with author and comments (eager load both) |

| `POST` | `/api/blog/posts/{id:int}/comments` | Add a comment to a post |

4. Enable auto-CRUD on the User model for admin access at `/api/users`.

Test with:

```
# Create a user (via auto-CRUD)
curl -X POST http://localhost:7147/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'

# Create a post
curl -X POST http://localhost:7147/api/blog/posts \
  -H "Content-Type: application/json" \
  -d '{"user_id": 1, "title": "My First Post", "body": "Hello world!", "published": true}'

# List posts
curl http://localhost:7147/api/blog/posts

# Add a comment
curl -X POST http://localhost:7147/api/blog/posts/1/comments \
  -H "Content-Type: application/json" \
  -d '{"author_name": "Bob", "body": "Great post!"}'

# Get post with comments
curl http://localhost:7147/api/blog/posts/1
```

14. Solution

Models

Create `src/orm/user.rb`:

```
class User < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :email
  string_field :created_at

  table_name "users"

  auto_crud true

  has_many :posts, class_name: "Post", foreign_key: "user_id"
end
```

Create `src/orm/post.rb`:

```
class Post < Tina4::ORM
  integer_field :id, primary_key: true
  integer_field :user_id
  string_field :title
  string_field :body
  boolean_field :published, default: false
  string_field :created_at

  table_name "posts"

  belongs_to :user, class_name: "User", foreign_key: "user_id"
  has_many :comments, class_name: "Comment", foreign_key: "post_id"
end
```

Create `src/orm/comment.rb`:

```
class Comment < Tina4::ORM
  integer_field :id, primary_key: true
  integer_field :post_id
  string_field :author_name
  string_field :body
  string_field :created_at

  table_name "comments"

  belongs_to :post, class_name: "Post", foreign_key: "post_id"
end
```

Routes

Create `src/routes/blog.rb`:

```
# List published posts with author
Tina4::Router.get("/api/blog/posts") do |request, response|
  post = Post.new
  posts = post.select("*", "published = :published", { published: 1 }, "created_at DESC", 0, 0, ["user"])

  results = posts.map(&:to_h)

  response.json({
    posts: results,
    count: results.length
  })
```

```
end

# Get a single post with author and comments
Tina4::Router.get("/api/blog/posts/{id:int}") do |request, response|
  post = Post.new
  post.load(request.params["id"])

  if post.id.nil?
    return response.json({ error: "Post not found" }, 404)
  end

  user = post.user
  comments = post.comments

# ...
```

Expected output for GET /api/blog/posts/1:

```
{
  "id": 1,
  "user_id": 1,
  "title": "My First Post",
  "body": "Hello world!",
  "published": true,
  "created_at": "2026-03-22 15:00:00",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  },
  "comments": [
    {
      "id": 1,
      "post_id": 1,
      "author_name": "Bob",
      "body": "Great post!",
      "created_at": "2026-03-22 15:01:00"
    }
  ],
  "comment_count": 1
}
```

15. Gotchas

1. Table Naming Convention

Problem: Your model class is `OrderItem` but queries fail because the table does not exist.

Cause: Tina4 converts `OrderItem` to `order_items` (plural, snake_case). If your table is named `order_item` (singular), it will not match.

Fix: Set `table_name` explicitly: `table_name "order_item"`. Or rename your table to match the convention.

2. Nil Handling

Problem: A field that should be nullable causes errors when the value is nil.

Cause: Ruby is generally nil-friendly, but the database column might have a NOT NULL constraint.

Fix: Ensure your migration allows null values for optional fields. In Ruby, check with `field.nil?` before accessing methods on potentially nil values.

3. Relationship Foreign Key Direction

Problem: You write `has_many :posts, foreign_key: "id"` and get wrong results.

Cause: The foreign key argument is the column on the related table, not the current table. `has_many :posts, foreign_key: "user_id"` means "find posts where posts.user_id = this.id", not "find posts where posts.id = this.user_id".

Fix: The foreign key is always on the "many" side. For `has_many`, it is the column on the child table. For `belongs_to`, it is the column on the current table.

4. snake_case Mapping

Problem: You have a field `user_id` but the database column is `userId`. Queries return nil for this field.

Cause: Tina4 Ruby uses `snake_case` for both field names and database columns. If your database uses `camelCase`, the mapping breaks.

Fix: Use consistent naming. Ruby fields and database columns should both be `snake_case`. If your column names differ, you may need to adjust them or override the mapping.

5. Forgetting save

Problem: You set properties on a model but the database does not change.

Cause: You forgot to call `model.save`. Setting properties only changes the in-memory object.

Fix: Always call `save` after modifying properties that should be persisted.

6. Auto-CRUD Endpoint Conflicts

Problem: Your custom route at `/api/products/{id}` does not work after enabling auto-CRUD on the Product model.

Cause: Both your custom route and the auto-CRUD route match the same path. The first one registered wins.

Fix: Custom routes defined in `src/routes/` files are loaded before auto-CRUD routes, so they take precedence. If that is not the behavior you want, use a different path for your custom route (e.g., `/api/shop/products/{id}`).

7. select Returns Objects, Not Hashes

Problem: You try to use hash syntax (`result["name"]`) on the result of `select` and get an error.

Cause: `select` returns an array of model objects, not hashes. Each item is an instance of your model class.

Fix: Access properties with dot syntax: `result.name`. Or convert to a hash with `result.to_h`.

Chapter 7: Authentication

1. Locking the Door

Up to now, every endpoint you have built is public. Anyone with the URL can read, create, update, and delete data. That is fine for a tutorial, but a real application needs to know who is making a request and whether they are allowed to make it.

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string that contains a payload (like a user ID and role). The server creates the token at login, the client sends it with every request, and the server verifies it without needing to look anything up in a database.

Generating a Token

```
payload = {
  user_id: 42,
  email: "alice@example.com",
  role: "admin"
}

token = Tina4::Auth.create_token(payload)
```

`create_token` signs the payload with a secret key and returns a JWT string like:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pb
```


The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

Token Expiry

By default, tokens expire after 1 hour. Configure this in `.env`:

```
TINA4_JWT_EXPIRY=3600
```

The value is in seconds. Common settings:

| Value | Duration |

|-------|----------|

| `900` | 15 minutes |

| `3600` | 1 hour (default) |

| `86400` | 24 hours |

| `604800` | 7 days |

Validating a Token

```
is_valid = Tina4::Auth.validate_token(token)
# Returns true if the token is valid and not expired, false otherwise
```

Reading the Payload

```
payload = Tina4::Auth.get_payload(token)
```

Returns the original payload hash:

```
{
  "user_id" => 42,
  "email" => "alice@example.com",
  "role" => "admin",
  "iat" => 1711112600,  # issued at (Unix timestamp)
  "exp" => 1711116200   # expires at (Unix timestamp)
}
```

If the token is invalid or expired, `get_payload` returns `nil`.

The Secret Key

Tina4 uses a secret key to sign and verify tokens. On first run, it automatically generates a random key and stores it in `secrets/jwt.key`. You can also set it explicitly:

```
TINA4_JWT_SECRET=my-very-long-and-random-secret-key-at-least-32-chars
```

Keep this key secret. If someone gets it, they can forge tokens.

3. Password Hashing

Never store passwords in plain text. Tina4 provides two methods for secure password handling:

Hashing a Password

```
hash = Tina4::Auth.hash_password("my-secure-password")
# Returns: "$2a$12$abc123...long-hash-string..."
```

This uses bcrypt under the hood. Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
is_correct = Tina4::Auth.check_password("my-secure-password", stored_hash)
# Returns true if the password matches the hash
```

Registration Example

```
# @noauth
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  # Validate input
  if body["name"].nil? || body["email"].nil? || body["password"].nil?
    return response.json({ error: "Name, email, and password are required" }, 400)
  end

  if body["password"].length < 8
    return response.json({ error: "Password must be at least 8 characters" }, 400)
  end

  db = Tina4::Database.connection

  # Check if email already exists
  existing = db.fetch_one("SELECT id FROM users WHERE email = :email", { email: body["email"] })
  unless existing.nil?
```

```
    return response.json({ error: "Email already registered" }, 409)
  end

  # Hash the password
  password_hash = Tina4::Auth.hash_password(body["password"])

  # Create the user
# ...
```

```
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

4. The Login Flow

Here is the complete login flow: the client sends credentials, the server validates them, and returns a JWT token.

```
# @noauth
Tina4::Router.post("/api/login") do |request, response|
  body = request.body

  if body["email"].nil? || body["password"].nil?
    return response.json({ error: "Email and password are required" }, 400)
  end

  db = Tina4::Database.connection

  # Find the user
  user = db.fetch_one(
    "SELECT id, name, email, password_hash FROM users WHERE email = :email",
    { email: body["email"] }
  )

  if user.nil?
    return response.json({ error: "Invalid email or password" }, 401)
  end

  # Check the password
  unless Tina4::Auth.check_password(body["password"], user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

# ...
```

Notice the `@noauth` comment. The login endpoint must be public -- you cannot require a token to get a token.

```
curl -X POST http://localhost:7147/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
```

```
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
def auth_middleware(request, response, next_handler)
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization header required" }, 401)
  end

  token = auth_header.sub("Bearer ", "")

  unless Tina4::Auth.validate_token(token)
    return response.json({ error: "Invalid or expired token" }, 401)
  end

  payload = Tina4::Auth.get_payload(token)
  request.user = payload  # Attach user data to the request

  next_handler.call(request, response)
end
```

Applying Middleware to Routes

```
Tina4::Router.get("/api/profile", middleware: "auth_middleware") do |request, response|
  response.json({
    user_id: request.user["user_id"],
    email: request.user["email"],
    name: request.user["name"]
  })
end
```

```
# Without token -- 401
curl http://localhost:7147/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user_id": 1,
```

```
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
Tina4::Router.group("/api/admin", middleware: "auth_middleware") do

  Tina4::Router.get("/stats") do |request, response|
    response.json({ active_users: 42 })
  end

  Tina4::Router.get("/logs") do |request, response|
    response.json({ logs: [] })
  end

end
```

Every route in the group requires a valid token.

7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
# @noauth
Tina4::Router.get("/api/public/health") do |request, response|
  response.json({ status: "ok" })
end

# @noauth
Tina4::Router.post("/api/login") do |request, response|
  # Login logic
end

# @noauth
Tina4::Router.post("/api/register") do |request, response|
  # Registration logic
end
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
# @secured
Tina4::Router.get("/api/me") do |request, response|
  # This GET route requires authentication
  response.json(request.user)
end
```

Role-Based Authorization

Combine auth middleware with role checks:

```
def require_role(role)
  lambda do |request, response, next_handler|
    # First check authentication
    auth_header = request.headers["Authorization"] || ""
```

```
    if auth_header.empty? || !auth_header.start_with?("Bearer ")
      return response.json({ error: "Authorization required" }, 401)
    end

    token = auth_header.sub("Bearer ", "")
    unless Tina4::Auth.validate_token(token)
      return response.json({ error: "Invalid or expired token" }, 401)
    end

    payload = Tina4::Auth.get_payload(token)

    # Check role
    if (payload["role"] || "") != role
      return response.json({ error: "Forbidden. Required role: #{role}" }, 403)
    end

    request.user = payload
    next_handler.call(request, response)
  end
end
```

8. CSRF Protection

For traditional form-based applications (not SPAs), Tina4 provides CSRF protection with form tokens.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
Tina4::Router.post("/profile/update") do |request, response|
  # Validate CSRF token
  unless Tina4::Auth.validate_form_token(request.body["_token"] || "")
    return response.json({ error: "Invalid form token. Please refresh and try again." }, 403)
  end

  # Process the form...
  response.redirect("/profile")
end
```

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. Sessions work alongside JWT tokens -- use JWTs for API authentication and sessions for stateful web pages.

Using Sessions

Access session data via `request.session`:

```
Tina4::Router.post("/login-form") do |request, response|
  # After validating credentials...
  request.session["user_id"] = 42
  request.session["user_name"] = "Alice"
  request.session["logged_in"] = true

  response.redirect("/dashboard")
end

Tina4::Router.get("/dashboard") do |request, response|
  if request.session["logged_in"].nil?
    return response.redirect("/login")
  end

  response.render("dashboard.html", {
    user_name: request.session["user_name"]
  })
end

Tina4::Router.post("/logout") do |request, response|
  # Clear all session data
  request.session.clear

  response.redirect("/login")
end
```

10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

1. Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`

2. Build these endpoints:

| Method | Path | Auth | Description |

|--------|------|------|-------------|

| `POST` | `/api/register` | @noauth | Create an account. Validate name, email, password (min 8 chars). |

| `POST` | `/api/login` | @noauth | Login. Return JWT token. |

| `GET` | `/api/profile` | secured | Get current user's profile from token. |

| `PUT` | `/api/profile` | secured | Update name and email. |

| `PUT` | `/api/profile/password` | secured | Change password. Require current password. |

3. Create auth middleware that extracts the user from the JWT and attaches it to `request.user`.

Test with:

```
# Register
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7147/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7147/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7147/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
# ...
```

11. Solution

Migration

Create `src/migrations/20260322160000_create_auth_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

Middleware

Create `src/routes/middleware.rb`:

```
def auth_middleware(request, response, next_handler)
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization required. Send: Authorization: Bearer <token>" }, 401)
  end

  token = auth_header.sub("Bearer ", "")

  unless Tina4::Auth.validate_token(token)
    return response.json({ error: "Invalid or expired token. Please login again." }, 401)
  end
```

```
  payload = Tina4::Auth.get_payload(token)
  request.user = payload

  next_handler.call(request, response)
end
```

Routes

Create `src/routes/auth.rb`:

```
# @noauth
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  errors = []
  errors << "Name is required" if body["name"].nil? || body["name"].empty?
  errors << "Email is required" if body["email"].nil? || body["email"].empty?
  if body["password"].nil? || body["password"].empty?
    errors << "Password is required"
  elsif body["password"].length < 8
    errors << "Password must be at least 8 characters"
  end

  unless errors.empty?
    return response.json({ errors: errors }, 400)
  end

  db = Tina4::Database.connection

  existing = db.fetch_one("SELECT id FROM users WHERE email = :email", { email: body["email"] })
  unless existing.nil?
    return response.json({ error: "Email already registered" }, 409)
  end

  hash = Tina4::Auth.hash_password(body["password"])
# ...
```

Expected output for register:

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

Expected output for successful password change:

```
{"message":"Password changed successfully"}
```

12. Gotchas

1. Token Expiry Confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 1 hour. After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret Key Management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted/regenerated during deployment.

Fix: Set `TINA4_JWT_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with Authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

4. Storing Tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on Login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` annotation to bypass it.

Fix: Add `# @noauth` before your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password Hash Column Too Short

Problem: Registration fails with a database error about the password hash being too long.

Cause: bcrypt hashes are 60 characters long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL Query Parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Chapter 8: Middleware

1. The Gatekeepers

Your API needs CORS headers for the React frontend, rate limiting for the public endpoints, and auth checking for admin routes -- all without cluttering your route handlers. You could copy-paste the same ten lines of CORS code into every handler, but that breaks the moment you forget one. You could pile all the checks into a giant `if` tree at the top of each handler, but that buries the actual business logic under boilerplate.

Middleware solves this. It lets you wrap routes with reusable logic that runs before (or after) the handler. Each middleware does one job -- check a token, set CORS headers, log the request, enforce rate limits -- and passes control to the next layer. Your route handlers stay focused on their actual purpose.

In Chapter 2 you saw a brief introduction to middleware. This chapter goes deep: built-in middleware, custom middleware, execution order, short-circuiting, and real-world patterns.

2. What Middleware Is

A middleware function sits between the incoming HTTP request and your route handler. It receives the request, the response, and a `next_handler` callable. It can:

1. Inspect or modify the request before passing it along

2. Decide not to pass it along (short-circuit)

3. Inspect or modify the response on the way back

Here is the simplest middleware that does nothing but pass through:

```
def passthrough(request, response, next_handler)
  next_handler.call(request, response)
end
```

And here is one that blocks everything:

```
def block_everything(request, response, next_handler)
  response.json({ error: "Service unavailable" }, 503)
end
```

The `next_handler` is what makes middleware composable. Call it to continue to the next middleware (or the route handler if there is no more middleware). Do not call it to stop the chain.

3. Built-in CorsMiddleware

Cross-Origin Resource Sharing (CORS) is the browser mechanism that controls which domains can call your API. Tina4 provides a built-in `CorsMiddleware` that handles this. Configure it in `.env`:

```
CORS_ORIGINS=http://localhost:3000,https://myapp.com
CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
CORS_HEADERS=Content-Type,Authorization,X-API-Key
CORS_MAX_AGE=86400
CORS_CREDENTIALS=true
```

Apply it to a group:

```
Tina4::Router.group("/api", middleware: "CorsMiddleware") do

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

  Tina4::Router.post("/products") do |request, response|
    response.json({ created: true }, 201)
  end

end
```

4. Built-in RateLimiter

Rate limiting prevents a single client from overwhelming your API. Configure it in `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_LIMIT_WINDOW=60
```

This means 60 requests per 60 seconds per IP. Apply it:

```
Tina4::Router.group("/api/public", middleware: "RateLimiter") do

  Tina4::Router.get("/search") do |request, response|
    q = request.query["q"] || ""
    response.json({ query: q, results: [] })
  end

end
```

When a client exceeds the limit:

```
{"error":"Rate limit exceeded. Try again in 42 seconds.","retry_after":42}
```

Custom Limits Per Group

```
# Public endpoints: 30 requests per minute
Tina4::Router.group("/api/public", middleware: "RateLimiter:30") do
  Tina4::Router.get("/search") do |request, response|
    response.json({ results: [] })
  end
end

# Authenticated endpoints: 120 requests per minute
Tina4::Router.group("/api/v1", middleware: ["auth_middleware", "RateLimiter:120"]) do
  Tina4::Router.get("/data") do |request, response|
    response.json({ data: [] })
  end
end
```

5. Writing Custom Middleware

Custom middleware follows the same pattern: receive `request`, `response`, and `next_handler`. Define it as a named method.

Request Logging Middleware

```
def log_request(request, response, next_handler)
```

```
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  method = request.method
  path = request.path
  ip = request.ip

  $stderr.puts "[#{Time.now.strftime('%Y-%m-%d %H:%M:%S')}] #{method} #{path} from #{ip}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  $stderr.puts "  Completed in #{duration}ms"

  result
end
```

Request Timing Middleware

```
def add_timing(request, response, next_handler)
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  response.header("X-Response-Time", "#{duration}ms")

  result
end
```

IP Whitelist Middleware

```
def ip_whitelist(request, response, next_handler)
  allowed_ips = (ENV["ALLOWED_IPS"] || "127.0.0.1").split(",")

  unless allowed_ips.include?(request.ip)
    return response.json({
      error: "Access denied",
      your_ip: request.ip
    }, 403)
  end

  next_handler.call(request, response)
end
```

Request Validation Middleware

```
def require_json(request, response, next_handler)
  if %w[POST PUT PATCH].include?(request.method)
    content_type = request.headers["Content-Type"] || ""

    unless content_type.include?("application/json")
      return response.json({
        error: "Content-Type must be application/json",
        received: content_type
      }, 415)
    end
  end

  next_handler.call(request, response)
end
```

6. Applying Middleware to Individual Routes

Pass middleware as a keyword argument to any route method:

```
Tina4::Router.get("/api/data", middleware: "log_request") do |request, response|
  response.json({ data: [1, 2, 3] })
end

Tina4::Router.post("/api/data", middleware: ["log_request", "require_json"]) do |request, response|
  response.json({ created: true }, 201)
end
```

For a single middleware, pass a string. For multiple, pass an array. Each middleware runs in the order listed.

7. Route Groups with Shared Middleware

Groups apply middleware to every route inside them:

```
# Public API -- rate limited, CORS enabled
Tina4::Router.group("/api/public", middleware: ["CorsMiddleware", "RateLimiter:30"]) do

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

  Tina4::Router.get("/categories") do |request, response|
    response.json({ categories: [] })
  end

end

# Admin API -- auth required, IP restricted, logged
Tina4::Router.group("/api/admin", middleware: ["log_request", "ip_whitelist", "auth_middleware"]) do

  Tina4::Router.get("/users") do |request, response|
    response.json({ users: [] })
  end

  Tina4::Router.delete("/users/{id:int}") do |request, response|
    id = request.params["id"]
    response.json({ deleted: id })
  end

# ...
```

8. Middleware Execution Order

When you stack middleware, they execute from outer to inner -- like layers of an onion. The first middleware listed runs first on the way in and last on the way out.

Consider this setup:

```
def middleware_a(request, response, next_handler)
  $stderr.puts "A: before"
  result = next_handler.call(request, response)
  $stderr.puts "A: after"
  result
end

def middleware_b(request, response, next_handler)
```

```
  $stderr.puts "B: before"
  result = next_handler.call(request, response)
  $stderr.puts "B: after"
  result
end

def middleware_c(request, response, next_handler)
  $stderr.puts "C: before"
  result = next_handler.call(request, response)
  $stderr.puts "C: after"
  result
end

Tina4::Router.get("/api/test", middleware: ["middleware_a", "middleware_b", "middleware_c"]) do |request, resp
  $stderr.puts "Handler"
  response.json({ ok: true })
end
```

Server log output:

```
A: before
B: before
C: before
Handler
C: after
B: after
A: after
```

The request flows inward: A, B, C, Handler. The response flows outward: C, B, A.

9. Short-Circuiting

When middleware does not call `next_handler`, the chain stops. No subsequent middleware runs and the route handler is never called.

Maintenance Mode

```
def maintenance_mode(request, response, next_handler)
  is_maintenance = (ENV["MAINTENANCE_MODE"] || "false") == "true"

  if is_maintenance
    if request.path == "/health"
      return next_handler.call(request, response)
    end

    return response.json({
      error: "Service is undergoing maintenance",
      retry_after: 300
    }, 503)
  end

  next_handler.call(request, response)
end
```

10. Modifying Requests in Middleware

Middleware can add data to the request before passing it to the handler:

```
def add_request_id(request, response, next_handler)
```

```
  request_id = SecureRandom.hex(8)
  request.request_id = request_id

  result = next_handler.call(request, response)

  response.header("X-Request-Id", request_id)

  result
end
```

11. Real-World Middleware Stack

Here is a realistic middleware setup for a production API:

```
# src/routes/middleware.rb

def add_request_id(request, response, next_handler)
  request.request_id = SecureRandom.hex(8)
  result = next_handler.call(request, response)
  response.header("X-Request-Id", request.request_id)
  result
end

def log_request(request, response, next_handler)
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  $stderr.puts "[#{request.request_id}] #{request.method} #{request.path}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  $stderr.puts "[#{request.request_id}] Completed in #{duration}ms"

  result
end

def require_api_key(request, response, next_handler)
  api_key = request.headers["X-API-Key"] || ""
  valid_keys = (ENV["API_KEYS"] || "").split(",")

# ...

# src/routes/api.rb

Tina4::Router.group("/api/v1", middleware: ["add_request_id", "log_request", "CorsMiddleware", "require_api_ke

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [
      { id: 1, name: "Widget", price: 9.99 },
      { id: 2, name: "Gadget", price: 19.99 }
    ]})
  end

end
```

12. Exercise: Build an API Key Middleware

Build a middleware called `validate_api_key` that:

1. Checks for an `X-API-Key` header on every request

2. Validates the key against a comma-separated list stored in the `API_KEYS` environment variable

3. If the key is missing, returns `401` with `{"error": "API key required"}`

4. If the key is invalid, returns `403` with `{"error": "Invalid API key"}`

5. If the key is valid, attaches the key to `request.api_key` and continues

6. Apply this middleware to a route group with at least two endpoints

Setup

Add this to your `.env`:

```
API_KEYS=key-alpha-001,key-beta-002,key-gamma-003
```

Test with:

```
# No API key -- should get 401
curl http://localhost:7147/api/partner/data

# Invalid API key -- should get 403
curl http://localhost:7147/api/partner/data \
  -H "X-API-Key: wrong-key"

# Valid API key -- should get 200
curl http://localhost:7147/api/partner/data \
  -H "X-API-Key: key-alpha-001"
```

13. Solution

Create `src/routes/api_key_middleware.rb`:

```
def validate_api_key(request, response, next_handler)
  api_key = request.headers["X-API-Key"] || ""

  if api_key.empty?
    return response.json({ error: "API key required" }, 401)
  end

  valid_keys = (ENV["API_KEYS"] || "").split(",").map(&:strip)

  unless valid_keys.include?(api_key)
    return response.json({ error: "Invalid API key" }, 403)
  end

  request.api_key = api_key

  next_handler.call(request, response)
end

Tina4::Router.group("/api/partner", middleware: "validate_api_key") do

  Tina4::Router.get("/data") do |request, response|
    response.json({
      authenticated_with: request.api_key,
      data: [
        { id: 1, value: "alpha" },
# ...
```

Expected output -- no key:

```
{"error":"API key required"}
```

(Status: `401 Unauthorized`)

Expected output -- valid key:

```
{
  "authenticated_with": "key-alpha-001",
  "data": [
    {"id": 1, "value": "alpha"},
    {"id": 2, "value": "beta"}
  ]
}
```

14. Gotchas

1. Middleware Must Be a Named Method

Problem: You pass an anonymous lambda as middleware and get an error.

Cause: Tina4 expects middleware to be referenced by a string name, not as an inline block. The string is resolved to a method at runtime.

Fix: Define your middleware as a named method: `def my_middleware(request, response, next_handler) ... end` and pass `"my_middleware"` as a string.

2. Forgetting to Return next_handler Result

Problem: Your middleware runs but the route handler never executes. The response is empty or a 500 error.

Cause: You called `next_handler.call(request, response)` but did not return the result.

Fix: Always return the result of `next_handler.call(request, response)`. Without the return, the middleware discards the response from the handler and returns `nil`.

3. Middleware Order Matters

Problem: Your logging middleware does not see the request ID, even though `add_request_id` is in the middleware list.

Cause: `log_request` runs before `add_request_id`. Middleware executes in the order listed.

Fix: Put `add_request_id` before `log_request` in the array: `["add_request_id", "log_request"]`.

4. CORS Preflight Returns 404

Problem: The browser's preflight `OPTIONS` request gets a 404, but `GET` and `POST` work fine when tested with curl.

Cause: You did not apply `CorsMiddleware` to the route, so the `OPTIONS` method is not handled.

Fix: Apply `CorsMiddleware` to the group. It automatically handles `OPTIONS` requests.

5. Rate Limiter Counts Preflight Requests

Problem: Your frontend hits the rate limit faster than expected because every `POST` request counts as two requests.

Cause: The rate limiter counts all requests, including `OPTIONS`.

Fix: Put `CorsMiddleware` before `RateLimiter` in the middleware chain.

6. Middleware File Not Auto-Loaded

Problem: You defined middleware in a file but get "method not found" when referencing it.

Cause: The file is not in `src/routes/`. Tina4 auto-loads all `.rb` files in `src/routes/`, but middleware defined outside that directory is not discovered.

Fix: Put your middleware methods in a file inside `src/routes/`, such as `src/routes/middleware.rb`.

7. Short-Circuiting Skips Cleanup Middleware

Problem: Your timing middleware logs the start time but never logs the completion time for blocked requests.

Cause: When an inner middleware short-circuits, the outer middleware's code after `next_handler.call` still runs. But if the short-circuiting middleware is listed before the timing middleware, the timing middleware never executes at all.

Fix: Put cleanup-dependent middleware (timing, logging) at the outermost layer.

Chapter 9: Sessions & Cookies

1. State in a Stateless World

Your e-commerce site needs a shopping cart that persists across page loads, remembers the user's language preference, and flashes success or error messages after form submissions. HTTP is stateless -- every request is independent, with no memory of what came before. Sessions and cookies solve this by giving the server a way to remember who is making requests and what they have been doing.

In Chapter 7 you saw a brief introduction to sessions for authentication. This chapter goes deeper: session backends, flash messages, cookies, remember-me tokens, and security configuration.

2. How Sessions Work

When a user visits your site for the first time, Tina4 generates a unique session ID (a long random string), stores it in a cookie on the user's browser, and creates a server-side storage entry keyed by that ID. On every subsequent request, the browser sends the cookie, Tina4 looks up the session data, and makes it available via `request.session`.

The flow looks like this:

1. Browser sends first request (no session cookie)

2. Tina4 generates session ID: `abc123def456`

3. Tina4 sets cookie: `tina4_session=abc123def456`

4. Tina4 creates empty session storage for `abc123def456`

5. Browser sends second request with cookie `tina4_session=abc123def456`

6. Tina4 loads session data for `abc123def456`

7. Your route handler reads and writes `request.session`

8. At the end of the request, Tina4 saves the updated session data

The session data is stored server-side. The browser only has the session ID -- it never sees the actual data.

3. File Sessions (Default)

Out of the box, Tina4 stores sessions in files. No configuration needed.

```
Tina4::Router.get("/visit-counter") do |request, response|
  count = (request.session["visit_count"] || 0) + 1
  request.session["visit_count"] = count

  response.json({
    visit_count: count,
    message: "You have visited this page #{count} time#{count == 1 ? '' : 's'}"
  })
end
```

```
curl http://localhost:7147/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

```
curl http://localhost:7147/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":2,"message":"You have visited this page 2 times"}
```

The `-c cookies.txt` flag tells curl to save cookies to a file, and `-b cookies.txt` tells it to send them back. This simulates how a browser works.

4. Redis Sessions

For production deployments with multiple servers (behind a load balancer), you need a shared session store. Redis is the most common choice.

```
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
TINA4_SESSION_PASSWORD=your-redis-password
```

That is the only change. Your code stays exactly the same. `request.session` works identically whether sessions are stored in files, Redis, MongoDB, or Valkey.

5. MongoDB Sessions

```
TINA4_SESSION_HANDLER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017
TINA4_SESSION_DATABASE=myapp
TINA4_SESSION_COLLECTION=sessions
```

6. Valkey Sessions

```
TINA4_SESSION_HANDLER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

7. Reading and Writing Session Data

Session data is a simple key-value store. You read and write it through `request.session`:

```
# Write to session
Tina4::Router.post("/api/preferences") do |request, response|
  body = request.body

  request.session["language"] = body["language"] || "en"
  request.session["theme"] = body["theme"] || "light"
  request.session["items_per_page"] = (body["items_per_page"] || 20).to_i

  response.json({
    message: "Preferences saved",
    preferences: {
      language: request.session["language"],
      theme: request.session["theme"],
      items_per_page: request.session["items_per_page"]
```

```
    }
  })
end

# Read from session
Tina4::Router.get("/api/preferences") do |request, response|
  response.json({
    language: request.session["language"] || "en",
    theme: request.session["theme"] || "light",
    items_per_page: request.session["items_per_page"] || 20
  })
# ...
```

Storing Complex Data

Sessions can hold arrays and nested structures:

```
Tina4::Router.post("/api/cart/add") do |request, response|
  body = request.body

  request.session["cart"] ||= []

  cart = request.session["cart"]

  cart << {
    product_id: body["product_id"].to_i,
    name: body["name"],
    price: body["price"].to_f,
    quantity: (body["quantity"] || 1).to_i,
    added_at: Time.now.iso8601
  }

  request.session["cart"] = cart

  total = cart.sum { |item| item[:price] * item[:quantity] }

  response.json({
    message: "Added to cart",
    cart_items: cart.length,
    cart_total: total
  })
end
```

8. Flash Messages

Flash messages are session data that exists for exactly one request. You set a flash message before redirecting, and it is available on the very next request. After that next request reads it, it is gone.

Setting a Flash Message

```
Tina4::Router.post("/profile/update") do |request, response|
  body = request.body

  # Update the profile (database logic here)

  request.session["_flash"] = {
    type: "success",
    message: "Profile updated successfully"
  }

  response.redirect("/profile")
```

```
end
```

Reading and Clearing Flash Messages

```
Tina4::Router.get("/profile") do |request, response|
  flash = request.session["_flash"]
  request.session.delete("_flash")

  response.render("profile.html", {
    user: { name: "Alice", email: "alice@example.com" },
    flash: flash
  })
end
```

Using Flash Messages in Templates

```
{% extends "base.html" %}

{% block content %}
    {% if flash %}
        <div class="alert alert-{{ flash.type }}">
            {{ flash.message }}
        </div>
    {% endif %}

    <h1>Profile</h1>
    <p>Name: {{ user.name }}</p>
    <p>Email: {{ user.email }}</p>
{% endblock %}
```

9. Setting and Reading Cookies

Cookies are small pieces of data stored in the browser. Unlike sessions, the data is stored client-side.

Setting a Cookie

```
Tina4::Router.post("/api/set-language") do |request, response|
  language = request.body["language"] || "en"

  response.cookie("language", language, {
    expires: Time.now + (365 * 24 * 60 * 60),  # 1 year
    path: "/",
    http_only: false,
    secure: false,
    same_site: "Lax"
  })

  response.json({ message: "Language set to #{language}" })
end
```

Reading a Cookie

```
Tina4::Router.get("/api/get-language") do |request, response|
  language = request.cookies["language"] || "en"

  response.json({ language: language })
end
```

Deleting a Cookie

```
Tina4::Router.post("/api/clear-language") do |request, response|
  response.cookie("language", "", {
```

```
    expires: Time.now - 3600,
    path: "/"
  })

  response.json({ message: "Language cookie cleared" })
end
```

When to Use Cookies vs Sessions

| Use Cookies For | Use Sessions For |

|----------|----------|

| Language preference | Shopping cart contents |

| Theme preference (light/dark) | User authentication state |

| "Remember this device" flag | Flash messages |

| Analytics tracking consent | Form wizard progress |

| Non-sensitive, long-lived data | Sensitive, short-lived data |

10. Remember Me Functionality

The "remember me" pattern uses a long-lived cookie to re-authenticate users after their session expires.

```
# @noauth
Tina4::Router.post("/login") do |request, response|
  body = request.body
  db = Tina4::Database.connection

  user = db.fetch_one(
    "SELECT id, name, email, password_hash FROM users WHERE email = :email",
    { email: body["email"] }
  )

  if user.nil? || !Tina4::Auth.check_password(body["password"], user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

  request.session["user_id"] = user["id"]
  request.session["user_name"] = user["name"]

  if body["remember_me"]
    remember_token = SecureRandom.hex(32)

    db.execute(
      "UPDATE users SET remember_token = :token WHERE id = :id",
      { token: Digest::SHA256.hexdigest(remember_token), id: user["id"] }
    )

# ...
```

11. Session Security

Configuration Options

```
TINA4_SESSION_LIFETIME=3600
TINA4_SESSION_NAME=tina4_session
```

```
TINA4_SESSION_SECURE=true
TINA4_SESSION_HTTPONLY=true
TINA4_SESSION_SAMESITE=Lax
```

Session Regeneration

After a user logs in, regenerate the session ID to prevent session fixation attacks:

```
Tina4::Router.post("/login") do |request, response|
  # Validate credentials...

  request.session_regenerate

  request.session["user_id"] = user["id"]

  response.redirect("/dashboard")
end
```

12. Exercise: Build a Shopping Cart with Session Storage

Build a shopping cart that stores items in the session. No database needed -- the cart lives entirely in session data.

Requirements

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/cart/add` | Add an item to the cart. Body: `{"product_id": 1, "name": "Widget", "price": 9.99, "quantity": 2}` |

| `GET` | `/api/cart` | View the cart. Show items, quantities, item subtotals, and cart total. |

| `PUT` | `/api/cart/{product_id:int}` | Update quantity. Body: `{"quantity": 3}`. Remove item if quantity is 0. |

| `DELETE` | `/api/cart/{product_id:int}` | Remove an item from the cart. |

| `DELETE` | `/api/cart` | Clear the entire cart. |

Business Rules

1. If adding a product that already exists in the cart, increment the quantity instead of adding a duplicate

2. Cart total should be calculated dynamically

3. Return the full cart state after every operation

Test with:

```
# Add first item
curl -X POST http://localhost:7147/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# Add second item
curl -X POST http://localhost:7147/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2}' \
  -c cookies.txt -b cookies.txt
```

```
# View cart
curl http://localhost:7147/api/cart -b cookies.txt

# Clear cart
curl -X DELETE http://localhost:7147/api/cart -b cookies.txt -c cookies.txt
```

13. Solution

Create `src/routes/cart.rb`:

```
def get_cart(session)
  session["cart"] || []
end

def cart_response(cart)
  total = 0.0
  item_count = 0
  items = []

  cart.each do |item|
    subtotal = item[:price] * item[:quantity]
    total += subtotal
    item_count += item[:quantity]
    items << item.merge(subtotal: subtotal)
  end

  {
    items: items,
    item_count: item_count,
    unique_items: cart.length,
    total: total.round(2)
  }
end

# Add item to cart
# ...
```

Expected output after adding two items:

```
{
  "items": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1, "subtotal": 79.99},
    {"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2, "subtotal": 99.98}
  ],
  "item_count": 3,
  "unique_items": 2,
  "total": 179.97
}
```

14. Gotchas

1. Sessions Do Not Work with curl Without Cookie Flags

Problem: Each curl request sees an empty session, as if it is a new user.

Cause: curl does not automatically save or send cookies.

Fix: Use `-c cookies.txt -b cookies.txt` with curl.

2. Session Data Disappears After Server Restart

Problem: All session data is gone after restarting the dev server.

Cause: File sessions stored in the system temp directory may be cleared on restart.

Fix: Set `TINA4_SESSION_PATH` to a persistent directory. For production, use Redis or Valkey.

3. Session Cookie Not Sent Over HTTP in Production

Problem: Sessions work locally but not in production.

Cause: `TINA4_SESSION_SECURE=true` means the cookie is only sent over HTTPS.

Fix: Ensure your reverse proxy sets `X-Forwarded-Proto: https`, or set `TINA4_SESSION_SECURE=false` for HTTP.

4. Flash Messages Show Twice

Problem: The flash message appears, then appears again on the next page load.

Cause: You read the flash message but did not clear it from the session.

Fix: Always clear the flash message immediately after reading it: `request.session.delete("_flash")`.

5. Large Session Data Causes Slow Requests

Problem: Pages load slowly and performance degrades over time.

Cause: You are storing large amounts of data in the session.

Fix: Keep session data small. Store IDs and references, not entire objects.

6. Remember Me Token Not Invalidated on Password Change

Problem: After a user changes their password, their "remember me" cookies on other devices still work.

Cause: The remember-me token in the database was not cleared when the password changed.

Fix: Clear the `remember_token` column whenever the password is updated.

7. Session Fixation

Problem: An attacker can hijack a user's session by setting a known session ID before the user logs in.

Cause: The session ID is not regenerated after login.

Fix: Call `request.session_regenerate` after successful login.

Chapter 10: API Documentation with Swagger

1. The 47-Endpoint Problem

Your team has 47 API endpoints and the frontend developer keeps asking "what does this endpoint accept?" You email a spreadsheet. It goes out of date. You write a wiki page. Nobody updates it. You add comments to the code. Nobody reads them.

Swagger (also called OpenAPI) solves this permanently. It generates interactive API documentation from annotations in your route files. The docs are always up to date because they come from the code itself. Your frontend developer can browse every endpoint, see the expected request and response formats, and even test endpoints directly from the browser.

Tina4 auto-generates a Swagger UI at `/swagger` from comment annotations on your routes. No build step. No extra tooling. Write the annotations, and the documentation appears.

2. What Swagger/OpenAPI Is

OpenAPI is a specification format for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and generates documentation, client SDKs, and server stubs.

An OpenAPI spec describes:

• Every endpoint (path + HTTP method)

• What parameters each endpoint accepts (path, query, header, body)

• What each endpoint returns (response codes, response bodies)

• Data schemas (what a "User" or "Product" object looks like)

• Authentication requirements

• Grouping and tagging

Tina4 builds this spec automatically from comment annotations in your Ruby code. You never write JSON or YAML by hand.

3. Enabling Swagger

Swagger is available out of the box when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7147/swagger
```

You should see the Swagger UI with any routes you have already defined. If you have not added any Swagger annotations yet, you will see the routes listed with default descriptions.

For production, you can explicitly enable or disable Swagger:

```
TINA4_SWAGGER=true
```

The Swagger JSON Endpoint

The raw OpenAPI spec is available at:

```
http://localhost:7147/swagger/json
```

```
curl http://localhost:7147/swagger/json
```


```
{
  "openapi": "3.0.0",
  "info": {
    "title": "My Store API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/products": {
      "get": {
        "summary": "List all products",
        "responses": {
          "200": {
            "description": "Successful response"
          }
        }
      }
    }
  }
}
```


4. Adding Descriptions to Routes

Add Swagger annotations as comments above your route definitions:

```
# List all products
# @description Returns a paginated list of all products in the catalog
# @tags Products
Tina4::Router.get("/api/products") do |request, response|
  response.json({ products: [] })
end
```

The first comment line becomes the `summary`. The `@description` tag provides a longer explanation.

Documenting Path Parameters

```
# Get a product by ID
# @description Returns a single product with full details including inventory status
# @tags Products
# @param int $id The unique product identifier
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params["id"]
  response.json({ id: id, name: "Wireless Keyboard", price: 79.99 })
end
```

Documenting Query Parameters

```
# Search products
# @description Search the product catalog by name, category, or price range
# @tags Products
# @query string $q Search query (searches product name and description)
# @query string $category Filter by category name
# @query float $min_price Minimum price filter
# @query float $max_price Maximum price filter
# @query int $page Page number (default: 1)
# @query int $limit Items per page (default: 20, max: 100)
Tina4::Router.get("/api/products/search") do |request, response|
  q = request.query["q"] || ""
  page = (request.query["page"] || 1).to_i
```

```
  limit = [(request.query["limit"] || 20).to_i, 100].min

  response.json({
    query: q,
    page: page,
    limit: limit,
    results: [],
    total: 0
  })
end
```

5. Documenting Request and Response Schemas

Request Body

```
# Create a new product
# @description Creates a product in the catalog. Requires admin authentication.
# @tags Products
# @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool", "description": "string"
# @response 201 {"id": "int", "name": "string", "category": "string", "price": "float", "in_stock": "bool", "c
# @response 400 {"error": "string"}
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  if body["name"].nil? || body["name"].empty?
    return response.json({ error: "Name is required" }, 400)
  end

  response.json({
    id: 1,
    name: body["name"],
    category: body["category"] || "Uncategorized",
    price: (body["price"] || 0).to_f,
    in_stock: body["in_stock"] != false,
    created_at: Time.now.iso8601
  }, 201)
end
```

6. Tags for Grouping Endpoints

Tags group related endpoints in the Swagger UI:

```
# List all users
# @tags Users
Tina4::Router.get("/api/users") do |request, response|
  response.json({ users: [] })
end

# List all orders
# @tags Orders
Tina4::Router.get("/api/orders") do |request, response|
  response.json({ orders: [] })
end
```

In the Swagger UI, you will see sections for "Users" and "Orders". An endpoint can belong to multiple groups:

```
# Get user's orders
# @tags Users, Orders
```

```
Tina4::Router.get("/api/users/{id:int}/orders") do |request, response|
  response.json({ orders: [] })
end
```

7. Example Values

Add example values to make the docs more useful:

```
# Create a new product
# @description Creates a product in the catalog
# @tags Products
# @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "in_stock": true
# @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "in_st
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  response.json({
    id: 42,
    name: body["name"],
    category: body["category"] || "Uncategorized",
    price: (body["price"] || 0).to_f,
    in_stock: body["in_stock"] != false,
    created_at: Time.now.iso8601
  }, 201)
end
```

8. Try-It-Out from the Swagger UI

The Swagger UI includes a "Try it out" button on every endpoint. Clicking it:

1. Expands the endpoint with editable input fields
2. Pre-fills example values (if provided)
3. Lets you edit the parameters, headers, and request body
4. Sends the actual HTTP request to your running server
5. Shows the response status, headers, and body

This is a live testing tool built into your documentation.

Authentication in Try-It-Out

If your endpoints require authentication, click the "Authorize" button at the top of the Swagger UI. Enter your JWT token or API key, and all subsequent "Try it out" requests will include the authentication header.

9. Customizing the Swagger Info Block

Configure the top-level API information in `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_CONTACT_EMAIL=api@mystore.com
TINA4_SWAGGER_LICENSE=MIT
```

10. Generating Client SDKs from the Spec

The OpenAPI spec at `/swagger/json` can be used with code generation tools:

```
# Install the OpenAPI Generator CLI
npm install -g @openapitools/openapi-generator-cli

# Generate a TypeScript client
openapi-generator-cli generate \
  -i http://localhost:7147/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client
```

11. A Complete Documented API

Here is a full example showing all the annotation features together:

```
# List all users
# @description Returns a paginated list of users. Supports filtering by role and searching by name.
# @tags Users
# @query int $page Page number (default: 1)
# @query int $limit Items per page (default: 20)
# @query string $role Filter by role (admin, user, moderator)
# @query string $search Search by name or email
# @response 200 {"users": [{"id": "int", "name": "string", "email": "string", "role": "string"}], "total": "in
# @example response {"users": [{"id": 1, "name": "Alice", "email": "alice@example.com", "role": "admin"}], "to
Tina4::Router.get("/api/users") do |request, response|
  page = (request.query["page"] || 1).to_i
  limit = (request.query["limit"] || 20).to_i

  response.json({
    users: [
      { id: 1, name: "Alice", email: "alice@example.com", role: "admin" },
      { id: 2, name: "Bob", email: "bob@example.com", role: "user" }
    ],
    total: 42,
    page: page,
    pages: (42.0 / limit).ceil
  })
end

# Create a new user
# ...
```

12. Exercise: Document a Complete User API

Take the User API from the example above and extend it with the following endpoints. Write full Swagger annotations for each one.

Requirements

Document these additional endpoints (you can use hardcoded data in the handlers):

| Method | Path | Description |

|--------|------|-------------|

| `PUT` | `/api/users/{id}` | Update a user. Body: name, email, role. Response: updated user. |

| `GET` | `/api/users/{id}/orders` | List a user's orders. Query: status filter, pagination. |

| `POST` | `/api/users/{id}/avatar` | Upload user avatar. Body: avatar_url string. |

Each endpoint should have:

1. A summary (first comment line)

2. A `@description`

3. A `@tags` annotation

4. `@param` for path parameters

5. `@query` for query parameters (where applicable)

6. `@body` for request body (where applicable)

7. `@response` for each possible response code

8. `@example` for request and response (where applicable)

Test by visiting:

```
http://localhost:7147/swagger
```

13. Solution

Create `src/routes/user_api_documented.rb`:

```
# Update a user
# @description Updates an existing user's profile information. Only provided fields are updated.
# @tags Users
# @param int $id User ID
# @body {"name": "string", "email": "string", "role": "string"}
# @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updated_at": "string"}
# @response 404 {"error": "string"}
# @example request {"name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin"}
# @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin", "upd
Tina4::Router.put("/api/users/{id:int}") do |request, response|
  id = request.params["id"]
  body = request.body

  if id > 100
    return response.json({ error: "User not found" }, 404)
  end

  response.json({
    id: id,
    name: body["name"] || "Alice",
    email: body["email"] || "alice@example.com",
    role: body["role"] || "user",
    updated_at: Time.now.iso8601
  })
end
# ...
```

14. Gotchas

1. Annotations Must Be Directly Above the Route

Problem: Your Swagger annotations do not appear in the docs.

Cause: There is a blank line or other code between the comment block and the `Tina4::Router` call.

Fix: Make sure the comments are on the lines directly before the route definition with no blank lines in between.

2. Missing @tags Makes Endpoints Hard to Find

Problem: All endpoints appear in one giant flat list in the Swagger UI.

Fix: Add `# @tags ResourceName` to every route comment block.

3. @body Must Be Valid JSON

Problem: The Swagger UI shows the body schema as empty or broken.

Fix: Validate your `@body` JSON. Every key and string value must be in double quotes.

4. Swagger Shows Routes You Did Not Annotate

Problem: Unannotated routes appear in the Swagger UI with minimal documentation.

Cause: Tina4 includes all registered routes in the Swagger spec. Add `# @hidden` to hide a route.

5. Response Examples Do Not Match Actual Responses

Problem: The example response in Swagger shows different fields than the actual API response.

Fix: Treat annotations as part of the code. When you change a handler's response format, update the annotations.

6. Swagger UI Not Available in Production

Problem: `/swagger` returns a 404 in production.

Fix: If you want Swagger in production, explicitly set `TINA4_SWAGGER=true` in your `.env`.

7. SDK Generation Produces Incorrect Types

Problem: The generated TypeScript client has `any` types instead of proper interfaces.

Fix: Use correct OpenAPI type format in your annotations: `"name": "string"` (lowercase), `"price": "number"`, `"in_stock": "boolean"`.

Chapter 11: Queue System

1. Do Not Make the User Wait

Your app sends welcome emails on signup, generates PDF invoices, and resizes uploaded images. Each of these tasks takes between two and thirty seconds. If you do them inside the HTTP request, the user stares at a loading spinner while the server processes their email, renders their invoice, and resizes their photo. That is a terrible experience.

Queues solve this by moving slow work to a background process. The HTTP handler drops a job onto a queue and immediately responds to the user. A separate consumer process picks up the job and does the work at its own pace. The user sees "Welcome! Check your email." in under 100 milliseconds, and the email arrives 5 seconds later.

Tina4 has a built-in queue system that works out of the box with SQLite. No Redis, no RabbitMQ, no external services. Just add jobs and process them.

2. Why Queues Matter

Without queues:

```
User clicks "Sign Up"
  -> Server validates input (10ms)
  -> Server creates user in database (20ms)
  -> Server sends welcome email (3000ms)
  -> Server generates PDF welcome kit (2000ms)
  -> Server resizes avatar (1500ms)
  -> User sees response (6530ms later)
```

With queues:

```
User clicks "Sign Up"
  -> Server validates input (10ms)
  -> Server creates user in database (20ms)
  -> Server queues: send welcome email (1ms)
  -> Server queues: generate PDF (1ms)
  -> Server queues: resize avatar (1ms)
  -> User sees response (33ms later)

Meanwhile, in the background:
  -> Consumer sends welcome email
  -> Consumer generates PDF
  -> Consumer resizes avatar
```

The user's experience goes from 6.5 seconds to 33 milliseconds.

3. SQLite Queue (Default)

Tina4's queue system uses SQLite by default. No configuration needed. When you produce your first job, Tina4 creates a `data/queue.db` database automatically.

Producing a Job

```
Tina4::Router.post("/api/register") do |request, response|
  body = request.body
```

```
  # Create the user (database logic)
  user_id = 42 # Simulated

  # Queue a welcome email
  Tina4::Queue.produce("send-welcome-email", {
    user_id: user_id,
    email: body["email"],
    name: body["name"]
  })

  response.json({
    message: "Registration successful. Welcome email will arrive shortly.",
    user_id: user_id
  }, 201)
end
```

```
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful. Welcome email will arrive shortly.",
  "user_id": 42
}
```

The Queue.produce Method

```
Tina4::Queue.produce(queue_name, payload, options = {})
```

• **queue_name**: A string that identifies which queue to put the job on.

• **payload**: A hash of data the consumer needs to process the job.

• **options**: Optional settings like delay, priority, and max retries.

Producing with Options

```
# Delay the job by 60 seconds
Tina4::Queue.produce("send-reminder", {
  user_id: 42,
  message: "Do not forget to verify your email!"
}, { delay: 60 })

# Set max retries (default is 3)
Tina4::Queue.produce("generate-invoice", {
  order_id: 101,
  format: "pdf"
}, { max_retries: 5 })

# Set priority (lower number = higher priority)
Tina4::Queue.produce("resize-image", {
  image_path: "/uploads/photo.jpg",
  sizes: [100, 300, 600]
}, { priority: 1 })
```

4. Consuming Jobs

A consumer listens for jobs on a specific queue and processes them:

```
Tina4::Queue.consume("send-welcome-email") do |job|
  email = job.payload["email"]
  name = job.payload["name"]
```

```
  $stderr.puts "Sending welcome email to #{email} for #{name}"

  # In a real app, you would use Tina4::Messenger here

  true  # Return true to mark the job as completed
end
```

Starting the Consumer

```
tina4 queue:work
```

```
Queue worker started
  Listening on: all queues
  Backend: sqlite:///data/queue.db
  Polling interval: 1s

[2026-03-22 14:30:01] Processing job #1 on "send-welcome-email"
[2026-03-22 14:30:01] Job #1 completed in 45ms
```

Listening to Specific Queues

```
tina4 queue:work --queue send-welcome-email

tina4 queue:work --queue send-welcome-email,generate-invoice
```

5. Job Lifecycle

Every job moves through a series of states:

```
pending -> reserved -> completed
                    -> failed -> pending (retry)
                              -> dead (max retries exceeded)
```

Inspecting Job State

```
stats = Tina4::Queue.stats("send-welcome-email")
```

```
{
  "queue" => "send-welcome-email",
  "pending" => 12,
  "reserved" => 2,
  "completed" => 1453,
  "failed" => 3,
  "dead" => 1
}
```

6. Retry Logic and Max Retries

When a job fails, Tina4 automatically retries it with exponential backoff:

• Retry 1: after 10 seconds

• Retry 2: after 30 seconds

• Retry 3: after 90 seconds

Handling Failures in Consumers

```
Tina4::Queue.consume("send-welcome-email") do |job|
  email = job.payload["email"]
```

```
  begin
    success = send_email(email, "Welcome!", "Welcome to our platform.")

    unless success
      $stderr.puts "Failed to send email to #{email} (attempt #{job.attempts})"
      next false  # Triggers retry
    end

    true  # Job completed

  rescue => e
    $stderr.puts "Exception sending email to #{email}: #{e.message}"
    false  # Triggers retry
  end
end
```

7. Dead Letter Queue

Jobs that exceed their max retries land in the dead letter queue.

Viewing Dead Letter Jobs

```
tina4 queue:dead
```

Re-Queuing Dead Letter Jobs

```
tina4 queue:retry 42
tina4 queue:retry --queue send-welcome-email
tina4 queue:retry --all
```

Clearing Dead Letter Jobs

```
tina4 queue:clear 42
tina4 queue:clear --older-than 7d
```

8. Switching to RabbitMQ

For production deployments:

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=5672
TINA4_QUEUE_USERNAME=guest
TINA4_QUEUE_PASSWORD=guest
TINA4_QUEUE_VHOST=/
```

Your code does not change. The same `Tina4::Queue.produce` and `Tina4::Queue.consume` calls work with RabbitMQ as the backend.

9. Switching to Kafka

For event streaming at scale:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=9092
TINA4_QUEUE_GROUP_ID=my-app-workers
```

10. Monitoring via Dev Dashboard

When `TINA4_DEBUG=true`, the dev dashboard at `/tina4/console` shows a "Queue Manager" section with:

• Queue overview: pending, reserved, completed, failed, and dead counts

• Recent jobs: the last 50 processed jobs

• Failed jobs with error messages and retry counts

• Dead letter queue

• Throughput graph: jobs processed per minute

11. Producing Multiple Jobs

Sometimes a single action triggers multiple background tasks:

```
Tina4::Router.post("/api/orders") do |request, response|
  body = request.body
  order_id = 101

  Tina4::Queue.produce("send-order-confirmation", {
    order_id: order_id,
    email: body["email"]
  })

  Tina4::Queue.produce("generate-invoice", {
    order_id: order_id,
    format: "pdf"
  })

  Tina4::Queue.produce("update-inventory", {
    items: body["items"]
  })

  Tina4::Queue.produce("notify-warehouse", {
    order_id: order_id,
    shipping_address: body["shipping_address"]
  })

  response.json({
    message: "Order placed successfully",
# ...
```

Four jobs are queued in under 5 milliseconds. The user gets an instant response.

12. Exercise: Build an Email Queue

Build a queue-based email system for user signup.

Requirements

1. Create a `POST /api/signup` endpoint that queues a job on the `welcome-emails` queue

2. Create a consumer for the `welcome-emails` queue that logs the email details

3. Create a `GET /api/queue/stats` endpoint that shows the queue statistics

Test with:

```
curl -X POST http://localhost:7147/api/signup \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

curl http://localhost:7147/api/queue/stats

tina4 queue:work --queue welcome-emails

curl http://localhost:7147/api/queue/stats
```

13. Solution

Create `src/routes/email_queue.rb`:

```
# @noauth
Tina4::Router.post("/api/signup") do |request, response|
  body = request.body

  if body["name"].nil? || body["email"].nil? || body["password"].nil?
    return response.json({ error: "Name, email, and password are required" }, 400)
  end

  user_id = rand(1..10000)

  Tina4::Queue.produce("welcome-emails", {
    user_id: user_id,
    name: body["name"],
    email: body["email"],
    signed_up_at: Time.now.iso8601
  })

  response.json({
    message: "Signup successful! A welcome email will be sent shortly.",
    user_id: user_id
  }, 201)
end

Tina4::Queue.consume("welcome-emails") do |job|
  name = job.payload["name"]
# ...
```

14. Gotchas

1. Worker Must Be Running Separately

Problem: You produce jobs but nothing happens.

Fix: Run `tina4 queue:work` in a separate terminal.

2. Consumer Not Registered

Problem: The worker reports "No consumer found for queue: my-queue".

Fix: Make sure your consumer is in a file inside `src/routes/`.

3. Job Payload Is Not Serializable

Problem: `Queue.produce` throws an error about serialization.

Fix: Only pass simple data types in the payload. If you need to reference a database record, pass the ID.

4. Jobs Process in Wrong Order

Problem: Jobs are processed in a different order than they were produced.

Fix: With multiple workers, jobs are processed in parallel. Use a single worker if order matters.

5. SQLite Queue Lock Contention

Problem: Multiple workers on the same SQLite queue cause "database is locked" errors.

Fix: For multiple workers, switch to RabbitMQ or Kafka.

6. Consumer Returns Nothing

Problem: Jobs are processed but immediately marked as failed.

Cause: Your consumer block does not return `true`. Without an explicit `true`, Ruby returns the result of the last expression, which might be `nil`.

Fix: Always return `true` when the job succeeds and `false` when it fails.

7. Dead Letter Jobs Accumulate

Problem: The dead letter queue grows over time.

Fix: Check `Tina4::Queue.stats` periodically. Review dead letter jobs regularly.

Chapter 12: Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. When someone moves a card from "In Progress" to "Done", everyone else on the team should see it instantly -- no page refresh, no polling, no waiting. Traditional HTTP is request-response: the client asks, the server answers. The server cannot push data to the client on its own.

WebSocket solves this by establishing a persistent, bi-directional connection between the browser and the server. Once connected, either side can send messages at any time. The connection stays open until explicitly closed.

Tina4 treats WebSocket like routing. You define a WebSocket handler the same way you define an HTTP route. The path determines which handler processes the connection.

2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
```

Key differences:

• **Persistent**: The connection stays open. No repeated handshakes.

• **Bi-directional**: The server can push data without the client asking.

• **Low overhead**: After the initial handshake, messages are tiny.

• **Real-time**: Messages arrive within milliseconds.

3. Router.websocket -- WebSocket as a Route

In Tina4, you define WebSocket handlers using `Tina4::Router.websocket`:

```
Tina4::Router.websocket("/ws/echo") do |connection, event, data|
  if event == "message"
    connection.send("Echo: #{data}")
  end
end
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The handler receives three arguments:

• **connection**: The WebSocket connection object. Use it to send messages.

• **event**: The event type: `"open"`, `"message"`, or `"close"`.

• **data**: The message data (only present for `"message"` events).

Starting the Server

WebSocket runs alongside your HTTP server:

```
tina4 serve

  Tina4 Ruby v3.0.0
  HTTP server running at http://0.0.0.0:7147
  WebSocket server running at ws://0.0.0.0:7147
  Press Ctrl+C to stop
```

4. Connection Events

A Complete Handler

```
Tina4::Router.websocket("/ws/chat") do |connection, event, data|
  case event
  when "open"
    $stderr.puts "[Chat] New connection: #{connection.id}"
    connection.send(JSON.generate({
      type: "system",
      message: "Welcome to the chat!",
      your_id: connection.id
    }))

  when "message"
    message = JSON.parse(data)
    $stderr.puts "[Chat] #{connection.id}: #{message['text'] || data}"

    connection.send(JSON.generate({
      type: "message",
      from: connection.id,
      text: message["text"] || data,
      timestamp: Time.now.iso8601
    }))

  when "close"
    $stderr.puts "[Chat] Disconnected: #{connection.id}"
  end
end
```

5. Sending to a Single Client

`connection.send` sends a message to the specific client that triggered the event:

```
Tina4::Router.websocket("/ws/private") do |connection, event, data|
  if event == "message"
    message = JSON.parse(data)
    action = message["action"] || ""

    if action == "get-time"
      connection.send(JSON.generate({
        type: "time",
        server_time: Time.now.iso8601
      }))
```

```
    end

    if action == "get-status"
      connection.send(JSON.generate({
        type: "status",
        uptime: 3600,
        connections: 42,
        memory_mb: (ObjectSpace.memsize_of_all / 1024.0 / 1024.0).round(2)
      }))
    end
  end
end
```

6. Broadcasting to All Clients

`connection.broadcast` sends a message to every client connected to the same WebSocket path:

```
Tina4::Router.websocket("/ws/announcements") do |connection, event, data|
  if event == "open"
    connection.broadcast(JSON.generate({
      type: "system",
      message: "A new user joined",
      online_count: connection.connection_count
    }))
  end

  if event == "message"
    message = JSON.parse(data)

    connection.broadcast(JSON.generate({
      type: "announcement",
      from: connection.id,
      text: message["text"] || "",
      timestamp: Time.now.iso8601
    }))
  end

  if event == "close"
    connection.broadcast(JSON.generate({
      type: "system",
      message: "A user left",
      online_count: connection.connection_count
# ...
```

Broadcast Excluding Sender

```
if event == "message"
  message = JSON.parse(data)

  # Send to sender (confirmation)
  connection.send(JSON.generate({ type: "sent", text: message["text"] }))

  # Send to everyone else
  connection.broadcast(JSON.generate({
    type: "message",
    from: message["username"] || "Anonymous",
    text: message["text"],
    timestamp: Time.now.iso8601
  }), true)  # true = exclude sender
end
```

7. Path-Scoped Isolation

Different WebSocket paths are completely isolated. Clients connected to `/ws/chat/room-1` do not see messages from `/ws/chat/room-2`:

```
Tina4::Router.websocket("/ws/chat/{room}") do |connection, event, data|
  room = connection.params["room"]

  if event == "open"
    $stderr.puts "[Room #{room}] New connection: #{connection.id}"
    connection.broadcast(JSON.generate({
      type: "system",
      message: "Someone joined room #{room}",
      room: room,
      online: connection.connection_count
    }))
  end

  if event == "message"
    message = JSON.parse(data)
    connection.broadcast(JSON.generate({
      type: "message",
      room: room,
      from: message["username"] || "Anonymous",
      text: message["text"] || "",
      timestamp: Time.now.iso8601
    }))
  end

  if event == "close"
# ...
```

8. Building a Live Chat

WebSocket Handler

Create `src/routes/chat_ws.rb`:

```
$chat_users = {}

Tina4::Router.websocket("/ws/livechat/{room}") do |connection, event, data|
  room = connection.params["room"]

  case event
  when "open"
    $chat_users[connection.id] = {
      id: connection.id,
      username: "Anonymous",
      room: room,
      joined_at: Time.now.iso8601
    }

    connection.send(JSON.generate({
      type: "welcome",
      message: "Connected to room: #{room}",
      your_id: connection.id,
      online: connection.connection_count
    }))

  when "message"
    message = JSON.parse(data)
```

```
    type = message["type"] || "message"

# ...
```

9. Live Notifications

WebSocket is great for pushing notifications to users in real time:

```
Tina4::Router.websocket("/ws/notifications/{user_id}") do |connection, event, data|
  user_id = connection.params["user_id"]

  if event == "open"
    $stderr.puts "[Notifications] User #{user_id} connected"
    connection.send(JSON.generate({
      type: "connected",
      message: "Listening for notifications"
    }))
  end
end

# HTTP endpoint that triggers a notification
Tina4::Router.post("/api/orders/{order_id:int}/ship") do |request, response|
  order_id = request.params["order_id"]
  user_id = request.body["user_id"] || 0

  Tina4::Router.push_to_websocket("/ws/notifications/#{user_id}", JSON.generate({
    type: "notification",
    title: "Order Shipped",
    message: "Your order ##{order_id} has been shipped!",
    action_url: "/orders/#{order_id}",
    timestamp: Time.now.iso8601
  }))

# ...
```

10. Connecting from JavaScript

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

    function sendMessage(text) {
        ws.send(JSON.stringify({ type: "message", text: text }));
    }
</script>
```

Auto-Reconnect

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,
    reconnectInterval: 3000,
    maxReconnectAttempts: 10
});
```

11. A Complete Chat Page

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding: 12px; margin

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border: none; border-r
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const ws = frond.ws("/ws/livechat/" + room);
        const messagesDiv = document.getElementById("messages");
        let username = prompt("Enter your username:") || "Anonymous";

        ws.on("open", function () {
# ...
```

Create the route:

```
Tina4::Router.get("/chat/{room}") do |request, response|
  room = request.params["room"]
  response.render("chat.html", { room: room })
end
```

12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with username support, message broadcasting, and join/leave notifications.

Requirements

1. WebSocket endpoint at `/ws/room/{room_name}` that handles open, set-name, chat, and close events

2. HTTP endpoint at `GET /room/{room_name}` that serves an HTML chat page

3. The chat page should prompt for a username, display messages in real time, and show online count

Test by:

1. Open `http://localhost:7147/room/test` in two browser tabs

2. Set different usernames

3. Send messages from both tabs and verify they appear in both

4. Close one tab and verify the "user left" message appears

13. Solution

Create `src/routes/chat_room.rb`:

```
$room_users = {}

Tina4::Router.websocket("/ws/room/{room_name}") do |connection, event, data|
  room = connection.params["room_name"]
  key = "#{room}:#{connection.id}"

  case event
  when "open"
    $room_users[key] = "Anonymous"

    connection.send(JSON.generate({
      type: "system",
      message: "Welcome to room: #{room}",
      online: connection.connection_count
    }))

    connection.broadcast(JSON.generate({
      type: "system",
      message: "A new user joined",
      online: connection.connection_count
    }), true)

  when "message"
    msg = JSON.parse(data)
    type = msg["type"] || "chat"
# ...
```

14. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop immediately.

Fix: Use `tina4 serve` which runs a persistent server. For production with WEBrick/Puma, configure WebSocket proxying with Nginx.

2. Messages Are Strings, Not Objects

Problem: `data` in the message handler is a string, not a Ruby hash.

Fix: Always `JSON.parse(data)` when you expect JSON messages. Always `JSON.generate(...)` when you send structured data.

3. Connection Count Is Per-Path

Problem: `connection.connection_count` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path.

4. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Fix: Use a pub/sub backend like Redis to relay messages across server instances.

5. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Fix: Keep messages small (under 64KB). Use HTTP endpoints for bulk data transfer.

6. Memory Leak from Tracking Connected Users

Problem: The server's memory usage grows over time.

Fix: Always clean up in the `close` handler: `$chat_users.delete(connection.id)`.

7. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint.

Fix: Pass the token as a query parameter: `ws://localhost:7147/ws/chat?token=eyJ...`. Validate in the `open` handler.

Chapter 13: Email with Messenger

1. Every App Sends Email

Your SaaS app needs to send signup confirmations, password resets, and weekly digest emails -- with attachments, HTML templates, and reliable delivery. Email is one of those features that every application needs but nobody enjoys implementing. SMTP configuration, plain text fallbacks, attachment encoding, connection timeouts, bounce handling -- the details pile up fast.

Tina4's `Messenger` class handles all of this. Configure it via `.env`, create a Messenger instance, and send. In development mode, emails are intercepted and shown in the dev dashboard so you can inspect them without setting up a real SMTP server.

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_SMTP_HOST=smtp.example.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@example.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
TINA4_MAIL_FROM_ADDRESS=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

| Variable | Description | Common Values |

|---|---|---|

| `TINA4_MAIL_SMTP_HOST` | SMTP server hostname | `smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net` |

| `TINA4_MAIL_SMTP_PORT` | SMTP port | `587` (TLS), `465` (SSL), `25` (unencrypted) |

| `TINA4_MAIL_SMTP_USERNAME` | Login username | Usually your email address |

| `TINA4_MAIL_SMTP_PASSWORD` | Login password or app-specific password | App passwords for Gmail |

| `TINA4_MAIL_SMTP_ENCRYPTION` | Encryption method | `tls` (recommended), `ssl`, `none` |

| `TINA4_MAIL_FROM_ADDRESS` | Default "From" address | `noreply@yourdomain.com` |

| `TINA4_MAIL_FROM_NAME` | Default "From" display name | `My Store`, `Acme Corp` |

3. Sending a Simple Email

```
Tina4::Router.post("/api/contact") do |request, response|
  body = request.body

  mail = Tina4::Messenger.new
  mail.to = body["email"]
  mail.subject = "Thanks for reaching out!"
  mail.body = "Hi #{body['name']},\n\nWe received your message and will get back to you within 24 hours.\n\nBe
  mail.send
```

```
  response.json({ message: "Email sent successfully" })
end
```

```
curl -X POST http://localhost:7147/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "Hello!"}'
```

```
{"message":"Email sent successfully"}
```

4. HTML Emails with Templates

Use Frond templates for rich HTML emails:

```
Tina4::Router.post("/api/send-welcome") do |request, response|
  body = request.body

  mail = Tina4::Messenger.new
  mail.to = body["email"]
  mail.subject = "Welcome to My Store, #{body['name']}!"
  mail.html_template = "emails/welcome.html"
  mail.template_data = {
    name: body["name"],
    login_url: "https://mystore.com/login",
    year: Time.now.year
  }
  mail.send

  response.json({ message: "Welcome email sent" })
end
```

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body { font-family: Arial, sans-serif; background: #f5f5f5; padding: 20px; }
        .container { max-width: 600px; margin: 0 auto; background: white; border-radius: 8px; padding: 32px; 
        .header { text-align: center; padding-bottom: 20px; border-bottom: 1px solid #eee; }
        .btn { display: inline-block; padding: 12px 24px; background: #2d8f2d; color: white; text-decoration:
        .footer { text-align: center; color: #999; font-size: 12px; margin-top: 20px; }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Welcome, {{ name }}!</h1>
        </div>
        <p>Thanks for signing up. Your account is ready to go.</p>
        <p style="text-align: center;">
            <a href="{{ login_url }}" class="btn">Log In to Your Account</a>
        </p>
        <div class="footer">
            <p>&copy; {{ year }} My Store. All rights reserved.</p>
        </div>
    </div>
</body>
# ...
```

5. Attachments

```
mail = Tina4::Messenger.new
mail.to = "alice@example.com"
mail.subject = "Your Invoice"
mail.body = "Please find your invoice attached."
mail.attach("/path/to/invoice.pdf")
mail.attach("/path/to/receipt.png")
mail.send
```

Multiple Attachments

```
mail = Tina4::Messenger.new
mail.to = "alice@example.com"
mail.subject = "Monthly Report"
mail.body = "Here are this month's reports."
mail.attach("/reports/sales.pdf")
mail.attach("/reports/analytics.xlsx")
mail.attach("/reports/summary.csv")
mail.send
```

6. CC and BCC

```
mail = Tina4::Messenger.new
mail.to = "alice@example.com"
mail.cc = ["bob@example.com", "charlie@example.com"]
mail.bcc = ["manager@example.com"]
mail.subject = "Project Update"
mail.body = "Here is the latest project update..."
mail.send
```

7. Dev Mode: Email Interception

When `TINA4_DEBUG=true`, emails are not sent to real SMTP servers. Instead, they are intercepted and stored in the dev dashboard. Navigate to `/tina4/console` and click "Emails" to see:

• Recipient, subject, and timestamp

• Full HTML preview

• Plain text fallback

• Attachments list

• Headers

This means you can test email functionality without configuring SMTP during development.

8. Queuing Emails

For production, always queue emails instead of sending them synchronously:

```
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  # Create user...
  user_id = 42

  # Queue the email instead of sending directly
```

```
  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Welcome to My Store!",
    template: "emails/welcome.html",
    data: { name: body["name"], login_url: "https://mystore.com/login" }
  })

  response.json({ message: "Registration successful", user_id: user_id }, 201)
end

# Consumer that sends queued emails
Tina4::Queue.consume("send-email") do |job|
  mail = Tina4::Messenger.new
  mail.to = job.payload["to"]
  mail.subject = job.payload["subject"]

  if job.payload["template"]
    mail.html_template = job.payload["template"]
# ...
```

9. Reply-To and Custom Headers

```
mail = Tina4::Messenger.new
mail.to = "customer@example.com"
mail.reply_to = "support@mystore.com"
mail.subject = "Your Support Ticket #123"
mail.body = "We are looking into your issue..."
mail.add_header("X-Ticket-Id", "123")
mail.add_header("X-Priority", "1")
mail.send
```

10. Exercise: Build an Email Notification System

Build an email system that sends different types of notifications.

Requirements

1. `POST /api/notify/welcome` -- Send a welcome email with an HTML template

2. `POST /api/notify/order` -- Send an order confirmation with order details

3. `POST /api/notify/reset` -- Send a password reset email with a token link

Each endpoint should queue the email rather than sending it directly.

Test with:

```
curl -X POST http://localhost:7147/api/notify/welcome \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'

curl -X POST http://localhost:7147/api/notify/order \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "order_id": 101, "total": 159.98}'

curl -X POST http://localhost:7147/api/notify/reset \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "reset_token": "abc123def456"}'
```

11. Solution

Create `src/routes/notifications.rb`:

```
# @noauth
Tina4::Router.post("/api/notify/welcome") do |request, response|
  body = request.body

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Welcome to My Store, #{body['name']}!",
    template: "emails/welcome.html",
    data: { name: body["name"], login_url: "https://mystore.com/login", year: Time.now.year }
  })

  response.json({ message: "Welcome email queued" })
end

# @noauth
Tina4::Router.post("/api/notify/order") do |request, response|
  body = request.body

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Order Confirmation ##{body['order_id']}",
    template: "emails/order-confirmation.html",
    data: { order_id: body["order_id"], total: body["total"] }
  })

# ...
```

12. Gotchas

1. Gmail Requires App Passwords

Problem: Gmail login fails with "authentication error".

Fix: Enable 2FA on your Google account, then generate an App Password at https://myaccount.google.com/apppasswords. Use the app password, not your regular password.

2. Emails Go to Spam

Problem: Emails arrive in the spam folder.

Fix: Set up SPF, DKIM, and DMARC DNS records for your domain. Use a reputable email service like Mailgun, SendGrid, or Postmark.

3. HTML Email Rendering Differences

Problem: Your email looks different in Gmail, Outlook, and Apple Mail.

Fix: Use inline CSS. Avoid flexbox and grid. Use tables for layout. Test with a tool like Litmus or Email on Acid.

4. Attachment File Not Found

Problem: `mail.attach("/path/to/file.pdf")` raises a file not found error.

Fix: Use absolute paths. Relative paths resolve from the working directory, which may differ in production.

5. SMTP Connection Timeout

Problem: Sending email hangs for 30 seconds and then times out.

Fix: Check your SMTP host, port, and encryption settings. Common mistake: using port 587 with SSL instead of TLS.

6. Dev Mode Emails Disappear on Restart

Problem: Intercepted emails in the dev dashboard vanish when you restart the server.

Fix: This is expected. Dev mode stores emails in memory. For persistent storage, configure a real SMTP server.

7. Unicode Characters Display as Question Marks

Problem: Non-ASCII characters (accents, CJK) show as `?` in the email.

Fix: Tina4 sets UTF-8 encoding by default. If you are constructing raw headers, make sure you include `Content-Type: text/html; charset=utf-8`.

Chapter 14: Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries and takes 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes maybe once a day. After adding caching, the first request takes 800ms and the next 10,000 requests take 3ms each. That is a 266x improvement for one line of configuration.

Caching stores the result of expensive operations so they can be reused without recomputing. Tina4 provides caching at multiple levels: response caching (entire HTTP responses), database query caching, and a direct cache API for custom use cases.

2. Response Caching with ResponseCache Middleware

The fastest way to cache is at the HTTP response level. The `ResponseCache` middleware stores the complete response and serves it directly on subsequent requests without calling your route handler at all.

```
Tina4::Router.get("/api/products", middleware: "ResponseCache:300") do |request, response|
  $stderr.puts "Handler called -- this should only appear once every 5 minutes"

  products = [
    { id: 1, name: "Wireless Keyboard", price: 79.99 },
    { id: 2, name: "USB-C Hub", price: 49.99 },
    { id: 3, name: "Monitor Stand", price: 129.99 }
  ]

  response.json({ products: products, generated_at: Time.now.iso8601 })
end
```

The `"ResponseCache:300"` middleware caches the response for 300 seconds (5 minutes).

3. The Direct Cache API

For custom caching, use the `Tina4::Cache` API directly:

```
# Store a value
Tina4::Cache.set("products:featured", featured_products, ttl: 300)

# Retrieve a value
cached = Tina4::Cache.get("products:featured")

# Check if a key exists
if Tina4::Cache.exists?("products:featured")
  # Use cached data
end

# Delete a cached value
Tina4::Cache.delete("products:featured")

# Clear all cache
Tina4::Cache.clear
```

Fetch-or-Compute Pattern

The most common caching pattern -- check the cache, compute if missing, store and return:

```
Tina4::Router.get("/api/products") do |request, response|
  products = Tina4::Cache.fetch("products:all", ttl: 300) do
    # This block only runs if the cache is empty
    db = Tina4::Database.connection
    db.fetch("SELECT * FROM products ORDER BY name")
  end

  response.json({ products: products, count: products.length })
end
```

`Tina4::Cache.fetch` checks the cache for the key. If it exists, returns the cached value. If not, runs the block, stores the result, and returns it. The `ttl` (time to live) is in seconds.

4. Cache Backends

File Cache (Default)

Out of the box, Tina4 caches to the filesystem. No configuration needed:

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_PATH=data/cache
```

Redis Cache

For production with multiple servers:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_HOST=localhost
TINA4_CACHE_PORT=6379
TINA4_CACHE_PREFIX=myapp:cache:
```

Memory Cache

For single-process development (fastest, but lost on restart):

```
TINA4_CACHE_BACKEND=memory
```

5. Database Query Caching

Enable automatic query caching:

```
TINA4_DB_CACHE=true
```

When enabled, `fetch` and `fetch_one` calls are cached. The cache is automatically invalidated when you call `execute`, `insert`, `update`, or `delete`.

6. Cache Invalidation

Manual Invalidation

```
# After updating products
Tina4::Cache.delete("products:all")
Tina4::Cache.delete("products:featured")
```

Pattern-Based Invalidation

```
# Delete all product-related caches
Tina4::Cache.delete_pattern("products:*")
```

Automatic Invalidation in Route Handlers

```
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  db = Tina4::Database.connection
  db.insert("products", {
    name: body["name"],
    price: body["price"].to_f
  })

  # Invalidate all product caches
  Tina4::Cache.delete_pattern("products:*")

  response.json({ message: "Product created, cache cleared" }, 201)
end
```

7. Cache Headers

Set HTTP cache headers for browser and CDN caching:

```
Tina4::Router.get("/api/public/config") do |request, response|
  response
    .header("Cache-Control", "public, max-age=3600")
    .header("ETag", Digest::MD5.hexdigest("config-v1"))
    .json({
      app_name: "My Store",
      version: "1.0.0",
      features: { dark_mode: true, notifications: true }
    })
end
```

8. Cache Statistics

Monitor your cache performance:

```
Tina4::Router.get("/api/cache/stats") do |request, response|
  stats = Tina4::Cache.stats

  response.json({
    backend: stats[:backend],
    keys: stats[:key_count],
    memory_mb: stats[:memory_mb],
    hit_rate: stats[:hit_rate],
    hits: stats[:hits],
    misses: stats[:misses]
  })
end
```

9. Exercise: Add Caching to a Product API

Take your product API from Chapter 5 and add caching at multiple levels.

Requirements

1. Cache the product list for 5 minutes
2. Cache individual product lookups for 10 minutes
3. Invalidate caches when products are created, updated, or deleted
4. Add a `GET /api/cache/stats` endpoint

Test with:

```
# First call (slow, hits database)
curl http://localhost:7147/api/products

# Second call (fast, from cache)
curl http://localhost:7147/api/products

# Create a product (should invalidate cache)
curl -X POST http://localhost:7147/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "New Widget", "price": 19.99}'

# Next call should hit database again
curl http://localhost:7147/api/products

# Check cache stats
curl http://localhost:7147/api/cache/stats
```

10. Solution

Create `src/routes/cached_products.rb`:

```
Tina4::Router.get("/api/products") do |request, response|
  products = Tina4::Cache.fetch("products:all", ttl: 300) do
    db = Tina4::Database.connection
    db.fetch("SELECT * FROM products ORDER BY name")
  end

  response.json({ products: products, count: products.length, cached: true })
end

Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params["id"]

  product = Tina4::Cache.fetch("products:#{id}", ttl: 600) do
    db = Tina4::Database.connection
    db.fetch_one("SELECT * FROM products WHERE id = :id", { id: id })
  end

  if product.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  response.json(product)
end

Tina4::Router.post("/api/products") do |request, response|
# ...
```

11. Gotchas

1. Stale Data After Updates

Problem: Users see old data after an update.

Fix: Always invalidate related cache keys when data changes.

2. Cache Key Collisions

Problem: Different queries return the same cached data.

Fix: Include all relevant parameters in the cache key: `"products:category:#{category}:page:#{page}"`.

3. Memory Cache Lost on Restart

Problem: Cache is empty after server restart.

Fix: This is expected with memory cache. Use file or Redis cache for persistence.

4. Over-Caching Prevents Updates

Problem: Cache TTL is too long and users see stale data for hours.

Fix: Choose appropriate TTL values. For frequently changing data, use short TTLs or invalidate on write.

5. Cache Stampede

Problem: When a popular cache key expires, hundreds of requests simultaneously hit the database.

Fix: Use `Tina4::Cache.fetch` which handles locking -- only one request computes the value while others wait.

6. Caching User-Specific Data

Problem: User A sees User B's profile because the cache key does not include the user ID.

Fix: Include the user ID in the cache key: `"profile:#{user_id}"`.

7. ResponseCache Ignores Query Parameters

Problem: `/api/products?page=1` and `/api/products?page=2` return the same cached response.

Fix: ResponseCache includes query parameters in the cache key by default. If it does not, ensure your middleware version is up to date.

Chapter 15: Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a beautiful dashboard. You know the drill -- install Node.js, run `npm install`, wait for 200MB of `node_modules` to download, configure webpack or Vite, set up PostCSS, add a CSS framework, maybe Tailwind with its purge config, and then pray nothing breaks when you upgrade a dependency six months from now.

Tina4 takes a different approach. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper library. Both are included automatically when you scaffold a project. No npm. No webpack. No build step for development. Just link the files and start building.

By the end of this chapter, you will have a complete admin dashboard with a sidebar, navigation, cards, tables, modals, and dark mode support -- all without touching npm.

2. What Ships with Tina4

When you run `tina4 init`, two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names:

```
<div class="container">
    <div class="row">
        <div class="col-md-4"><p>One third</p></div>
        <div class="col-md-4"><p>One third</p></div>
        <div class="col-md-4"><p>One third</p></div>
    </div>
</div>
```

Responsive Breakpoints

| Prefix | Min Width | Typical Device |

|--------|-----------|----------------|

| `col-` | 0px | All |

| `col-sm-` | 576px | Phones (landscape) |

| `col-md-` | 768px | Tablets |

| `col-lg-` | 992px | Laptops |

| `col-xl-` | 1200px | Desktops |

4. Cards

```
<div class="card">
    <div class="card-header">Product Details</div>
    <div class="card-body">
        <h3 class="card-title">Wireless Keyboard</h3>
        <p class="card-text">Ergonomic keyboard with backlit keys.</p>
        <p class="price">$79.99</p>
    </div>
    <div class="card-footer">
        <button class="btn btn-primary">Add to Cart</button>
    </div>
</div>
```

Card Grid

```
<div class="row">
    {% for product in products %}
        <div class="col-md-4">
            <div class="card">
                <div class="card-body">
                    <h3 class="card-title">{{ product.name }}</h3>
                    <p class="card-text">${{ product.price | number_format(2) }}</p>
                    <button class="btn btn-primary btn-sm">View</button>
                </div>
            </div>
        </div>
    {% endfor %}
</div>
```

5. Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        {% for product in products %}
            <tr>
                <td>{{ product.id }}</td>
                <td>{{ product.name }}</td>
                <td>${{ product.price | number_format(2) }}</td>
                <td>
                    <button class="btn btn-sm btn-primary">Edit</button>
                    <button class="btn btn-sm btn-danger">Delete</button>
                </td>
            </tr>
        {% endfor %}
```

```
    </tbody>
</table>
```

6. Forms

```
<form method="POST" action="/api/products">
    <div class="form-group">
        <label for="name">Product Name</label>
        <input type="text" class="form-control" id="name" name="name" required>
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" class="form-control" id="price" name="price" step="0.01" required>
    </div>
    <div class="form-group">
        <label for="category">Category</label>
        <select class="form-control" id="category" name="category">
            <option value="Electronics">Electronics</option>
            <option value="Fitness">Fitness</option>
            <option value="Kitchen">Kitchen</option>
            <option value="Office">Office</option>
        </select>
    </div>
    <button type="submit" class="btn btn-primary">Create Product</button>
</form>
```

7. Buttons and Alerts

Buttons

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-outline-primary">Outline</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Alerts

```
<div class="alert alert-success">Product created successfully!</div>
<div class="alert alert-danger">Error: Name is required.</div>
<div class="alert alert-warning">Stock is running low.</div>
<div class="alert alert-info">New features are available.</div>
```

8. frond.js -- AJAX Without jQuery

`frond.js` provides lightweight AJAX helpers for form submission, API calls, and token management.

API Calls

```
// GET request
frond.get("/api/products", function (data) {
    console.log(data.products);
});
```

```
// POST request
frond.post("/api/products", {
    name: "Widget",
    price: 9.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Widget",
    price: 12.99
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.del("/api/products/1", function (data) {
    console.log("Deleted");
});
```

Token Management

```
// Store token after login
frond.post("/api/login", { email: "alice@example.com", password: "pass123" }, function (data) {
    frond.setToken(data.token);
});

// All subsequent requests automatically include the Authorization header
frond.get("/api/profile", function (data) {
    console.log(data);  // Token is sent automatically
});
```

Form Submission via AJAX

```
<form id="product-form" data-frond-submit="/api/products" data-frond-method="POST">
    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" class="form-control" name="name" id="name">
    </div>
    <button type="submit" class="btn btn-primary">Create</button>
</form>

<div id="result"></div>

<script>
    frond.onSubmit("#product-form", function (data) {
        document.getElementById("result").innerHTML = '<div class="alert alert-success">Product created: ' + d
    });
</script>
```

9. Dark Mode

tina4css supports dark mode via a CSS class on the `<html>` or `<body>` element:

```
<html data-theme="dark">
```

Toggle with JavaScript:

```
function toggleDarkMode() {
    const html = document.documentElement;
    const current = html.getAttribute("data-theme");
```

```
    html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
    localStorage.setItem("theme", html.getAttribute("data-theme"));
}

// Load saved theme
const savedTheme = localStorage.getItem("theme") || "light";
document.documentElement.setAttribute("data-theme", savedTheme);
```

10. Building a Dashboard

Here is a complete admin dashboard template:

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}Dashboard{% endblock %}

{% block content %}
    <h1>Dashboard</h1>

    <div class="row">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>{{ stats.total_products }}</h2>
                    <p class="text-muted">Products</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>{{ stats.total_orders }}</h2>
                    <p class="text-muted">Orders</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
# ...
```

Route handler:

```
Tina4::Router.get("/admin") do |request, response|
  response.render("dashboard.html", {
    stats: {
      total_products: 156,
      total_orders: 1423,
      revenue: 45678.90,
      total_users: 342
    },
    recent_orders: [
      { id: 1001, customer: "Alice Smith", total: 159.98, status: "Shipped", status_class: "success" },
      { id: 1000, customer: "Bob Jones", total: 49.99, status: "Processing", status_class: "warning" },
      { id: 999, customer: "Charlie Brown", total: 299.99, status: "Delivered", status_class: "info" }
    ]
  })
end
```

11. Exercise: Build a Product Management Page

Build a product management page with a table, add/edit form, and AJAX interactions.

Requirements

1. `GET /admin/products` -- HTML page with a product table and "Add Product" form

2. Use frond.js for AJAX form submission

3. Display flash messages for success/error

4. Include search and category filter

12. Solution

Create `src/templates/admin-products.html` with a table displaying products, a form for adding new ones using `data-frond-submit`, and JavaScript handlers using `frond.post` and `frond.get` for CRUD operations. The template extends `base.html` and uses tina4css for styling.

Create `src/routes/admin_products.rb`:

```
Tina4::Router.get("/admin/products") do |request, response|
  db = Tina4::Database.connection
  products = db.fetch("SELECT * FROM products ORDER BY name")

  response.render("admin-products.html", {
    products: products,
    count: products.length
  })
end
```

13. Gotchas

1. tina4css Classes Do Not Work

Problem: CSS classes have no effect.

Fix: Make sure `<link rel="stylesheet" href="/css/tina4.css">` is in your HTML head.

2. frond.js Not Loading

Problem: `frond is not defined` error in the browser console.

Fix: Make sure `<script src="/js/frond.js"></script>` is included before your custom scripts.

3. AJAX Calls Return HTML Instead of JSON

Problem: `frond.get` returns HTML instead of JSON.

Fix: Your route handler is returning `response.render` instead of `response.json`. API endpoints should use `response.json`.

4. Forms Submit Twice

Problem: The form submits via AJAX and then also submits normally.

Fix: Call `event.preventDefault()` in your submit handler, or use `data-frond-submit` which handles this automatically.

5. Dark Mode Resets on Page Load

Problem: Dark mode resets to light mode when navigating.

Fix: Save the theme preference in `localStorage` and apply it on page load before the page renders.

6. Bootstrap Classes Not Working

Problem: Some Bootstrap classes do not work with tina4css.

Fix: tina4css is Bootstrap-compatible but not a complete clone. Stick to the documented classes. Complex Bootstrap components (modals, tooltips) require additional JavaScript.

7. SCSS Changes Not Reflected

Problem: You edited `tina4.scss` but the browser shows the old CSS.

Fix: Compile SCSS to CSS with `tina4 build:css` or `sass src/public/scss/tina4.scss src/public/css/tina4.css`. The browser loads the compiled CSS file, not the SCSS source.

Chapter 16: GraphQL

1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product has a name, price, 20 image URLs, a full description, 15 review objects, and 8 other fields you do not need right now. With REST, you call `GET /api/products` and receive all of it -- 50KB of JSON when you only needed 2KB. On a spotty mobile connection, that wasted bandwidth matters.

Now your web dashboard needs the same products, but it also needs the category, stock status, and supplier info. With REST, you either make three requests (products, categories, suppliers) and stitch them together on the client, or you build a custom endpoint.

GraphQL solves both problems. The client asks for exactly the fields it needs, and the server returns exactly those fields. One endpoint, one request, one response with the right shape.

Tina4 includes a built-in GraphQL engine. No external packages. No Apollo Server. It is part of the framework.

2. GraphQL vs REST -- A Quick Comparison

| Aspect | REST | GraphQL |

|--------|------|---------|

| Endpoints | One per resource (`/products`, `/users`) | One endpoint (`/graphql`) |

| Data shape | Server decides | Client decides |

| Over-fetching | Common (get all fields) | Never (get only requested fields) |

| Under-fetching | Common (multiple requests needed) | Never (get related data in one query) |

| Versioning | `/api/v1/`, `/api/v2/` | Schema evolves, no versioning needed |

| Caching | HTTP caching works naturally | Requires client-side cache management |

| Learning curve | Low | Moderate |

3. Enabling GraphQL

GraphQL is available by default in Tina4. The engine serves requests at `/graphql`:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
```

```
}
```

4. Defining a Schema

Create your GraphQL schema in `src/routes/graphql.rb`:

```
Tina4::GraphQL.query("products") do |args|
  db = Tina4::Database.connection
  products = db.fetch("SELECT * FROM products ORDER BY name")
  products
end

Tina4::GraphQL.query("product") do |args|
  db = Tina4::Database.connection
  db.fetch_one("SELECT * FROM products WHERE id = :id", { id: args["id"].to_i })
end

Tina4::GraphQL.query("users") do |args|
  db = Tina4::Database.connection
  db.fetch("SELECT id, name, email, role FROM users ORDER BY name")
end
```

Querying

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
      {"id": 2, "name": "USB-C Hub", "price": 49.99}
    ]
  }
}
```

The client asked for `id`, `name`, and `price` only. The response contains exactly those fields -- no `category`, no `in_stock`, no `created_at`.

5. Query Arguments

```
Tina4::GraphQL.query("product") do |args|
  db = Tina4::Database.connection
  db.fetch_one("SELECT * FROM products WHERE id = :id", { id: args["id"].to_i })
end
```

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 1) { name price category } }"}'
```

```
{
  "data": {
    "product": {
      "name": "Wireless Keyboard",
      "price": 79.99,
      "category": "Electronics"
    }
  }
}
```

6. Mutations

Mutations are write operations. Define them with `Tina4::GraphQL.mutation`:

```
Tina4::GraphQL.mutation("createProduct") do |args|
  db = Tina4::Database.connection

  db.execute(
    "INSERT INTO products (name, category, price) VALUES (:name, :category, :price)",
    { name: args["name"], category: args["category"] || "Uncategorized", price: args["price"].to_f }
  )

  db.fetch_one("SELECT * FROM products WHERE id = last_insert_rowid()")
end

Tina4::GraphQL.mutation("updateProduct") do |args|
  db = Tina4::Database.connection
  id = args["id"].to_i

  db.execute(
    "UPDATE products SET name = :name, price = :price WHERE id = :id",
    { name: args["name"], price: args["price"].to_f, id: id }
  )

  db.fetch_one("SELECT * FROM products WHERE id = :id", { id: id })
end

Tina4::GraphQL.mutation("deleteProduct") do |args|
  db = Tina4::Database.connection
# ...
```

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createProduct(name: \"Widget\", price: 9.99) { id name price } }"}'
```

```
{
  "data": {
    "createProduct": {
      "id": 6,
      "name": "Widget",
      "price": 9.99
    }
  }
}
```

7. Nested Queries (Relationships)

```
Tina4::GraphQL.query("users") do |args|
  db = Tina4::Database.connection
  users = db.fetch("SELECT id, name, email FROM users ORDER BY name")

  users.each do |user|
    user["posts"] = db.fetch("SELECT id, title FROM posts WHERE user_id = :id", { id: user["id"] })
  end

  users
end
```

```
curl -X POST http://localhost:7147/graphql \
```

```
  -H "Content-Type: application/json" \
  -d '{"query": "{ users { name email posts { title } } }"}'
```

```
{
  "data": {
    "users": [
      {
        "name": "Alice",
        "email": "alice@example.com",
        "posts": [
          {"title": "First Post"},
          {"title": "Second Post"}
        ]
      }
    ]
  }
}
```

8. Authentication in GraphQL

```
Tina4::GraphQL.query("me") do |args, context|
  if context[:user].nil?
    raise "Authentication required"
  end

  db = Tina4::Database.connection
  db.fetch_one("SELECT id, name, email, role FROM users WHERE id = :id", { id: context[:user]["user_id"] })
end
```

The `context` hash is populated by middleware that validates the JWT token.

9. GraphQL Playground

When `TINA4_DEBUG=true`, visit `/graphql/playground` for an interactive GraphQL IDE where you can:

• Write and execute queries

• Browse the schema documentation

• See auto-completion suggestions

• View query history

10. Exercise: Build a GraphQL API for a Blog

Requirements

1. Define queries: `posts` (list all), `post(id)` (get one with comments)

2. Define mutations: `createPost`, `addComment`

3. Include nested data (post with author and comments)

Test with:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
```

```
  -d '{"query": "{ posts { id title user { name } } }"}'

curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(title: \"Hello\", body: \"World\", userId: 1) { id title } }"}'
```

11. Solution

Create `src/routes/graphql_blog.rb`:

```
Tina4::GraphQL.query("posts") do |args|
  db = Tina4::Database.connection
  posts = db.fetch("SELECT * FROM posts WHERE published = 1 ORDER BY created_at DESC")

  posts.each do |post|
    post["user"] = db.fetch_one("SELECT id, name, email FROM users WHERE id = :id", { id: post["user_id"] })
  end

  posts
end

Tina4::GraphQL.query("post") do |args|
  db = Tina4::Database.connection
  post = db.fetch_one("SELECT * FROM posts WHERE id = :id", { id: args["id"].to_i })

  return nil if post.nil?

  post["user"] = db.fetch_one("SELECT id, name, email FROM users WHERE id = :id", { id: post["user_id"] })
  post["comments"] = db.fetch("SELECT * FROM comments WHERE post_id = :id ORDER BY created_at", { id: post["i

  post
end

Tina4::GraphQL.mutation("createPost") do |args|
  db = Tina4::Database.connection
# ...
```

12. Gotchas

1. N+1 Queries in Nested Resolvers

Problem: Fetching 100 posts with their authors makes 101 database queries.

Fix: Use batch loading or pre-fetch all authors in one query before resolving.

2. All Fields Returned Despite Client Selection

Problem: The response includes fields the client did not ask for.

Fix: Tina4's GraphQL engine handles field selection automatically. If all fields appear, ensure you are using the GraphQL endpoint, not a REST endpoint.

3. Mutations Return Null

Problem: A mutation executes but returns null.

Fix: Make sure your mutation block returns the created/updated object. `db.execute` returns affected rows, not the data. Follow it with a `db.fetch_one` to return the actual record.

4. Authentication Not Applied

Problem: GraphQL queries work without authentication.

Fix: Apply auth middleware to the `/graphql` endpoint, or check `context[:user]` in resolvers.

5. Query Too Complex

Problem: A deeply nested query causes a timeout.

Fix: Set query depth limits: `TINA4_GRAPHQL_MAX_DEPTH=5` in `.env`.

6. Error Messages Expose Internal Details

Problem: Database error messages appear in GraphQL responses.

Fix: In production (`TINA4_DEBUG=false`), Tina4 returns generic error messages. Use `begin/rescue` in resolvers to handle errors gracefully.

7. Cannot Use GET for Queries

Problem: `GET /graphql?query={products{id}}` returns a 404 or error.

Fix: GraphQL in Tina4 uses POST by default. All queries and mutations should be sent as POST requests with a JSON body.

Chapter 17: Testing

1. Why Tests Matter More Than You Think

It is Friday afternoon. Your client just reported a critical bug in production. You fix it -- one line of code. But did that fix break anything else? You have 47 routes, 12 ORM models, and 3 middleware functions. Manually clicking through every page would take an hour. Running the test suite takes 2 seconds.

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_list_products_with_filter

  AuthTest
    [PASS] test_login_with_valid_credentials
    [PASS] test_login_with_invalid_password
    [PASS] test_protected_route_without_token
    [PASS] test_protected_route_with_valid_token

  9 tests, 9 passed, 0 failed (0.34s)
```

Everything still works. You deploy with confidence and enjoy your weekend.

Tina4 supports RSpec for testing. Tests live in the `tests/` directory. Every `_spec.rb` file is auto-discovered when you run `tina4 test`.

2. Your First Test

Create `tests/basic_spec.rb`:

```
require "tina4"

RSpec.describe "Basic tests" do
  it "adds two numbers" do
    expect(2 + 2).to eq(4)
  end

  it "concatenates strings" do
    result = "Hello" + " " + "World"
    expect(result).to eq("Hello World")
  end

  it "handles nil correctly" do
    value = nil
    expect(value).to be_nil
  end
end
```

Run it:

```
tina4 test

Running tests...
```

```
  Basic tests
    [PASS] adds two numbers
    [PASS] concatenates strings
    [PASS] handles nil correctly

  3 tests, 3 passed, 0 failed (0.01s)
```

You can also use `bundle exec rspec` directly:

```
bundle exec rspec tests/
```

3. Testing Routes

The `Tina4::TestClient` lets you make HTTP requests to your routes without starting a server:

```
require "tina4"

RSpec.describe "Product API" do
  let(:client) { Tina4::TestClient.new }

  it "returns products list" do
    result = client.get("/api/products")

    expect(result.status).to eq(200)

    data = result.json
    expect(data).to have_key("products")
    expect(data["products"]).to be_an(Array)
  end

  it "creates a product" do
    result = client.post("/api/products", {
      name: "Test Widget",
      category: "Test",
      price: 9.99
    })

    expect(result.status).to eq(201)

    data = result.json
# ...
```

TestClient Methods

```
client = Tina4::TestClient.new

# GET request
result = client.get("/api/products")
result = client.get("/api/products?category=Electronics")

# POST request with JSON body
result = client.post("/api/products", { name: "Widget", price: 9.99 })

# PUT request
result = client.put("/api/products/1", { name: "Updated Widget" })

# DELETE request
result = client.delete("/api/products/1")

# With custom headers
result = client.get("/api/profile", headers: { "Authorization" => "Bearer #{token}" })
```

TestResult Properties

```
result.status      # HTTP status code (200, 201, 404, etc.)
result.json        # Parsed JSON body as a hash
result.body        # Raw response body as a string
result.headers     # Response headers hash
```

4. Testing ORM Models

```
require "tina4"

RSpec.describe Product do
  before(:each) do
    db = Tina4::Database.connection
    db.execute("DELETE FROM products")
  end

  it "creates and saves a product" do
    product = Product.new
    product.name = "Test Product"
    product.price = 29.99
    product.category = "Testing"
    product.save

    expect(product.id).not_to be_nil
    expect(product.name).to eq("Test Product")
  end

  it "loads a product by ID" do
    product = Product.new
    product.name = "Load Test"
    product.price = 19.99
    product.save

# ...
```

5. Testing Authentication

```
require "tina4"

RSpec.describe "Authentication" do
  let(:client) { Tina4::TestClient.new }

  before(:each) do
    # Register a test user
    client.post("/api/register", {
      name: "Test User",
      email: "test@example.com",
      password: "securePass123"
    })
  end

  it "logs in with valid credentials" do
    result = client.post("/api/login", {
      email: "test@example.com",
      password: "securePass123"
    })

    expect(result.status).to eq(200)
```

```
    expect(result.json).to have_key("token")
    expect(result.json["user"]["email"]).to eq("test@example.com")
  end

# ...
```

6. Test Database Isolation

Use a separate test database to avoid polluting development data:

```
# .env.test
DATABASE_URL=sqlite:///data/test.db
TINA4_DEBUG=false
```

In your test helper:

```
# tests/spec_helper.rb
ENV["TINA4_ENV"] = "test"
require "tina4"

RSpec.configure do |config|
  config.before(:suite) do
    # Run migrations on test database
    system("tina4 migrate --env test")
  end

  config.after(:suite) do
    # Clean up test database
    File.delete("data/test.db") if File.exist?("data/test.db")
  end
end
```

7. Running Tests

```
# Run all tests
tina4 test

# Run a specific test file
tina4 test tests/product_spec.rb

# Run with verbose output
tina4 test --verbose

# Using RSpec directly
bundle exec rspec tests/
bundle exec rspec tests/product_spec.rb
bundle exec rspec tests/ --format documentation
```

8. Exercise: Write Tests for a Notes API

Write a comprehensive test suite for the Notes API from Chapter 5.

Requirements

Test these scenarios:

1. Create a note with valid data (201)

2. Create a note with missing title (400)

3. List all notes (200)

4. Get a note by ID (200)

5. Get a non-existent note (404)

6. Update a note (200)

7. Delete a note (204)

8. Search notes by content

9. Filter notes by tag

9. Solution

Create `tests/notes_spec.rb`:

```
require "tina4"

RSpec.describe "Notes API" do
  let(:client) { Tina4::TestClient.new }

  before(:each) do
    db = Tina4::Database.connection
    db.execute("DELETE FROM notes")
  end

  it "creates a note with valid data" do
    result = client.post("/api/notes", {
      title: "Test Note",
      content: "This is a test",
      tag: "testing"
    })

    expect(result.status).to eq(201)
    expect(result.json["title"]).to eq("Test Note")
    expect(result.json["tag"]).to eq("testing")
    expect(result.json["id"]).not_to be_nil
  end

  it "rejects note with missing title" do
    result = client.post("/api/notes", {
# ...
```

10. Gotchas

1. Tests Share Database State

Problem: Tests pass individually but fail when run together.

Fix: Clean up in `before(:each)` blocks. Delete test data before each test.

2. TestClient Does Not Start a Real Server

Problem: External services (SMTP, Redis) are not available in tests.

Fix: Mock external services or use test doubles. The TestClient simulates HTTP requests without network I/O.

3. Test Order Dependency

Problem: Test B depends on data created by Test A.

Fix: Each test should set up its own data. Use `before(:each)` blocks to create required state.

4. Database Migrations Not Applied

Problem: Tests fail with "table does not exist" errors.

Fix: Run `tina4 migrate` before running tests, or add migration logic to your test setup.

5. Token Expired During Test

Problem: Auth tests fail intermittently with "token expired".

Fix: Set a long JWT expiry for tests: `TINA4_JWT_EXPIRY=86400` in `.env.test`.

6. Floating Point Comparison

Problem: `expect(product.price).to eq(9.99)` fails with `9.990000000000001`.

Fix: Use `be_within` for floating point: `expect(product.price).to be_within(0.01).of(9.99)`.

7. Test Output Too Verbose

Problem: Test output includes log messages from the application.

Fix: Set `TINA4_LOG_LEVEL=ERROR` in `.env.test` to suppress info and debug logs.

Chapter 18: Dev Tools

1. Debugging at 2am

It is 2am. Your production monitoring just pinged you -- a 500 error on the checkout endpoint. You pull up the dev dashboard, find the failing request in the request inspector, and see the full stack trace with source code context. Line 47 of `src/routes/checkout.rb` -- a nil reference on the shipping address because the user did not fill in the form. You add a nil check, push the fix, and go back to sleep. Total time: 30 seconds.

Tina4's dev tools are not an afterthought bolted on later. They are built into the framework from day one. When `TINA4_DEBUG=true`, you get a full development dashboard, an error overlay with source code, live reload, a request inspector, a SQL query runner, and more -- all without installing any additional packages.

2. Enabling the Dev Dashboard

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Navigate to `http://localhost:7147/tina4/console` and enter your token.

3. Dashboard Overview

System Overview

• **Framework version** -- The installed Tina4 Ruby version

• **Ruby version** -- The running Ruby version, loaded gems

• **Uptime** -- How long the server has been running

• **Memory usage** -- Current and peak memory consumption

• **Database status** -- Connection status, database engine, file size

• **Environment** -- Current `.env` variables (sensitive values are masked)

Request Inspector

Every HTTP request is logged with:

• Method, path, status code, response time

• Request headers, query parameters, body

• Response headers and body

• Database queries executed (with timing)

• Template renders (with timing)

Click any request to drill into the details. This replaces `puts` debugging entirely.

Error Log

Unhandled exceptions are captured with:

• Full stack trace with source code context

• The request that triggered the error

• Occurrence count and timestamps

• Ruby version and gem information

Queue Manager

See queue statistics, recent jobs, failed jobs, and dead letter queue.

WebSocket Monitor

Active WebSocket connections with metadata, message counts, and connection duration.

Routes

All registered routes with methods, paths, middleware, and auth requirements.

4. The Debug Overlay

When `TINA4_DEBUG=true`, every HTML page shows a debug toolbar at the bottom:

• Request details (method, URL, duration)

• Database queries executed (with timing and SQL)

• Template renders (with timing)

• Session data

• Recent log entries

Click any section to expand it. The overlay is injected automatically -- you do not add any code.

5. SQL Query Runner

The dev dashboard includes an interactive SQL query runner:

```
http://localhost:7147/tina4/console#sql
```

Type any SQL query and see the results instantly. Useful for exploring your data without a separate database client:

```
SELECT * FROM products WHERE price > 50 ORDER BY price DESC
```

The runner shows the result as a formatted table with column types and row count.

6. Log Viewer

View application logs in real time from the dashboard:

```
http://localhost:7147/tina4/console#logs
```

Filter by level (DEBUG, INFO, WARNING, ERROR), search by keyword, and view timestamps. In development, all log levels are shown. In production with `TINA4_LOG_LEVEL=WARNING`, only warnings and errors appear.

Logging from Code

```
Tina4::Logger.debug("Processing order #{order_id}")
Tina4::Logger.info("User #{user_id} logged in")
Tina4::Logger.warning("Rate limit approaching for IP #{ip}")
Tina4::Logger.error("Failed to send email: #{error.message}")
```

Logs are written to `logs/app.log` and displayed in the dev dashboard.

7. Live Reload

When `TINA4_DEBUG=true`, the dev server watches your files for changes and automatically restarts:

• Route files (`src/routes/*.rb`)

• ORM models (`src/orm/*.rb`)

• Templates (`src/templates/*`)

• The `.env` file

When you save a file, the server reloads within 1 second. The browser page refreshes automatically if you have the debug overlay enabled.

8. Error Pages

Development Error Page

When `TINA4_DEBUG=true`, unhandled exceptions show a detailed error page with:

• The exception class and message

• The full stack trace with source code context (5 lines above and below)

• The request details (method, path, headers, body)

• Environment variables

• Database connection status

Production Error Page

When `TINA4_DEBUG=false`, users see a generic error page. Create custom error pages:

• `src/templates/errors/404.html` -- Page not found

• `src/templates/errors/500.html` -- Internal server error

9. Health Check Endpoint

The `/health` endpoint reports application status:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3600,
  "version": "3.0.0",
  "framework": "tina4-ruby",
  "ruby_version": "3.3.0",
  "memory_mb": 42.5,
  "pid": 12345
}
```

In production with `TINA4_DEBUG=false`, create a `.broken` file in the project root to make the health check return a failure status. This is useful for graceful shutdown: set the file, wait for the load balancer to stop sending traffic, then restart the server.

10. Exercise: Debug a Broken Endpoint

Create a route with an intentional bug and use the dev tools to find and fix it.

Setup

Create `src/routes/buggy.rb`:

```
Tina4::Router.get("/api/orders/{id:int}/total") do |request, response|
  db = Tina4::Database.connection
  id = request.params["id"]

  order = db.fetch_one("SELECT * FROM orders WHERE id = :id", { id: id })

  # Bug: order might be nil, causing a NoMethodError
  items = db.fetch("SELECT * FROM order_items WHERE order_id = :id", { id: order["id"] })

  total = items.sum { |item| item["price"] * item["quantity"] }

  response.json({ order_id: id, total: total, items: items.length })
end
```

Test

```
curl http://localhost:7147/api/orders/999/total
```

Task

1. Visit the dev dashboard and find the error in the request inspector

2. Read the stack trace to identify the bug

3. Fix the nil check

4. Verify the fix works

11. Solution

The bug is on the line `order["id"]` -- when the order does not exist, `order` is nil, causing a `NoMethodError: undefined method '[]' for nil:NilClass`.

Fix:

```
Tina4::Router.get("/api/orders/{id:int}/total") do |request, response|
  db = Tina4::Database.connection
  id = request.params["id"]

  order = db.fetch_one("SELECT * FROM orders WHERE id = :id", { id: id })

  if order.nil?
    return response.json({ error: "Order not found", id: id }, 404)
  end

  items = db.fetch("SELECT * FROM order_items WHERE order_id = :id", { id: order["id"] })

  total = items.sum { |item| item["price"] * item["quantity"] }

  response.json({ order_id: id, total: total, items: items.length })
end
```

12. Gotchas

1. Console Token Exposed in .env

Problem: Anyone with access to your `.env` can access the dev dashboard.

Fix: Use a strong random token. Never commit `.env` to version control. In shared environments, restrict access by IP.

2. Debug Mode in Production

Problem: Stack traces and database queries visible to users.

Fix: Always set `TINA4_DEBUG=false` in production. Set `TINA4_CONSOLE=false` to disable the dashboard entirely.

3. Log Files Growing Without Bound

Problem: `logs/app.log` grows to gigabytes over time.

Fix: Use log rotation. Set `TINA4_LOG_MAX_SIZE=10mb` and `TINA4_LOG_MAX_FILES=5` to automatically rotate logs.

4. Live Reload Causes Crashes

Problem: The server crashes during live reload when a file has syntax errors.

Fix: Fix the syntax error and save the file again. The watcher will detect the change and attempt to reload. Check the terminal for the error message.

5. Debug Overlay Breaks Page Layout

Problem: The debug toolbar interferes with your page's CSS.

Fix: The overlay uses isolated CSS with high specificity. If conflicts occur, it is usually because your CSS uses `!important` on body or footer elements. The overlay only appears when `TINA4_DEBUG=true`.

6. SQL Runner Allows Destructive Queries

Problem: Someone runs `DROP TABLE products` in the SQL runner.

Fix: The SQL runner is only available via the dev dashboard, which requires the console token. Never expose the console in production. The SQL runner is a power tool for development only.

7. Memory Usage Grows During Development

Problem: The server's memory usage increases over time during development.

Fix: This is normal during live reload -- each reload may not fully release old objects. Restart the server periodically during long development sessions.

Chapter 19: CLI & Scaffolding

1. Getting a New Developer Up to Speed

A new developer joins your team on Monday morning. You hand them the repo URL, and by 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No hunting through documentation. No copy-pasting boilerplate from old projects. The Tina4 CLI handles the scaffolding.

The Tina4 CLI is a single Rust binary that manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). The commands are identical across languages. Once you learn the CLI for Ruby, you know it for PHP too.

2. tina4 init -- Project Scaffolding

```
tina4 init my-project
```

Language Detection

The CLI detects the language from the directory contents:

| File Found | Language Detected |

|----------|----------|

| `Gemfile` | Ruby |

| `composer.json` | PHP |

| `requirements.txt` or `pyproject.toml` | Python |

| `package.json` | Node.js |

If no language-specific file exists, the CLI asks which language to use.

Specifying Language Explicitly

```
tina4 init my-project --lang ruby
```

3. tina4 serve -- Dev Server

```
tina4 serve

  Tina4 Ruby v3.0.0
  Server running at http://0.0.0.0:7147
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Options

```
tina4 serve --port 8080        # Custom port
tina4 serve --host 0.0.0.0     # Listen on all interfaces
tina4 serve --no-reload        # Disable live reload
```

The dev server uses WEBrick by default. For production, use Puma:

```
bundle exec puma -C config/puma.rb
```

Or start with `bundle exec ruby app.rb` which starts the built-in server.

4. tina4 routes -- Route Listing

```
tina4 routes
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/products                 -                   public
POST     /api/products                 -                   secured
GET      /api/products/{id:int}        -                   public
PUT      /api/products/{id:int}        -                   secured
DELETE   /api/products/{id:int}        -                   secured
GET      /api/users                    auth_middleware      public
```

Filtering

```
tina4 routes --method POST
tina4 routes --filter products
tina4 routes --filter admin
```

5. tina4 migrate -- Database Migrations

```
# Create a new migration
tina4 migrate:create create_products_table

# Run pending migrations
tina4 migrate

# Check migration status
tina4 migrate:status

# Roll back the last migration
tina4 migrate:rollback

# Roll back all migrations
tina4 migrate:reset

# Roll back and re-run all migrations
tina4 migrate:fresh
```

6. tina4 generate -- Code Generation

Generate a Model

```
tina4 generate:model Product
```

Creates `src/orm/product.rb`:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true
  # Add your fields here

  table_name "products"
end
```

Generate a Route

```
tina4 generate:route products
```

Creates `src/routes/products.rb` with CRUD route stubs:

```
Tina4::Router.get("/api/products") do |request, response|
  # TODO: implement
  response.json({ products: [] })
end

Tina4::Router.get("/api/products/{id:int}") do |request, response|
  # TODO: implement
  response.json({ id: request.params["id"] })
end

Tina4::Router.post("/api/products") do |request, response|
  # TODO: implement
  response.json({ created: true }, 201)
end

Tina4::Router.put("/api/products/{id:int}") do |request, response|
  # TODO: implement
  response.json({ updated: true })
end

Tina4::Router.delete("/api/products/{id:int}") do |request, response|
  # TODO: implement
  response.json(nil, 204)
end
```

Generate a Migration

```
tina4 generate:migration create_products_table
```

Creates `src/migrations/TIMESTAMP_create_products_table.sql`.

Generate All Three at Once

```
tina4 generate:crud Product
```

Creates the model, migration, and route file for a complete CRUD resource.

7. tina4 test -- Running Tests

```
tina4 test                          # Run all tests
tina4 test tests/product_spec.rb    # Run a specific file
tina4 test --verbose                # Verbose output
tina4 test --coverage               # Generate coverage report
```

8. tina4 queue -- Queue Management

```
tina4 queue:work                    # Start processing all queues
tina4 queue:work --queue emails     # Process specific queue
tina4 queue:dead                    # View dead letter queue
tina4 queue:retry 42                # Retry a dead letter job
tina4 queue:retry --all             # Retry all dead letter jobs
tina4 queue:clear --older-than 7d   # Clear old dead letter jobs
tina4 queue:stats                   # Show queue statistics
```

9. tina4 build -- Build Commands

```
tina4 build:css                     # Compile SCSS to CSS
tina4 build:js                      # Bundle and minify JavaScript
tina4 build                         # Run all build steps
```

10. tina4 deploy -- Deployment

```
tina4 deploy:docker                 # Generate Dockerfile and docker-compose.yml
tina4 deploy:systemd                # Generate systemd service file
tina4 deploy:nginx                  # Generate Nginx config
```

11. Environment-Specific Commands

```
tina4 serve --env production        # Use .env.production
tina4 migrate --env test            # Run migrations on test database
tina4 test --env test               # Run tests with test environment
```

12. Exercise: Scaffold a Complete Project

Using only the CLI, scaffold a project called `taskflow` with:

1. A `User` model and migration

2. A `Task` model and migration

3. CRUD routes for both

4. Run the migrations

5. Start the server

Test with:

```
tina4 init taskflow --lang ruby
cd taskflow
tina4 generate:crud User
tina4 generate:crud Task
tina4 migrate
tina4 serve
```

Then verify the routes:

```
tina4 routes
```

13. Solution

```
tina4 init taskflow --lang ruby
cd taskflow
bundle install

# Generate User CRUD
```

```
tina4 generate:crud User

# Generate Task CRUD
tina4 generate:crud Task

# Edit the generated migrations to add fields...
# Edit the generated models to add fields...

# Run migrations
tina4 migrate

# Start the server
tina4 serve
```

After editing, verify:

```
tina4 routes

Method   Path                     Middleware   Auth
------   ----                     ----------   ----
GET      /api/users               -            public
POST     /api/users               -            secured
GET      /api/users/{id:int}      -            public
PUT      /api/users/{id:int}      -            secured
DELETE   /api/users/{id:int}      -            secured
GET      /api/tasks               -            public
POST     /api/tasks               -            secured
GET      /api/tasks/{id:int}      -            public
PUT      /api/tasks/{id:int}      -            secured
DELETE   /api/tasks/{id:int}      -            secured
```

14. Gotchas

1. CLI Not Found

Problem: `tina4: command not found`.

Fix: Install the CLI with `brew install tina4stack/tap/tina4` or the install script.

2. Wrong Language Detected

Problem: The CLI detects PHP when you wanted Ruby.

Fix: Use `--lang ruby` to force the language.

3. Migration Timestamp Collisions

Problem: Two migrations created in the same second have the same timestamp.

Fix: Wait a second between `migrate:create` calls, or rename the file manually.

4. generate:crud Overwrites Existing Files

Problem: Running `generate:crud Product` again overwrites your customized files.

Fix: The generator checks for existing files and skips them by default. If you force overwrite with `--force`, your changes will be lost. Use version control.

5. serve Uses Wrong Port

Problem: `tina4 serve` starts on port 7145 instead of 7147.

Fix: Ruby uses port 7147 by default. If it starts on a different port, check your `.env` file for `TINA4_PORT`.

6. build:css Fails

Problem: `tina4 build:css` fails with "sass not found".

Fix: Install the Sass compiler: `gem install sass` or `npm install -g sass`.

7. deploy:docker Generates PHP Dockerfile

Problem: The generated Dockerfile uses PHP instead of Ruby.

Fix: Make sure your project has a `Gemfile` so the CLI detects Ruby. Or use `tina4 deploy:docker --lang ruby`.

Chapter 20: Deployment

1. From Development to Production

You have built the app. It works perfectly on `localhost:7147`. Now it needs to run 24/7 on a real server, handle 10,000 concurrent users, survive server restarts, and not leak memory. The gap between "it works on my machine" and "it works in production" is where most projects stumble.

This chapter covers everything you need to deploy a Tina4 Ruby application to production: environment configuration, Docker packaging, web server setup, SSL, scaling, monitoring, and graceful shutdown handling.

2. Production .env Configuration

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7147

# Database
DATABASE_URL=sqlite:///data/app.db

# Security
CORS_ORIGINS=https://yourdomain.com
JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_CACHE_TEMPLATES=true
TINA4_MINIFY_HTML=true

# Console (disabled in production)
TINA4_CONSOLE=false
```

Key Differences from Development

| Setting | Development | Production | Why |

|---------|------------|------------|-----|

| `TINA4_DEBUG` | `true` | `false` | Hides stack traces, disables toolbar |

| `TINA4_LOG_LEVEL` | `ALL` | `WARNING` | Reduces log noise |

| `CORS_ORIGINS` | `*` | Your domain | Prevents cross-origin abuse |

| `TINA4_CONSOLE` | `true` | `false` | Hides dev dashboard |

| `TINA4_CACHE_TEMPLATES` | `false` | `true` | Caches compiled templates |

| `TINA4_MINIFY_HTML` | `false` | `true` | Reduces response size |

3. Docker Deployment

Dockerfile

```
FROM ruby:3.3-slim

WORKDIR /app

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libsqlite3-dev \
    && rm -rf /var/lib/apt/lists/*

# Install gems
COPY Gemfile Gemfile.lock ./
RUN bundle install --without development test

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p data logs secrets

# Run migrations
RUN tina4 migrate 2>/dev/null || true

# Expose port
EXPOSE 7147
# ...
```

docker-compose.yml

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7147:7147"
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
      - app-secrets:/app/secrets
    env_file:
      - .env.production
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:7147/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  worker:
    build: .
    command: tina4 queue:work
    volumes:
      - app-data:/app/data
# ...
```

Build and Run

```
docker compose build
docker compose up -d
docker compose logs -f app
```

4. Puma Configuration

Create `config/puma.rb`:

```
# Puma configuration for production

# Port
port ENV.fetch("TINA4_PORT", 7147)

# Workers (processes) -- set to number of CPU cores
workers ENV.fetch("WEB_CONCURRENCY", 2)

# Threads per worker
threads_count = ENV.fetch("RAILS_MAX_THREADS", 5)
threads threads_count, threads_count

# Environment
environment ENV.fetch("RACK_ENV", "production")

# Preload app for faster worker boot
preload_app!

# PID file
pidfile ENV.fetch("PIDFILE", "tmp/pids/server.pid")

# Logging
stdout_redirect "logs/puma.stdout.log", "logs/puma.stderr.log", true

# Worker timeout
# ...
```

Start with Puma:

```
bundle exec puma -C config/puma.rb
```

5. Nginx Reverse Proxy

Create `config/nginx.conf`:

```
upstream tina4_app {
    server 127.0.0.1:7147;
}

server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Static files
    location /css/ {
        root /app/src/public;
        expires 30d;
        add_header Cache-Control "public, immutable";
```

```
    }

    location /js/ {
# ...
```

6. SSL with Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com
```

Certbot automatically configures Nginx with SSL and sets up auto-renewal.

7. Process Management with systemd

Create `/etc/systemd/system/tina4-app.service`:

```
[Unit]
Description=Tina4 Ruby Application
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/app
ExecStart=/usr/local/bin/bundle exec puma -C config/puma.rb
Restart=always
RestartSec=5
Environment=RACK_ENV=production

[Install]
WantedBy=multi-user.target
```

Create `/etc/systemd/system/tina4-worker.service`:

```
[Unit]
Description=Tina4 Queue Worker
After=network.target tina4-app.service

[Service]
Type=simple
User=deploy
WorkingDirectory=/app
ExecStart=/usr/local/bin/tina4 queue:work
Restart=always
RestartSec=5
Environment=RACK_ENV=production

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable tina4-app tina4-worker
sudo systemctl start tina4-app tina4-worker
sudo systemctl status tina4-app
```

8. Graceful Shutdown

Tina4 handles graceful shutdown automatically. When it receives a `SIGTERM`:

1. Stops accepting new connections

2. Waits for in-flight requests to complete (up to 30 seconds)

3. Closes database connections

4. Shuts down cleanly

For the health check pattern:

```
# Create .broken file to signal the load balancer
touch .broken

# Wait for health checks to fail and traffic to drain
sleep 30

# Restart the application
sudo systemctl restart tina4-app

# Remove the .broken file
rm .broken
```

9. Monitoring

Health Check

```
curl http://localhost:7147/health
```

Log Monitoring

```
tail -f logs/app.log
tail -f logs/puma.stdout.log
```

Prometheus Metrics

Enable metrics in `.env`:

```
TINA4_METRICS=true
```

Metrics are available at `/metrics` in Prometheus format:

```
curl http://localhost:7147/metrics
```

10. Scaling

Horizontal Scaling with Multiple Workers

In `config/puma.rb`:

```
workers ENV.fetch("WEB_CONCURRENCY", 4)
threads 5, 5
```

Scaling with Docker Compose

```
docker compose up -d --scale app=4
```

Scaling Queue Workers

```
docker compose up -d --scale worker=3
```

11. Exercise: Deploy to Docker

Deploy your TaskFlow application to Docker with:

1. A production `.env`

2. A Dockerfile

3. A docker-compose.yml with app and worker services

4. Health check configuration

Test with:

```
docker compose build
docker compose up -d
curl http://localhost:7147/health
curl http://localhost:7147/api/products
docker compose logs -f
docker compose down
```

12. Solution

Create `.env.production`, `Dockerfile`, and `docker-compose.yml` as shown in sections 2, 3, and 4 above. Then:

```
docker compose build
docker compose up -d
```

Verify:

```
curl http://localhost:7147/health
```

```
{"status":"ok","database":"connected","version":"3.0.0","framework":"tina4-ruby"}
```

13. Gotchas

1. Database File Not Persisted

Problem: Data disappears after Docker container restarts.

Fix: Mount the `data/` directory as a Docker volume.

2. .env Not Loaded in Docker

Problem: Environment variables are not set inside the container.

Fix: Use `env_file: .env.production` in docker-compose.yml, not `COPY .env .`.

3. WebSocket Not Working Behind Nginx

Problem: WebSocket connections fail behind Nginx.

Fix: Add `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"` to the Nginx config.

4. Puma Workers Not Reconnecting to Database

Problem: Database errors after Puma forks worker processes.

Fix: Use `on_worker_boot` in Puma config to reconnect: `Tina4::Database.reconnect`.

5. SSL Certificate Renewal Fails

Problem: Let's Encrypt certificate expires because auto-renewal failed.

Fix: Run `sudo certbot renew --dry-run` to test renewal. Set up a cron job: `0 3 * * * certbot renew`.

6. Memory Grows Over Time

Problem: The application's memory usage increases steadily.

Fix: Set Puma's `worker_timeout` and restart workers periodically with `workers_max_requests 1000`.

7. Queue Worker Stops Processing

Problem: The queue worker stops processing jobs silently.

Fix: Use `Restart=always` in systemd. Monitor the worker with health checks. Set up alerting on dead letter queue growth.

Chapter 21: Building a Complete App

1. Putting It All Together

Over the last 20 chapters, you have learned routing, templates, databases, ORM, authentication, middleware, queues, WebSocket, caching, frontend, GraphQL, testing, dev tools, CLI scaffolding, and deployment. Now let us use all of it to build a real application from scratch.

We are building **TaskFlow** -- a task management system with:

• User registration and JWT authentication

• Task creation, assignment, and tracking

• A dashboard with real-time updates

• Email notifications when tasks are assigned

• Caching for dashboard performance

• A full test suite

• Docker deployment

This is not a toy project. It is a complete, production-ready application that demonstrates every major Tina4 feature working together.

2. Planning the App

Models

| Model | Table | Fields |

|-------|-------|--------|

| User | users | id, name, email, password_hash, role, created_at |

| Task | tasks | id, title, description, status, priority, created_by, assigned_to, due_date, completed_at, created_at, updated_at |

Relationships

• A User has many Tasks (created by them)

• A User has many Tasks (assigned to them)

• A Task belongs to a User (creator)

• A Task belongs to a User (assignee)

Routes

| Method | Path | Description | Auth |

|--------|------|-------------|------|

| POST | /api/auth/register | Register a new user | public |

| POST | /api/auth/login | Login, get JWT | public |

| GET | /api/profile | Get current user profile | secured |

| GET | /api/tasks | List tasks (with filters) | secured |

| GET | /api/tasks/{id} | Get a single task | secured |

| POST | /api/tasks | Create a task | secured |

| PUT | /api/tasks/{id} | Update a task | secured |

| DELETE | /api/tasks/{id} | Delete a task | secured |

| GET | /api/dashboard/stats | Dashboard statistics | secured |

| GET | /admin | Dashboard HTML page | public |

Templates

• `base.html` -- Base layout with sidebar and topbar

• `dashboard.html` -- Dashboard with stats, task list, quick actions

• `login.html` -- Login page

• `register.html` -- Registration page

3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow --lang ruby
cd taskflow
bundle install
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=dev-token
JWT_SECRET=taskflow-dev-secret-change-in-production
JWT_EXPIRY=86400
```

Create Migrations

Create `src/migrations/20260322150000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322150100_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
```

```
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run migrations:

```
tina4 migrate
```

4. Step 2: ORM Models

Create `src/orm/user.rb`:

```
class User < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :email
  string_field :password_hash
  string_field :role, default: "user"
  string_field :created_at

  table_name "users"

  has_many :created_tasks, class_name: "Task", foreign_key: "created_by"
  has_many :assigned_tasks, class_name: "Task", foreign_key: "assigned_to"
end
```

Create `src/orm/task.rb`:

```
class Task < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :title
  string_field :description, default: ""
  string_field :status, default: "todo"
  string_field :priority, default: "medium"
  integer_field :created_by
  integer_field :assigned_to
  string_field :due_date
  string_field :completed_at
  string_field :created_at
  string_field :updated_at

  table_name "tasks"

  belongs_to :creator, class_name: "User", foreign_key: "created_by"
  belongs_to :assignee, class_name: "User", foreign_key: "assigned_to"
end
```

5. Step 3: Authentication

Create `src/routes/middleware.rb`:

```
def auth_middleware(request, response, next_handler)
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization required" }, 401)
  end

  token = auth_header.sub("Bearer ", "")

  unless Tina4::Auth.validate_token(token)
    return response.json({ error: "Invalid or expired token" }, 401)
  end

  request.user = Tina4::Auth.get_payload(token)
  next_handler.call(request, response)
end
```

Create `src/routes/auth.rb`:

```
# @noauth
Tina4::Router.post("/api/auth/register") do |request, response|
  body = request.body

  errors = []
  errors << "Name is required" if body["name"].nil? || body["name"].empty?
  errors << "Email is required" if body["email"].nil? || body["email"].empty?
  errors << "Password must be at least 8 characters" if body["password"].nil? || body["password"].to_s.length

  return response.json({ errors: errors }, 400) unless errors.empty?

  db = Tina4::Database.connection
  existing = db.fetch_one("SELECT id FROM users WHERE email = :email", { email: body["email"] })
  return response.json({ error: "Email already registered" }, 409) unless existing.nil?

  hash = Tina4::Auth.hash_password(body["password"])
  db.execute("INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
    { name: body["name"], email: body["email"], hash: hash })

  user = db.fetch_one("SELECT id, name, email, role FROM users WHERE id = last_insert_rowid()")
  response.json({ message: "Registration successful", user: user }, 201)
end

# @noauth
Tina4::Router.post("/api/auth/login") do |request, response|
# ...
```

6. Step 4: Task CRUD

Create `src/routes/tasks.rb`:

```
Tina4::Router.group("/api/tasks", middleware: "auth_middleware") do

  # List tasks with filters
  Tina4::Router.get("") do |request, response|
    db = Tina4::Database.connection
    user_id = request.user["user_id"]

    status = request.query["status"]
```

```
    priority = request.query["priority"]
    assigned = request.query["assigned_to_me"]

    sql = "SELECT t.*, u1.name AS creator_name, u2.name AS assignee_name FROM tasks t LEFT JOIN users u1 ON t
    conditions = []
    params = {}

    if assigned == "true"
      conditions << "t.assigned_to = :user_id"
      params[:user_id] = user_id
    end

    if status && !status.empty?
      conditions << "t.status = :status"
      params[:status] = status
    end

# ...
```

7. Step 5: Dashboard

Create `src/routes/dashboard.rb`:

```
Tina4::Router.get("/api/dashboard/stats", middleware: "auth_middleware") do |request, response|
  user_id = request.user["user_id"]

  stats = Tina4::Cache.fetch("dashboard:stats:#{user_id}", ttl: 60) do
    db = Tina4::Database.connection

    total = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE assigned_to = :id OR created_by = :id", {
    todo = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = :id OR created_by = :id) AN
    in_progress = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = :id OR created_by =
    done = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = :id OR created_by = :id) AN
    overdue = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = :id OR created_by = :id)

    {
      total: total["count"],
      todo: todo["count"],
      in_progress: in_progress["count"],
      done: done["count"],
      overdue: overdue["count"]
    }
  end

  response.json(stats)
end

Tina4::Router.get("/admin") do |request, response|
# ...
```

8. Step 6: Email Consumer

Create the queue consumer for task assignment notifications:

```
Tina4::Queue.consume("send-email") do |job|
  mail = Tina4::Messenger.new
  mail.to = job.payload["to"]
  mail.subject = job.payload["subject"]
```

```
  if job.payload["template"]
    mail.html_template = job.payload["template"]
    mail.template_data = job.payload["data"] || {}
  else
    mail.body = job.payload["body"] || ""
  end

  mail.send
  true
end
```

9. Step 7: Tests

Create `tests/taskflow_spec.rb`:

```
require "tina4"

RSpec.describe "TaskFlow" do
  let(:client) { Tina4::TestClient.new }
  let(:token) do
    client.post("/api/auth/register", { name: "Test", email: "test#{rand(10000)}@example.com", password: "pass
    result = client.post("/api/auth/login", { email: "test@example.com", password: "password123" })
    result.json["token"]
  end

  it "registers a user" do
    result = client.post("/api/auth/register", {
      name: "Alice", email: "alice#{rand(10000)}@example.com", password: "securePass123"
    })
    expect(result.status).to eq(201)
  end

  it "creates a task" do
    result = client.post("/api/tasks", { title: "Test Task" },
      headers: { "Authorization" => "Bearer #{token}" })
    expect(result.status).to eq(201)
    expect(result.json["title"]).to eq("Test Task")
    expect(result.json["status"]).to eq("todo")
  end

# ...
```

10. Step 8: Docker Deployment

Use the Dockerfile and docker-compose.yml from Chapter 20 with the TaskFlow-specific `.env.production`.

11. Gotchas

1. Circular Dependencies Between Models

Problem: User loads tasks, tasks load users, infinite loop.

Fix: Use lazy loading. Do not eager-load relationships that create cycles.

2. Cache Not Invalidated on Related Model Changes

Problem: Dashboard stats are stale after task updates.

Fix: Invalidate cache in every write operation: `Tina4::Cache.delete_pattern("dashboard:*")`.

3. Queue Worker Not Processing Emails

Problem: Task assignment emails are queued but never sent.

Fix: Run `tina4 queue:work` in a separate terminal or as a systemd service.

4. Token Expired During Long Sessions

Problem: Users get logged out while actively using the dashboard.

Fix: Set a reasonable JWT expiry (`86400` for 24 hours) and implement token refresh.

5. WebSocket Notifications Not Received

Problem: Real-time dashboard updates do not work.

Fix: Ensure the WebSocket endpoint is configured and the client JavaScript connects correctly.

6. Database Locked Under Load

Problem: SQLite returns "database is locked" errors with concurrent requests.

Fix: For production with concurrent users, switch to PostgreSQL. SQLite handles one writer at a time.

7. Missing Migration Before Deployment

Problem: The app crashes in production because a table does not exist.

Fix: Always run `tina4 migrate` as part of your deployment pipeline.

Chapter 22: Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Imagine telling your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI would need to know which packages to install, which config files to create, which naming conventions to follow, and how to wire everything together. It would hallucinate half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature, every import path, every convention. It generates correct, runnable code on the first try because there is only one way to do things in Tina4.

This is not accidental. Tina4 was designed from the ground up to be the best framework for AI-assisted development. Here is why.

The Zero-Dependency Advantage

When an AI generates code for Rails, it might suggest `ActiveRecord::Base` or `ApplicationRecord` -- both exist, both work differently. It might suggest `render json:` or `respond_to do |format|`. Which one? The AI has to guess.

With Tina4, there is one ORM. One queue. One template engine. One cache. No ambiguity, no alternatives, no "it depends on which gem you installed." The AI does not need to ask -- it knows.

```
# There is only one way to cache in Tina4
cached = Tina4::Cache.fetch("products", ttl: 300) { db.fetch("SELECT * FROM products") }

# There is only one way to queue in Tina4
Tina4::Queue.produce("emails", { to: "user@test.com", subject: "Welcome" })

# There is only one way to send email in Tina4
mail = Tina4::Messenger.new
mail.to = "user@test.com"
mail.subject = "Welcome"
mail.send
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works. It contains:

• Every method signature with parameters and return types

• The project structure convention (`src/routes/`, `src/orm/`, `src/templates/`)

• The `.env` variable reference

• Code style rules (routes use `response.json`, ORM uses `to_h`)

• Database connection string formats

• Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

The Convention Advantage

AI thrives on convention. When every Tina4 project follows the same structure, the AI never has to ask "where should I put this?"

```
src/
  routes/hello.rb     -> AI knows: this is a route file
  orm/product.rb      -> AI knows: this is an ORM model
  templates/page.html -> AI knows: this is a Frond template
  migrations/         -> AI knows: SQL migration files go here
```

The AI generates code that drops into the right directory with the right naming convention, every time.

Prompt Engineering for Tina4

Here are effective prompts for AI-assisted Tina4 Ruby development:

Creating a New Feature

```
Add a product catalog to my Tina4 Ruby project:
1. Create a Product model with fields: name, category, price, in_stock
2. Create a migration for the products table
3. Create CRUD routes at /api/products
4. Create a product listing template with category filters
5. Add response caching for the product list (5 minutes)
```

The AI generates all five files with correct Tina4 syntax because it read CLAUDE.md.

Adding Authentication

```
Add JWT authentication to my Tina4 Ruby project:
1. Create a users table migration with name, email, password_hash, role
```

```
2. Create register and login routes at /api/auth/register and /api/auth/login
3. Create auth middleware that validates Bearer tokens
4. Protect the /api/tasks group with auth middleware
5. Use Tina4::Auth.create_token and Tina4::Auth.validate_token
```

Building a Dashboard

```
Create an admin dashboard for my Tina4 Ruby project:
1. Create a GET /admin route that renders a dashboard template
2. The dashboard shows: total users, total products, recent orders
3. Use tina4css for styling (cards, tables, grid)
4. Use frond.js for AJAX data loading
5. Cache the dashboard stats for 60 seconds
```

Adding WebSocket

```
Add a real-time notification system to my Tina4 Ruby project:
1. Create a WebSocket endpoint at /ws/notifications/{user_id}
2. When a task is assigned, push a notification via WebSocket
3. Include a JavaScript snippet that connects and shows browser notifications
4. Auto-reconnect on disconnect
```

The Ruby Advantage for AI

Ruby's expressiveness makes AI-generated code particularly clean:

Block Syntax

```
Tina4::Router.get("/api/products") do |request, response|
  products = Tina4::Cache.fetch("products:all", ttl: 300) do
    db = Tina4::Database.connection
    db.fetch("SELECT * FROM products ORDER BY name")
  end

  response.json({ products: products, count: products.length })
end
```

The block syntax is unambiguous. The AI knows exactly where the route handler starts and ends. No curly brace confusion, no indentation ambiguity.

Hash Syntax

```
response.json({
  user: { id: user.id, name: user.name, email: user.email },
  tasks: tasks.map(&:to_h),
  stats: { total: total, completed: completed }
})
```

Ruby's symbol-key hash syntax (`name:` instead of `"name" =>`) is concise and consistent. The AI generates clean, idiomatic Ruby.

Enumerable Methods

```
# Filter products
electronics = products.select { |p| p[:category] == "Electronics" }

# Transform data
names = products.map { |p| p[:name] }

# Aggregate
total = items.sum { |item| item[:price] * item[:quantity] }
```

```
# Count with condition
featured_count = products.count { |p| p[:featured] }
```

These are the Ruby idioms the AI knows and uses. No external libraries, no complex method chains -- just Ruby.

Vibe Coding Workflow

The "vibe coding" workflow with AI and Tina4 looks like this:

1. Describe What You Want

Tell the AI what you need in plain English. Be specific about the data model, the endpoints, and the behavior.

2. Review the Generated Code

The AI generates the routes, models, migrations, and templates. Review them for correctness and completeness.

3. Run and Test

```
tina4 migrate
tina4 serve
```

Test the endpoints with curl or the Swagger UI. If something is wrong, tell the AI what needs to change.

4. Iterate

"The product list should be paginated. Add page and per_page query parameters." The AI updates the code.

5. Deploy

```
docker compose build
docker compose up -d
```

From idea to production in minutes, not days.

Real-World Example: Building an API in 5 Minutes

Here is a real conversation with an AI assistant:

You: "Add a blog to my Tina4 Ruby app. Posts with title, body, published flag, and user_id. CRUD API at /api/posts. Only published posts should be visible to anonymous users. Authors can see their own drafts."

AI generates:

1. `src/migrations/20260322_create_posts_table.sql` -- the migration

2. `src/orm/post.rb` -- the model with fields and relationships

3. `src/routes/posts.rb` -- CRUD routes with authentication and filtering

4. `tests/posts_spec.rb` -- RSpec tests for all scenarios

All using correct Tina4 conventions:

```
Tina4::Router.get("/api/posts") do |request, response|
  db = Tina4::Database.connection

  # Check if user is authenticated (optional)
  user_id = nil
  auth_header = request.headers["Authorization"] || ""
  if auth_header.start_with?("Bearer ")
    token = auth_header.sub("Bearer ", "")
    if Tina4::Auth.validate_token(token)
      payload = Tina4::Auth.get_payload(token)
      user_id = payload["user_id"]
    end
  end

  if user_id
    # Authenticated: show published + own drafts
    posts = db.fetch(
      "SELECT * FROM posts WHERE published = 1 OR user_id = :user_id ORDER BY created_at DESC",
      { user_id: user_id }
    )
  else
    # Anonymous: show published only
    posts = db.fetch("SELECT * FROM posts WHERE published = 1 ORDER BY created_at DESC")
  end

# ...
```

The AI knows:

• `Tina4::Router.get` for routes

• `request.headers["Authorization"]` for the token

• `Tina4::Auth.validate_token` and `Tina4::Auth.get_payload` for JWT

• `db.fetch` with named parameters for queries

• `response.json` for JSON responses

It did not hallucinate. It did not guess. It read CLAUDE.md and wrote correct code.

Why This Matters

Traditional development: you write every line, look up every API, debug every typo. A simple CRUD feature takes an hour.

Vibe coding with Tina4: you describe what you want, the AI generates correct code, you review and deploy. The same feature takes 5 minutes.

The key insight is that AI can only be as good as the framework it targets. A framework with 200 gems, 15 configuration files, and 3 ways to do everything gives the AI too many choices. A framework with zero dependencies, one way to do everything, and a complete reference in CLAUDE.md gives the AI exactly what it needs.

Tina4 was built for this. Every design decision -- zero deps, convention over configuration, identical API across languages, CLAUDE.md -- was made to create the best possible experience for AI-assisted development.

You bring the ideas. The AI brings the implementation. Tina4 bridges the gap.

Exercise: Vibe-Code a Feature

Open your Tina4 Ruby project with an AI assistant (Claude Code, Cursor, or Copilot) and try this prompt:

```
Add a comment system to my blog:
1. Create a comments table with post_id, author_name, body, created_at
2. Create a Comment model with belongs_to :post relationship
3. Add POST /api/posts/{id}/comments to add a comment
4. Add GET /api/posts/{id}/comments to list comments
5. Modify GET /api/posts/{id} to include comments
6. Add RSpec tests for the comment endpoints
```

Verify the AI generates:

• A migration file in `src/migrations/`

• A model file in `src/orm/`

• Route handlers in `src/routes/`

• Test file in `tests/`

Run `tina4 migrate && tina4 test` to verify everything works.

Gotchas

1. AI Generates Rails Syntax Instead of Tina4

Problem: The AI writes `render json:` instead of `response.json`.

Cause: The AI defaulted to Rails conventions.

Fix: Make sure CLAUDE.md is in your project root. Mention "Tina4 Ruby" explicitly in your prompt.

2. AI Invents Non-Existent Methods

Problem: The AI calls `Tina4::Router.resource` which does not exist.

Cause: The AI hallucinated a method from another framework.

Fix: Review generated code against the CLAUDE.md reference. Ask the AI to only use methods documented in CLAUDE.md.

3. AI Generates Tests That Do Not Match the Implementation

Problem: Tests reference endpoints or response formats that differ from the actual routes.

Fix: Generate routes first, then ask the AI to generate tests that match the existing routes.

4. AI Forgets to Run Migrations

Problem: The app crashes because the table does not exist.

Fix: Always run `tina4 migrate` after generating migrations. Include it in your prompts: "...and tell me the commands to run."

5. AI Uses require Instead of Auto-Loading

Problem: The AI adds `require_relative "../orm/product"` at the top of route files.

Cause: Ruby convention is to require files explicitly, but Tina4 auto-loads everything in `src/routes/` and `src/orm/`.

Fix: Remove the require statements. Tina4 handles loading automatically.

6. AI Generates Over-Engineered Solutions

Problem: The AI creates service objects, repository patterns, and dependency injection for a simple CRUD feature.

Fix: Tell the AI: "Keep it simple. Use Tina4's built-in features only. No extra abstractions." Tina4's philosophy is simplicity -- the AI should follow it.

7. AI Does Not Know About to_h

Problem: The AI serializes model objects with `to_json` instead of `to_h`.

Fix: Tina4 ORM objects use `to_h` (Ruby idiom for hash conversion) followed by `response.json`. The CLAUDE.md file documents this pattern.